Code Source Complet du Projet

Généré pour analyse dans l'espace Canvas de Google Studio

■ FICHIER : api\index.ts

```
import express, { Request, Response, NextFunction } from 'express';
import { GoogleGenAI, Modality } from '@google/genai';
import dotenv from 'dotenv';

dotenv.config({ override: true });

function createGenAIClient(): GoogleGenAI {
  return new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
}

function extractErrorMessage(err: unknown): string {
  if (!(err instanceof Error)) return 'Une erreur inconnue est survenue.';
  return err.message || 'Une erreur inconnue est survenue.';
}

interface ChatMessage {
  role: 'user' | 'model';
  parts: { text: string }[];
}

interface ChatRequestBody {
  contents: ChatMessage[];
  systemInstruction?: string;
}

function encodeWAV(pcmBytes: Buffer, sampleRate = 24000): Buffer {
  const numChannels = 1;
  const bitsPerSample = 16;
  const byteRate = sampleRate * numChannels * (bitsPerSample / 8);
  const blockAlign = numChannels * (bitsPerSample / 8);

  const buffer = Buffer.alloc(44 + pcmBytes.length);
  buffer.write('RIFF', 0);
  buffer.writeUInt32LE(36 + pcmBytes.length, 4);
  buffer.write('WAVE', 8);
  buffer.write('fmt ', 12);
  buffer.writeUInt32LE(16, 16);
  buffer.writeUInt16LE(1, 20);
  buffer.writeUInt16LE(numChannels, 22);
  buffer.writeUInt32LE(sampleRate, 24);
  buffer.writeUInt32LE(byteRate, 28);
  buffer.writeUInt16LE(blockAlign, 32);
  buffer.writeUInt16LE(bitsPerSample, 34);
  buffer.write('data', 36);
  buffer.writeUInt32LE(pcmBytes.length, 40);
  pcmBytes.copy(buffer, 44);
  return buffer;
}

const app = express();
app.use(express.json({ limit: '10mb' }));

function validateChatBody(
  req: Request&amp;lt;{}, {}, ChatRequestBody&amp;gt;,
  res: Response,
  next: NextFunction
): void {
  if (!req.body?.contents || !Array.isArray(req.body.contents)) {
    res.status(400).json({ error: '`contents` est requis et doit être un tableau.' });
    return;
  }
  next();
```

```
}

app.post(
  '/api/chat',
  validateChatBody,
  async (req: Request&lt;{}, {}, ChatRequestBody&gt;, res: Response): Promise&lt;void&gt; =&gt; {
    try {
      const { contents, systemInstruction } = req.body;
      const ai = createGenAIClient();

      const formattedContents = contents.map(m =&gt; ({
        role: m.role,
        parts: m.parts.map(p =&gt; ({ text: p.text }))
      }));

      const response = await ai.models.generateContent({
        model: 'gemini-2.5-flash',
        contents: formattedContents,
        config: systemInstruction ? { systemInstruction } : undefined
      });

      res.json({ text: response.text });
    } catch (err) {
      console.error('Chat API Error:', err);
      res.status(500).json({ error: extractErrorMessage(err) });
    }
  }
);

app.get('/api/tts', async (req: Request, res: Response): Promise&lt;void&gt; =&gt; {
  try {
    const { text, lang } = req.query;
    if (!text || typeof text !== 'string') {
      res.status(400).json({ error: '`text` is required.' });
      return;
    }
    if (!process.env.GEMINI_API_KEY) {
      res.status(503).json({ error: 'GEMINI_API_KEY non configurée sur le serveur.' });
      return;
    }

    const ai = createGenAIClient();

    // NETTOYAGE : Suppression des kashidas (■), des points de suspension et des points d'interrogation pour le m
    let cleanText = text.trim()
      .replace(/[\u0640]/g, '') // Enlever le tatweel/kashida de prolongation arabe
      .replace(/[■?\.…_■]+/g, ' ') // Remplacer la ponctuation graphique/trous de texte par des espaces simples
      .replace(/\s+/g, ' ') // Nettoyer les doubles espaces créés
      .trim();

    // Si après nettoyage le texte est vide (ex: juste un point d'interrogation envoyé)
    if (!cleanText) {
      res.status(400).json({ error: "Le texte ne contient aucun caractère prononçable après nettoyage." });
      return;
    }

    const isArabic = (lang === 'ar') || /[\u0600-\u06FF]/.test(cleanText);

    // Répétition 3x pour les lettres courtes afin de fournir assez de matière phonétique au synthétiseur
    const ttsText = (isArabic &amp;&amp; cleanText.length &lt;= 3)
      ? `${cleanText} ${cleanText} ${cleanText}`
      : cleanText;

    // Utilisation stricte de gemini-2.5-flash
    const response = await ai.models.generateContent({
      model: 'gemini-2.5-flash',
      contents: [{ parts: [{ text: ttsText }] }],
      config: {
        responseModalities: [Modality.AUDIO],
        speechConfig: {
          voiceConfig: {
            prebuiltVoiceConfig: { voiceName: 'Charon' },
```

```
          },
        },
        // L'instruction système définit un cadre applicatif strict pour contourner la modération médicale/sensib
        systemInstruction: isArabic
          ? "Tu agis exclusivement comme un moteur automatisé de synthèse vocale (Text-To-Speech) pour une applic
          : "Tu dois lire à voix haute le texte fourni, de manière claire et articulée, sans ajouter de fioriture
      },
    });

    const candidate = response.candidates?.[0];
    const audioPart = candidate?.content?.parts?.find(p =&gt; p.inlineData?.data);
    const base64Audio = audioPart?.inlineData?.data;

    if (!base64Audio) {
      const textResponse = candidate?.content?.parts?.find(p =&gt; p.text)?.text;
      console.warn(`■■ Pas de flux audio généré pour "${cleanText}". Réponse texte :`, textResponse);

      res.status(422).json({
        error: "Le modèle n'a pas retourné de flux audio.",
        details: textResponse
      });
      return;
    }

    const pcmBytes = Buffer.from(base64Audio, 'base64');
    const wavBuffer = encodeWAV(pcmBytes, 24000);

    res.set('Content-Type', 'audio/wav');
    res.set('Cache-Control', 'public, max-age=31536000, immutable');
    res.send(wavBuffer);
  } catch (err) {
    console.error('TTS API Error:', err);
    res.status(500).json({ error: extractErrorMessage(err) });
  }
});

app.post('/api/transcribe', async (req: Request, res: Response): Promise&lt;void&gt; =&gt; {
  try {
    const { audioData, mimeType, expectedLanguage } = req.body;
    if (!audioData || typeof audioData !== 'string') {
      res.status(400).json({ error: 'audioData is required and must be a string' });
      return;
    }

    if (audioData.length &gt; 15 * 1024 * 1024) {
      res.status(413).json({ error: 'audioData base64 payload is too large' });
      return;
    }

    let prompt = 'Please carefully transcribe this audio.';
    if (expectedLanguage) {
      prompt += ` The expected language is ${expectedLanguage}. Only output the exact transcription text, with no
    }

    const ai = createGenAIClient();
    const response = await ai.models.generateContent({
      model: 'gemini-2.5-flash',
      contents: [
        {
          parts: [
            { text: prompt },
            { inlineData: { data: audioData, mimeType: mimeType || 'audio/webm' } }
          ]
        }
      ]
    });

    res.json({ text: (response.text || "").trim() });
  } catch (err) {
    console.error('STT API Error:', err);
    res.status(500).json({ error: extractErrorMessage(err) });
  }
```

```
});

export default app;
```

■ FICHIER : api\tts.ts

```
import { GoogleGenerativeAI } from '@google/generative-ai';

// Sécurité pour vérifier que la clé est bien présente au démarrage
const apiKey = process.env.GEMINI_API_KEY || '';
const genAI = new GoogleGenerativeAI(apiKey);

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const text = searchParams.get('text');

  if (!text) return new Response('Texte manquant', { status: 400 });
  if (!apiKey) return new Response('Clé API Gemini manquante dans le fichier .env', { status: 500 });

  try {
    // Utilisation du modèle de génération standard
    const model = genAI.getGenerativeModel({ model: 'gemini-2.5-flash' });

    // Format d'appel ultra-compatible et simplifié pour le TTS
    const response = await model.generateContent({
      contents: [{
        role: 'user',
        parts: [{ text: `Dis uniquement ce texte en arabe, de manière claire, lente et articulée, sans ajouter de
      }],
      config: {
        responseModalities: ['AUDIO'],
        speechConfig: {
          voiceConfig: {
            prebuiltVoiceConfig: { voiceName: 'Charon' }
          }
        }
      }
    });

    // Extraction sécurisée des données
    const part = response.response.candidates?.[0]?.content?.parts?.[0];

    if (!part || !part.inlineData || !part.inlineData.data) {
      console.error("Format de réponse inattendu de Gemini :", JSON.stringify(response));
      throw new Error("Aucune donnée audio dans la réponse de l'IA.");
    }

    const audioBuffer = Buffer.from(part.inlineData.data, 'base64');
    const mimeType = part.inlineData.mimeType || 'audio/wav';

    return new Response(audioBuffer, {
      headers: {
        'Content-Type': mimeType,
        'Content-Length': audioBuffer.length.toString(),
      },
    });

  } catch (error: any) {
    // Cela permettra de voir le vrai coupable dans le terminal du serveur !
    console.error(' [Erreur Serveur TTS]:', error?.message || error);
    return new Response(`Erreur interne: ${error?.message || error}`, { status: 500 });
  }
}
```

■ FICHIER : capacitor.config.json

```
{
```

```
  "appId": "com.noutq.app",
  "appName": "Noutq",
  "webDir": "dist"
}
```

■ FICHIER : index.html

```
&lt;!doctype html&gt;
&lt;html lang="hy"&gt; &lt;!-- 'hy' pour Arménien ou 'ar' pour Arabe aide les moteurs de recherche --&gt;
  &lt;head&gt;
    &lt;meta charset="UTF-8" /&gt;
    &lt;link rel="preconnect" href="https://fonts.googleapis.com"&gt;
    &lt;link rel="preconnect" href="https://fonts.gstatic.com" crossorigin&gt;
    &lt;meta name="viewport" content="width=device-width, initial-scale=1.0, maximum-scale=5.0" /&gt;
    &lt;meta name="theme-color" content="#16a34a" /&gt;
    &lt;meta name="apple-mobile-web-app-capable" content="yes" /&gt;
    &lt;meta name="apple-mobile-web-app-status-bar-style" content="black-translucent" /&gt;
    &lt;title&gt;Armin02 - Learn Arabic&lt;/title&gt;
  &lt;/head&gt;
  &lt;body&gt;
    &lt;div id="root"&gt;&lt;/div&gt;
    &lt;script type="module" src="/src/main.tsx"&gt;&lt;/script&gt;
  &lt;/body&gt;
&lt;/html&gt;
```

■ FICHIER : metadata.json

```
{
  "name": "Armin03",
  "description": "",
  "requestFramePermissions": [
    "microphone"
  ],
  "majorCapabilities": [
    "MAJOR_CAPABILITY_SERVER_SIDE_GEMINI_API"
  ]
}
```

■ FICHIER : package.json

```
{
  "name": "react-example",
  "private": true,
  "version": "0.0.0",
  "type": "module",
  "scripts": {
    "dev": "tsx server.ts",
    "build": "vite build &amp;&amp; esbuild server.ts --bundle --platform=node --format=cjs --packages=external -
    "preview": "vite preview",
    "clean": "rm -rf dist",
    "lint": "tsc --noEmit",
    "start": "node dist/server.cjs",
    "android:build": "npm run build &amp;&amp; npx cap sync android",
    "android:open": "npx cap open android"
  },
  "dependencies": {
    "@capacitor/android": "8.4.0",
    "@capacitor/core": "8.4.0",
    "@google/genai": "^1.29.0",
    "@google/generative-ai": "^0.24.1",
    "@tailwindcss/vite": "^4.1.14",
    "@vitejs/plugin-react": "^5.0.4",
    "autoprefixer": "^10.4.21",
    "dotenv": "^17.2.3",
```

```
    "esbuild": "^0.28.1",
    "express": "^4.21.2",
    "groq-sdk": "^1.1.2",
    "lucide-react": "^0.546.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "tailwindcss": "^4.1.14",
    "tsx": "^4.21.0",
    "typescript": "~5.8.2",
    "vite": "^6.2.0"
  },
  "devDependencies": {
    "@capacitor/cli": "8.4.0",
    "@types/express": "^4.17.21",
    "@types/node": "^22.14.0"
  }
}
```

■ FICHIER : scripts\generateAll.ts

```
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Importation des leçons
import { lessonsData } from '../src/data/lessons';

// Chemin vers le dossier public
const PUBLIC_AUDIO_DIR = path.resolve(__dirname, '../public/audio');

async function downloadAudio(text: string, filePath: string) {
  const url = `http://localhost:3000/api/tts?text=${encodeURIComponent(text)}&amp;lang=ar`;
  console.log(`■ Téléchargement depuis Gemini : ${text}`);

  try {
    const res = await fetch(url);

    // Si le serveur répond avec un code d'erreur (422, 400, 500, etc.)
    if (!res.ok) {
      console.warn(`■■  [Échec] Impossible de générer l'audio pour "${text}" (Code serveur : ${res.status})`);
      return false; // Signale qu'aucun fichier n'a été créé
    }

    const buffer = await res.arrayBuffer();
    fs.writeFileSync(filePath, Buffer.from(buffer));
    console.log(`■ Enregistré sous : ${filePath}`);
    return true; // Signale un téléchargement réussi
  } catch (err) {
    console.error(`■ Échec réseau critique pour "${text}" :`, err);
    return false;
  }
}

export async function generateAll() {
  if (!fs.existsSync(PUBLIC_AUDIO_DIR)) {
    fs.mkdirSync(PUBLIC_AUDIO_DIR, { recursive: true });
  }

  const allWords = new Set&lt;string&gt;();

  for (const lessonId of Object.keys(lessonsData)) {
    const lesson = lessonsData[lessonId];
    for (const step of lesson.steps) {
      if (step.arabic) allWords.add(step.arabic);
      if (step.pairs) {
        for (const pair of step.pairs) {
```

```
          allWords.add(pair.arabic);
        }
      }
      if (step.options) {
        for (const opt of step.options as any[]) {
          if (opt.arabic) allWords.add(opt.arabic);
        }
      }
    }
  }

  console.log(`■ Trouvé : ${allWords.size} phrases/mots arabes uniques à traiter.`);

  const manifest: Record&lt;string, string&gt; = {};
  const chunks = [];
  const wordsArr = Array.from(allWords);

  for (let i = 0; i &lt; wordsArr.length; i += 10) {
    chunks.push(wordsArr.slice(i, i + 10));
  }

  for (const chunk of chunks) {
    let requestsMade = false;

    await Promise.all(chunk.map(async (word) =&gt; {
      // 1. Nom de fichier standard (Garde les espaces et caractères arabes propres)
      const cleanStandardName = word.replace(/[\/\\\:\*\?\"&lt;&gt;\|]/g, '_').trim();
      const fileNameStandard = `${cleanStandardName}.wav`;
      const filePathStandard = path.join(PUBLIC_AUDIO_DIR, fileNameStandard);

      // 2. Nom de fichier de secours (Ancienne méthode avec uniquement des underscores)
      const safeNameUnderscore = word.replace(/[^a-zA-Z0-9\u0600-\u06FF]/g, '_').substring(0, 50);
      const fileNameUnderscore = `${safeNameUnderscore}.wav`;
      const filePathUnderscore = path.join(PUBLIC_AUDIO_DIR, fileNameUnderscore);

      // --- VERIFICATION DE L'EXISTENCE PHYSIQUE ---
      if (fs.existsSync(filePathStandard)) {
        console.log(`■■  Déjà existant (Standard) : ${word}`);
        manifest[word] = `/audio/${fileNameStandard}`;
        return;
      }

      if (fs.existsSync(filePathUnderscore)) {
        console.log(`■■  Déjà existant (Underscore) : ${word}`);
        manifest[word] = `/audio/${fileNameUnderscore}`;
        return;
      }

      // Si le fichier n'existe nulle part, on tente de le télécharger
      requestsMade = true;
      const success = await downloadAudio(word, filePathStandard);

      if (success) {
        manifest[word] = `/audio/${fileNameStandard}`;
      } else {
        console.error(`■ Non inclus dans le manifeste (Audio manquant) : ${word}`);
      }
    }));

    // On fait la pause uniquement si on a réellement sollicité l'API
    if (requestsMade) {
      console.log("■ Pause de sécurité de 4.5s pour respecter les quotas de Google AI Studio...");
      await new Promise(resolve =&gt; setTimeout(resolve, 4500));
    }
  }

  // Sauvegarde du manifeste
  const manifestPath = path.join(PUBLIC_AUDIO_DIR, 'manifest.json');
  fs.writeFileSync(manifestPath, JSON.stringify(manifest, null, 2));
  console.log(`\n■ Manifeste mis à jour et sauvegardé sous : ${manifestPath}`);
  console.log('■ Génération et compilation de tous les fichiers audio terminée !');
}
```

```
generateAll().catch(console.error);
```

■ FICHIER : scripts\generate_audio.ts

```
import fs from 'fs';
import path from 'path';
import { fileURLToPath } from 'url';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

// Import lessons
// Note: This script assumes it is run with ts-node or tsx from the project root.
import { lessonsData } from '../src/data/lessons';

const PUBLIC_AUDIO_DIR = path.resolve(__dirname, '../public/audio');

async function downloadAudio(text: string, filePath: string) {
  const url = `http://localhost:3000/api/tts?text=${encodeURIComponent(text)}&amp;lang=ar`;
  console.log(`Downloading: ${text}`);

  try {
    const res = await fetch(url);
    if (!res.ok) {
      throw new Error(`Server returned ${res.status} ${res.statusText}`);
    }
    const buffer = await res.arrayBuffer();
    fs.writeFileSync(filePath, Buffer.from(buffer));
    console.log(`■ Saved to ${filePath}`);
  } catch (err) {
    console.error(`■ Failed to download ${text}:`, err);
  }
}

async function generateAll() {
  if (!fs.existsSync(PUBLIC_AUDIO_DIR)) {
    fs.mkdirSync(PUBLIC_AUDIO_DIR, { recursive: true });
  }

  const allWords = new Set&lt;string&gt;();

  for (const lessonId of Object.keys(lessonsData)) {
    const lesson = lessonsData[lessonId];
    for (const step of lesson.steps) {
      if (step.arabic) {
        allWords.add(step.arabic);
      }
      if (step.pairs) {
        for (const pair of step.pairs) {
          allWords.add(pair.arabic);
        }
      }
      if (step.options) {
        for (const opt of step.options) {
          if (opt.arabic) allWords.add(opt.arabic);
        }
      }
    }
  }

  console.log(`Found ${allWords.size} unique Arabic phrases to generate.`);

  const manifest: Record&lt;string, string&gt; = {};

  const chunks = [];
  const wordsArr = Array.from(allWords);
  for (let i = 0; i &lt; wordsArr.length; i += 10) {
    chunks.push(wordsArr.slice(i, i + 10));
  }
```

```
  for (const chunk of chunks) {
    await Promise.all(chunk.map(async (word) =&gt; {
      const safeName = word.replace(/[^a-zA-Z0-9\u0600-\u06FF]/g, '_').substring(0, 50);
      const fileName = `${safeName}.wav`;
      const filePath = path.join(PUBLIC_AUDIO_DIR, fileName);

      manifest[word] = `/audio/${fileName}`;

      if (fs.existsSync(filePath)) {
        console.log(`■■ Skipping (already exists): ${word}`);
        return;
      }

      await downloadAudio(word, filePath);
    }));
    await new Promise(resolve =&gt; setTimeout(resolve, 100)); // small delay between batches
  }

  // Write manifest.json
  const manifestPath = path.join(PUBLIC_AUDIO_DIR, 'manifest.json');
  fs.writeFileSync(manifestPath, JSON.stringify(manifest, null, 2));
  console.log(`■ Saved manifest to ${manifestPath}`);

  console.log('■ Done compiling audio files!');
}

generateAll().catch(console.error);
```

■ FICHIER : server.ts

```
import express, { Request, Response } from 'express';
import { createServer as createViteServer } from 'vite';
import path from 'path';

import apiApp from './api/index.js'; // Note: Node respects .js extension for TS files with ts-node/tsx

// ■■■ Serveur principal ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
async function startServer(): Promise&lt;void&gt; {
  const app = express();
  const PORT = 3000;
  const IS_PROD = process.env.NODE_ENV === 'production';

  // Mount API paths
  app.use(apiApp);

  // ■■ Middleware Vite (dev) ou fichiers statiques (prod) ■■■■■■■■■■■■■■■■■■
  if (!IS_PROD) {
    const vite = await createViteServer({
      server: { middlewareMode: true },
      appType: 'spa',
    });
    app.use(vite.middlewares);
  } else {
    const distPath = path.resolve(process.cwd(), 'dist');
    app.use(express.static(distPath));

    app.get(/^(?!\/api).*/, (_req: Request, res: Response) =&gt; {
      res.sendFile(path.join(distPath, 'index.html'));
    });
  }

  // ■■ Démarrage ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  app.listen(PORT, '0.0.0.0', () =&gt; {
    const env = IS_PROD ? 'production' : 'développement';
    console.log(`■ Serveur ${env} démarré → http://localhost:${PORT}`);
    console.log(`■ Modèle Gemini : gemini-2.5-flash`);
  });
}
```

```
startServer().catch((err) =&gt; {
  console.error('■ Erreur fatale au démarrage :', err);
  process.exit(1);
});
```

■ FICHIER : src\App.tsx

```
/**
 * @license
 * SPDX-License-Identifier: Apache-2.0
 */

import React, { useState, useCallback, useEffect } from 'react';
import HomeScreen from './screens/HomeScreen';
import LessonScreen from './screens/LessonScreen';
import ProfileScreen from './screens/ProfileScreen';
import ChatScreen from './screens/ChatScreen';
import PracticeScreen from './screens/PracticeScreen';
import BottomNav from './components/BottomNav';
import { LanguageProvider } from './contexts/LanguageContext';

// ■■■ Types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

import SpeechSetupScreen from './screens/SpeechSetupScreen';

type Screen = 'home' | 'lesson' | 'profile' | 'chat' | 'practice';

// ■■■ App ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

export default function App() {
  return (
    &lt;LanguageProvider&gt;
      &lt;AppContent /&gt;
    &lt;/LanguageProvider&gt;
  );
}

function AppContent() {
  const [showSetup, setShowSetup] = useState(() =&gt;
    localStorage.getItem('speechSetupDone') !== 'true'
  );

  const [currentScreen, setCurrentScreen] = useState&lt;Screen&gt;('home');
  const [activeLesson, setActiveLesson] = useState&lt;string | null&gt;(null);

  const [completedUnits, setCompletedUnits] = useState&lt;string[]&gt;(() =&gt; {
    try { return JSON.parse(localStorage.getItem('completedUnits') || '[]'); }
    catch { return []; }
  });

  const [totalXP, setTotalXP] = useState&lt;number&gt;(() =&gt; {
    return Number(localStorage.getItem('totalXP') || '0');
  });

  const [streak, setStreak] = useState&lt;number&gt;(() =&gt; {
    const rawStreak = Number(localStorage.getItem('streak') || '0');
    const lastStudy = localStorage.getItem('lastStudyDate');
    if (!lastStudy) return rawStreak;
    const today = new Date().toDateString();
    if (lastStudy === today) return rawStreak;

    const yesterday = new Date();
    yesterday.setDate(yesterday.getDate() - 1);
    if (lastStudy !== yesterday.toDateString()) {
      localStorage.setItem('streak', '0');
      return 0; // reset visually
    }
    return rawStreak;
  });
```

```
useEffect(() =&gt; {
  // any initialization logic that doesn't conflict with setStreak
}, []);

const markUnitComplete = useCallback((lessonId: string, xpEarned: number) =&gt; {
  const today = new Date().toDateString();

  setCompletedUnits(prev =&gt; {
    const alreadyDone = prev.includes(lessonId);
    const next = alreadyDone ? prev : [...prev, lessonId];
    if (!alreadyDone) {
      localStorage.setItem('completedUnits', JSON.stringify(next));
      setTotalXP(xp =&gt; {
        const n = xp + xpEarned;
        localStorage.setItem('totalXP', String(n));
        return n;
      });
    }
    return next;
  });

  const lastStudyDate = localStorage.getItem('lastStudyDate') || '';

  if (lastStudyDate !== today) {
    setStreak(prev =&gt; {
      const yesterday = new Date();
      yesterday.setDate(yesterday.getDate() - 1);
      const newStreak = lastStudyDate === yesterday.toDateString() ? prev + 1 : 1;
      localStorage.setItem('streak', String(newStreak));
      return newStreak;
    });
    localStorage.setItem('lastStudyDate', today);
  }

  const studyHistory: string[] = JSON.parse(
    localStorage.getItem('studyHistory') || '[]'
  );
  if (!studyHistory.includes(today)) {
    studyHistory.push(today);
    localStorage.setItem('studyHistory', JSON.stringify(studyHistory));
  }
}, []);

const navigateToLesson = useCallback((lessonId: string) =&gt; {
  setActiveLesson(lessonId);
  setCurrentScreen('lesson');
}, []);

const goBack = useCallback(() =&gt; {
  setActiveLesson(null);
  setCurrentScreen('home');
}, []);

const handleSetupDone = useCallback(() =&gt; {
  localStorage.setItem('speechSetupDone', 'true');
  setShowSetup(false);
}, []);

const showNav = currentScreen !== 'lesson';

if (showSetup) {
  return (
    &lt;div className="flex flex-col flex-1 h-full w-full bg-gray-50 relative md:rounded-3xl shadow-none md:sha
      &lt;SpeechSetupScreen onDone={handleSetupDone} /&gt;
    &lt;/div&gt;
  );
}

return (
  &lt;div className="flex flex-col md:flex-row flex-1 h-full w-full bg-gray-50 relative rounded-none md:rounded
    {showNav &amp;&amp; (
      &lt;BottomNav currentScreen={currentScreen} setCurrentScreen={setCurrentScreen} /&gt;
```

```
      )}

      &lt;div className="flex-1 w-full flex flex-col h-full bg-white md:border-l border-gray-100 relative"&gt;
        {currentScreen === 'home'    &amp;&amp; &lt;HomeScreen onStartLesson={navigateToLesson} completedUnits={c
        {currentScreen === 'lesson'  &amp;&amp; &lt;LessonScreen onBack={goBack} lessonId={activeLesson} onComple
        {currentScreen === 'profile' &amp;&amp; &lt;ProfileScreen completedUnits={completedUnits} totalXP={totalX
        {currentScreen === 'chat'    &amp;&amp; &lt;ChatScreen /&gt;}
        {currentScreen === 'practice' &amp;&amp; &lt;PracticeScreen /&gt;}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\components\AboutModal.tsx

```
import React from 'react';
import { X, Info, Users } from 'lucide-react';
import { useLanguage } from '../contexts/LanguageContext';

export default function AboutModal({ onClose }: { onClose: () =&gt; void }) {
  const { t, language } = useLanguage();

  return (
    &lt;div className="fixed inset-0 z-50 flex items-center justify-center p-4 bg-black/50 backdrop-blur-sm anima
      {/* Click outside to close (optional, but let's just use the X for now to keep it simple, or cover the whol
      &lt;div className="absolute inset-0" onClick={onClose} /&gt;

      &lt;div
        className="relative bg-white rounded-3xl w-full max-w-md max-h-[85vh] overflow-hidden shadow-2xl flex fle
        dir={language === 'ar' ? 'rtl' : 'ltr'}
      &gt;
        {/* Header */}
        &lt;div className="flex items-center justify-between p-5 border-b border-gray-100 bg-emerald-50"&gt;
          &lt;div className="flex items-center gap-2 text-emerald-700"&gt;
            &lt;Info size={24} /&gt;
            &lt;h2 className="text-xl font-bold"&gt;{t('about.title')}&lt;/h2&gt;
          &lt;/div&gt;
          &lt;button
            onClick={onClose}
            className="p-2 text-emerald-600 hover:bg-emerald-100 rounded-full transition-colors"
          &gt;
            &lt;X size={20} /&gt;
          &lt;/button&gt;
        &lt;/div&gt;

        {/* Content */}
        &lt;div className="p-6 overflow-y-auto space-y-5 text-gray-700 text-sm leading-relaxed"&gt;
          &lt;p&gt;{t('about.p1')}&lt;/p&gt;
          &lt;p&gt;{t('about.p2')}&lt;/p&gt;
          &lt;p&gt;{t('about.p3')}&lt;/p&gt;

          &lt;div className="bg-emerald-50 text-emerald-900 p-4 rounded-xl border border-emerald-100 shadow-sm"&g
            &lt;h3 className="font-bold mb-3"&gt;{t('about.features_title')}&lt;/h3&gt;
            &lt;ul className="list-disc mx-5 space-y-2 marker:text-emerald-500"&gt;
              &lt;li&gt;{t('about.feature1')}&lt;/li&gt;
              &lt;li&gt;{t('about.feature2')}&lt;/li&gt;
              &lt;li&gt;{t('about.feature3')}&lt;/li&gt;
              &lt;li&gt;{t('about.feature4')}&lt;/li&gt;
            &lt;/ul&gt;
          &lt;/div&gt;

          &lt;p className="font-bold text-center text-emerald-600 text-base"&gt;{t('about.goal')}&lt;/p&gt;

          &lt;hr className="border-gray-200" /&gt;

          &lt;div className="space-y-4 pt-2"&gt;
            &lt;div className="flex items-center gap-2 text-blue-600"&gt;
              &lt;Users size={22} /&gt;
              &lt;h3 className="text-lg font-bold"&gt;{t('about.team_title')}&lt;/h3&gt;
```

```
            &lt;/div&gt;
            &lt;p&gt;{t('about.team_desc')}&lt;/p&gt;
            &lt;div className="bg-blue-50 text-blue-800 p-4 rounded-xl border border-blue-100 text-center font-bo
              {t('about.team_members')}
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\components\BottomNav.tsx

```
import React, { useState, useEffect, useCallback, useRef } from 'react';
import {
  Home,
  BookOpen,
  User,
  MessageCircle,
  Sparkles,
  Zap,
  type LucideIcon,
} from 'lucide-react';

import { useLanguage } from '../contexts/LanguageContext';

// ===== Types =====
interface NavItem {
  id: string;
  icon: LucideIcon;
  label: string;
  badge?: number;
  hasNotification?: boolean;
}

interface BottomNavProps {
  currentScreen: string;
  setCurrentScreen: (s: string) =&gt; void;
  unreadMessages?: number;
  dailyStreak?: number;
}

// ===== Ripple Effect Hook =====
function useRipple() {
  const [ripples, setRipples] = useState&lt;Array&lt;{ x: number; y: number; id: number }&gt;&gt;([]);

  const addRipple = useCallback((e: React.MouseEvent&lt;HTMLButtonElement&gt;) =&gt; {
    const rect = e.currentTarget.getBoundingClientRect();
    const x = e.clientX - rect.left;
    const y = e.clientY - rect.top;
    const id = Date.now();
    setRipples((prev) =&gt; [...prev, { x, y, id }]);
    setTimeout(() =&gt; {
      setRipples((prev) =&gt; prev.filter((r) =&gt; r.id !== id));
    }, 600);
  }, []);

  return { ripples, addRipple };
}

// ===== Ripple Component =====
function RippleEffect({ ripples }: { ripples: Array&lt;{ x: number; y: number; id: number }&gt; }) {
  return (
    &lt;&gt;
      {ripples.map((ripple) =&gt; (
        &lt;span
          key={ripple.id}
          className="absolute rounded-full bg-current opacity-20 animate-ripple pointer-events-none"
```

```
          style={{
            left: ripple.x - 20,
            top: ripple.y - 20,
            width: 40,
            height: 40,
          }}
        /&gt;
      ))}
    &lt;/&gt;
  );
}

// ===== Badge Component =====
function NotificationBadge({ count }: { count: number }) {
  if (count &lt;= 0) return null;

  return (
    &lt;span className="absolute -top-1 -right-1 min-w-[18px] h-[18px] bg-red-500 text-white text-[10px] font-bol
      {count &gt; 99 ? '99+' : count}
    &lt;/span&gt;
  );
}

// ===== Dot Indicator =====
function ActiveDot({ active }: { active: boolean }) {
  return (
    &lt;div
      className={`w-1 h-1 rounded-full transition-all duration-500 ${
        active ? 'bg-emerald-500 scale-100 opacity-100' : 'bg-transparent scale-0 opacity-0'
      }`}
    /&gt;
  );
}

// ===== Nav Item Button =====
function NavButton({
  item,
  isActive,
  onClick,
}: {
  key?: React.Key;
  item: NavItem;
  isActive: boolean;
  onClick: (e: React.MouseEvent&lt;HTMLButtonElement&gt;) =&gt; void;
}) {
  const { ripples, addRipple } = useRipple();
  const Icon = item.icon;
  const [bouncing, setBouncing] = useState(false);
  const prevActiveRef = useRef(isActive);

  useEffect(() =&gt; {
    if (isActive &amp;&amp; !prevActiveRef.current) {
      setBouncing(true);
      const timer = setTimeout(() =&gt; setBouncing(false), 400);
      prevActiveRef.current = isActive;
      return () =&gt; clearTimeout(timer);
    }
    prevActiveRef.current = isActive;
  }, [isActive]);

  const handleClick = (e: React.MouseEvent&lt;HTMLButtonElement&gt;) =&gt; {
    addRipple(e);
    onClick(e);

    // Haptic feedback
    if ('vibrate' in navigator) {
      navigator.vibrate(10);
    }
  };

  return (
    &lt;button
```

```
      onClick={handleClick}
      className={`relative flex flex-col md:flex-row md:justify-start md:px-5 md:py-3 items-center justify-center
        isActive
          ? 'text-emerald-600 bg-emerald-50 ml-0 mr-0 md:bg-emerald-50'
          : 'text-gray-400 hover:text-gray-600 hover:bg-gray-50 active:scale-95'
      }`}
      aria-label={item.label}
      aria-current={isActive ? 'page' : undefined}
    &gt;
      {/* Ripple Container */}
      &lt;RippleEffect ripples={ripples} /&gt;

      {/* Background glow for active */}
      {isActive &amp;&amp; (
        &lt;div className="absolute inset-0 bg-emerald-50 rounded-2xl transition-all duration-500 animate-in fade
      )}

      {/* Icon container */}
      &lt;div
        className={`relative z-10 transition-all duration-300 ${
          isActive ? 'scale-110' : 'scale-100 group-hover:scale-105'
        } ${bouncing ? 'animate-bounce-once' : ''}`}
      &gt;
        &lt;Icon
          size={24}
          strokeWidth={isActive ? 2.5 : 2}
          className={`transition-all duration-300 ${
            isActive ? 'drop-shadow-sm' : ''
          }`}
        /&gt;

        {/* Notification badge */}
        {item.badge !== undefined &amp;&amp; item.badge &gt; 0 &amp;&amp; (
          &lt;NotificationBadge count={item.badge} /&gt;
        )}

        {/* Small notification dot */}
        {item.hasNotification &amp;&amp; !item.badge &amp;&amp; (
          &lt;span className="absolute -top-0.5 -right-0.5 w-2.5 h-2.5 bg-red-500 rounded-full border-2 border-wh
        )}
      &lt;/div&gt;

      {/* Label */}
      &lt;span
        className={`relative z-10 text-[10px] md:text-sm md:ml-4 mt-1 md:mt-0 font-semibold transition-all durati
          isActive ? 'text-emerald-700 opacity-100' : 'opacity-70'
        }`}
      &gt;
        {item.label}
      &lt;/span&gt;

      {/* Active dot indicator / line */}
      &lt;div className="relative z-10 mt-0.5 md:hidden"&gt;
        &lt;ActiveDot active={isActive} /&gt;
      &lt;/div&gt;
      {/* Active line indicator on Desktop */}
      {isActive &amp;&amp; (
        &lt;div className="absolute left-0 top-1/2 -translate-y-1/2 w-1.5 h-8 bg-emerald-500 rounded-r-full hidde
      )}
    &lt;/button&gt;
  );
}

// ===== Main Bottom Nav =====
export default function BottomNav({
  currentScreen,
  setCurrentScreen,
  unreadMessages = 0,
  dailyStreak = 0,
}: BottomNavProps) {
  const { t } = useLanguage();
```

```
const navItems: NavItem[] = [
  {
    id: 'home',
    icon: Home,
    label: t('nav.home'),
  },
  {
    id: 'practice',
    icon: BookOpen,
    label: t('nav.practice'),
    hasNotification: true,
  },
  {
    id: 'chat',
    icon: MessageCircle,
    label: t('nav.chat'),
    badge: unreadMessages,
  },
  {
    id: 'profile',
    icon: User,
    label: t('nav.profile'),
  },
];

// Calculate active indicator position
const activeIndex = navItems.findIndex((item) =&gt; item.id === currentScreen);

return (
  &lt;&gt;
    {/* Global styles for animations */}
    &lt;style&gt;{`
      @keyframes ripple {
        0% {
          transform: scale(0);
          opacity: 0.3;
        }
        100% {
          transform: scale(10);
          opacity: 0;
        }
      }
      .animate-ripple {
        animation: ripple 0.6s ease-out forwards;
      }

      @keyframes bounce-once {
        0%, 100% {
          transform: scale(1.1) translateY(0);
        }
        50% {
          transform: scale(1.1) translateY(-4px);
        }
      }
      .animate-bounce-once {
        animation: bounce-once 0.4s ease-in-out;
      }

      @keyframes slide-up-fade {
        from {
          opacity: 0;
          transform: translateY(10px);
        }
        to {
          opacity: 1;
          transform: translateY(0);
        }
      }
      .animate-slide-up {
        animation: slide-up-fade 0.3s ease-out;
      }
```

```
        /* Safe area padding for iOS */
        .pb-safe {
          padding-bottom: max(12px, env(safe-area-inset-bottom));
        }
      `}&lt;/style&gt;

      &lt;nav
        className="md:relative md:w-64 md:h-full md:border-r border-gray-200/60 flex-shrink-0 bg-white/95 md:bg-w
        role="navigation"
        aria-label="Main navigation"
      &gt;
        {/* Streak banner (shown when streak is active) */}
        {dailyStreak &gt;= 3 &amp;&amp; (
          &lt;div className="flex justify-center md:px-5 md:pt-6 mb-1 md:mb-6 animate-slide-up"&gt;
            &lt;div className="bg-gradient-to-r from-orange-500 to-amber-500 text-white text-[10px] md:text-xs fo
              &lt;Zap size={14} /&gt;
              {dailyStreak} {t('nav.streak_banner')} ■
            &lt;/div&gt;
          &lt;/div&gt;
        )}

        {/* Main nav bar */}
        &lt;div className="relative md:h-full md:flex md:flex-col md:px-4 md:gap-2 bg-transparent backdrop-blur-x
          {/* Top sliding indicator line (Mobile only) */}
          &lt;div className="absolute top-0 left-0 right-0 h-[3px] md:hidden"&gt;
            &lt;div
              className="h-full bg-gradient-to-r from-emerald-400 to-teal-500 rounded-full transition-all duratio
              style={{
                width: `${100 / navItems.length}%`,
                marginLeft: `${(activeIndex / navItems.length) * 100}%`,
              }}
            /&gt;
          &lt;/div&gt;

          {/* Nav items */}
          &lt;div className="flex justify-around md:flex-col md:justify-start md:items-start items-center px-2 md
            {navItems.map((item) =&gt; (
              &lt;NavButton
                key={item.id}
                item={item}
                isActive={currentScreen === item.id}
                onClick={() =&gt; setCurrentScreen(item.id)}
              /&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/nav&gt;
    &lt;/&gt;
  );
}
```

■ FICHIER : src\components\VocabularyPractice.tsx

```
import React, { useState, useMemo } from 'react';
import { ArrowLeft, Volume2 } from 'lucide-react';
import { lessonsData } from '../data/lessons';

interface VocabularyPracticeProps {
  onBack: () =&gt; void;
}

export default function VocabularyPractice({ onBack }: VocabularyPracticeProps) {
  const [currentIndex, setCurrentIndex] = useState(0);
  const [isFlipped, setIsFlipped] = useState(false);

  const allVocab = useMemo(() =&gt; {
    const vocabList: { arabic: string; armenian: string; transliteration: string }[] = [];
    Object.values(lessonsData).forEach((lesson) =&gt; {
      lesson.steps.forEach((step) =&gt; {
```

```
      if (step.type === 'listen' &amp;&amp; step.arabic &amp;&amp; step.armenian &amp;&amp; step.arabic.length
        vocabList.push({
          arabic: step.arabic,
          armenian: step.armenian,
          transliteration: step.transliteration || '',
        });
      }
    });
  });
  return vocabList.sort(() =&gt; Math.random() - 0.5);
}, []);

const currentWord = allVocab[currentIndex];

const nextWord = () =&gt; {
  setIsFlipped(false);
  setTimeout(() =&gt; {
    setCurrentIndex((prev) =&gt; (prev + 1) % allVocab.length);
  }, 150);
};

const prevWord = () =&gt; {
  setIsFlipped(false);
  setTimeout(() =&gt; {
    setCurrentIndex((prev) =&gt; (prev - 1 + allVocab.length) % allVocab.length);
  }, 150);
};

if (!currentWord) {
  return (
    &lt;div className="flex-1 flex flex-col items-center justify-center p-6"&gt;
      &lt;p&gt;No vocabulary available.&lt;/p&gt;
      &lt;button onClick={onBack} className="mt-4 px-4 py-2 bg-blue-500 text-white rounded-xl"&gt;Go Back&lt;/b
    &lt;/div&gt;
  );
}

return (
  &lt;div className="flex-1 flex flex-col bg-gray-50 h-full w-full absolute inset-0 z-10 overflow-hidden"&gt;
    {/* Header */}
    &lt;div className="bg-white px-4 pt-6 pb-4 flex items-center shadow-sm"&gt;
      &lt;button onClick={onBack} className="p-2 -ml-2 text-gray-500 hover:text-gray-900 rounded-full hover:bg-
        &lt;ArrowLeft size={24} /&gt;
      &lt;/button&gt;
      &lt;div className="flex-1 text-center"&gt;
        &lt;h2 className="text-lg font-bold text-gray-800"&gt;[redacted] ([redacted])&lt;/h2&gt;
        &lt;p className="text-xs text-gray-500"&gt;{currentIndex + 1} / {allVocab.length}&lt;/p&gt;
      &lt;/div&gt;
      &lt;div className="w-10"&gt;&lt;/div&gt;
    &lt;/div&gt;

    {/* Flashcard Area */}
    &lt;div className="flex-1 flex flex-col items-center justify-center p-6"&gt;
      &lt;div
        className="relative w-full max-w-sm aspect-[4/3] perspective-1000 cursor-pointer"
        onClick={() =&gt; setIsFlipped(!isFlipped)}
      &gt;
        &lt;div className={`w-full h-full transition-transform duration-500 transform-style-3d ${isFlipped ? 'r

          {/* Front — Arabic */}
          &lt;div className="absolute w-full h-full backface-hidden bg-white rounded-3xl shadow-lg border borde
            &lt;span className="text-5xl font-bold text-gray-800 leading-tight text-center" dir="rtl"&gt;
              {currentWord.arabic}
            &lt;/span&gt;
            &lt;p className="text-sm text-gray-400 mt-8"&gt;[redacted] [redacted] [redacted] [redacted] / [redacted] [redacted]&l
          &lt;/div&gt;

          {/* Back — Armenian &amp; Transliteration */}
          &lt;div className="absolute w-full h-full backface-hidden bg-blue-50 rounded-3xl shadow-lg border bor
            &lt;span className="text-3xl font-bold text-blue-900 text-center mb-4"&gt;
              {currentWord.armenian}
            &lt;/span&gt;
```

```
              {currentWord.transliteration &amp;&amp; (
                &lt;span className="text-lg text-blue-700 bg-white px-4 py-2 rounded-full shadow-sm"&gt;
                  {currentWord.transliteration}
                &lt;/span&gt;
              )}
            &lt;/div&gt;

          &lt;/div&gt;
        &lt;/div&gt;

        {/* Controls */}
        &lt;div className="flex items-center justify-center gap-6 mt-12 w-full max-w-sm"&gt;
          &lt;button
            onClick={prevWord}
            className="w-14 h-14 rounded-full bg-white shadow flex items-center justify-center text-gray-600 hove
          &gt;
            ←
          &lt;/button&gt;

          &lt;button
            className="w-16 h-16 rounded-full bg-blue-500 shadow-md flex items-center justify-center text-white h
            onClick={(e) =&gt; {
              e.stopPropagation();
              if ('speechSynthesis' in window) {
                const utterance = new SpeechSynthesisUtterance(currentWord.arabic);
                utterance.lang = 'ar-SA';
                window.speechSynthesis.speak(utterance);
              }
            }}
          &gt;
            &lt;Volume2 size={28} /&gt;
          &lt;/button&gt;

          &lt;button
            onClick={nextWord}
            className="w-14 h-14 rounded-full bg-white shadow flex items-center justify-center text-gray-600 hove
          &gt;
            →
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\contexts\LanguageContext.tsx

```
import React, { createContext, useContext, useState, useEffect } from 'react';

type Language = 'hy' | 'ar';

interface LanguageContextType {
  language: Language;
  setLanguage: (lang: Language) =&gt; void;
  t: (key: string) =&gt; string;
}

const translations: Record&lt;string, Record&lt;Language, string&gt;&gt; = {
  // Bottom Nav
  'nav.home': { hy: '████████', ar: '█████████' },
  'nav.practice': { hy: '████████████', ar: '██████' },
  'nav.chat': { hy: '███████ (AI)', ar: '███████ (AI)' },
  'nav.profile': { hy: '███████', ar: '██████' },
  'nav.streak_banner': { hy: '██ ██████████', ar: '█████ ████████' },

  // Home Screen
  'home.title': { hy: '█████████', ar: '████████' },
  'home.subtitle': { hy: '█████████ A1 · 600 ████', ar: '████████ A1 · 600 █████' },
  'home.unit1.title': { hy: '███████ 1: ██████', ar: '███████ 1: ███████' },
```

```
'home.unit1.subtitle': { hy: '██████ ■ ████████ ██████████', ar: '███████ ████████ ████████' },
'home.unit2.title': { hy: '███████ 2: ███████████████', ar: '███████ 2: ███████' },
'home.unit2.subtitle': { hy: '███████████████ ■ █████ ██████', ar: '███████ ███████ ██████' },
'home.unit3.title': { hy: '███████ 3: ████████', ar: '███████ 3: ████████' },
'home.unit3.subtitle': { hy: '████████ ██████████', ar: '████████' },
'home.unit4.title': { hy: '███████ 4: █████', ar: '███████ 4: ████████ ██████████' },
'home.unit4.subtitle': { hy: '█████████ «█████»', ar: '████████ ██████████ ███████' },
'home.unit5.title': { hy: '███████ 5: █████', ar: '███████ 5: ████████' },
'home.unit5.subtitle': { hy: '█████ 1–10', ar: '████████ ██ 1 ███ 10' },
'home.unit6.title': { hy: '███████ 6: ██████', ar: '███████ 6: ██████ ███████' },
'home.unit6.subtitle': { hy: '█████████ ██████ ■ ████████████', ar: '██████ ███████ ███████' },
'home.unit7.title': { hy: '███████ 7: ███████████████', ar: '███████ 7: ████████' },
'home.unit7.subtitle': { hy: '███████████████ ■ █████ ████████████████████', ar: '████████ █████ ██████' },
'home.unit8.title': { hy: '███████ 8: ████████', ar: '███████ 8: ██████' },
'home.unit8.subtitle': { hy: '████████, █████ ■ ████████', ar: '██████ ██████ ████████ ██████ ██████' },
'home.unit9.title': { hy: '███████ 9: ████████', ar: '███████ 9: ████████' },
'home.unit9.subtitle': { hy: '██████████ ██████████ ██████████', ar: '████████ ██████████' },
'home.unit10.title': { hy: '███████ 10: ████████', ar: '███████ 10: ████████' },
'home.unit10.subtitle': { hy: '█████████ ██████████', ar: '██████ ████████' },
'home.unit11.title': { hy: '███████ 11: ████████', ar: '███████ 11: ███████' },
'home.unit11.subtitle': { hy: '████████████ ■ ████████████', ar: '███████ ████████' },
'home.unit12.title': { hy: '███████ 12: ██████', ar: '███████ 12: ████████' },
'home.unit12.subtitle': { hy: '██████ ■ ███████████', ar: '████████ ██████ █████' },
'home.unit13.title': { hy: '███████ 13: ████████████', ar: '███████ 13: ██████ ██████' },
'home.unit13.subtitle': { hy: '███████ ███████ ██████████', ar: '██████ ██████ ██████████' },
'home.unit14.title': { hy: '███████ 14: ████████████', ar: '███████ 14: ██████' },
'home.unit14.subtitle': { hy: '████████████ ■ ████████████', ar: '██████ ████████' },
'home.unit15.title': { hy: '███████ 15: ██████████', ar: '███████ 15: ██ ████████' },
'home.unit15.subtitle': { hy: '█████████ ██████████', ar: '█████████ ██████████' },
'home.unit16.title': { hy: '███████ 16: ███████ ██', ar: '███████ 16: ██████ ███████' },
'home.unit16.subtitle': { hy: '█████████ ███████ ████████████', ar: '███████ ██████ ██████████' },
'home.unit17.title': { hy: '███████ 17: █████████', ar: '███████ 17: ███████' },
'home.unit17.subtitle': { hy: '█████████ ■ ████', ar: '███████ ██████████' },
'home.unit18.title': { hy: '███████ 18: ████████████████', ar: '███████ 18: ███████████' },
'home.unit18.subtitle': { hy: '████████████████ ██████████', ar: '███████████ ██████████' },
'home.unit19.title': { hy: '███████ 19: █████ ██████', ar: '███████ 19: ████████' },
'home.unit19.subtitle': { hy: '████████ ████████████████████', ar: '█████████ ████████' },
'home.unit20.title': { hy: '███████ 20: ███████████ ██████', ar: '███████ 20: ██████████' },
'home.unit20.subtitle': { hy: '████████████ ███████████ ██████', ar: '███████ ██████ ████████' },
'home.unit21.title': { hy: '███████ 21: ██████ ██████████', ar: '███████ 21: ██████ ████████' },
'home.unit21.subtitle': { hy: '█████████ ██████ ■ ██████ ████████████', ar: '████████ ██████████ ██ ████' },
'home.unit22.title': { hy: '███████ 22: ████████████', ar: '███████ 22: ████████ ███████████' },
'home.unit22.subtitle': { hy: '█████████████ ████████████', ar: '████████ ███████████ ██████' },

'home.completed': { hy: '██████████', ar: '██████' },
'home.start': { hy: '██████ →', ar: '█████ →' },
'home.lessons_completed': { hy: '████ ██████████', ar: '████ ██████' },

'home.node.u1': { hy: '██████', ar: '████████' },
'home.node.u2': { hy: '████████████████', ar: '████████' },
'home.node.u3': { hy: '█████████', ar: '█████████' },
'home.node.u4': { hy: '██████', ar: '█████████' },
'home.node.u5': { hy: '██████', ar: '█████████' },
'home.node.u6': { hy: '██████', ar: '██████' },
'home.node.u7': { hy: '███████████████', ar: '██████' },
'home.node.u8': { hy: '█████████', ar: '██████' },
'home.node.u9': { hy: '█████████', ar: '█████████' },
'home.node.u10': { hy: '█████████', ar: '█████████' },
'home.node.u11': { hy: '█████████', ar: '████████' },
'home.node.u12': { hy: '██████', ar: '█████████' },
'home.node.u13': { hy: '████████', ar: '██████' },
'home.node.u14': { hy: '████████', ar: '██████' },
'home.node.u15': { hy: '██████████', ar: '█████████' },
'home.node.u16': { hy: '███████ ██', ar: '██████' },
'home.node.u17': { hy: '██████████', ar: '████████' },
'home.node.u18': { hy: '████████████████', ar: '███████████' },
'home.node.u19': { hy: '██████ ██████', ar: '█████████' },
'home.node.u20': { hy: '██████', ar: '██████████' },
'home.node.u21': { hy: '██████ ██████████', ar: '█████████' },
'home.node.u22': { hy: '████████████', ar: '█████████' },

// Lesson Screen
```

```
'arabic': { hy: '[redacted]', ar: '[redacted]' },
'armenian': { hy: '[redacted]', ar: '[redacted]' },
'lesson.listen_and_learn': { hy: '[redacted]', ar: '[redacted]' },
'lesson.speak': { hy: '[redacted]', ar: '[redacted]' },
'lesson.quiz': { hy: '[redacted]', ar: '[redacted]' },
'lesson.match': { hy: '[redacted]', ar: '[redacted]' },
'lesson.write': { hy: '[redacted]', ar: '[redacted]' },
'lesson.listen_and_read': { hy: '[redacted]', ar: '[redacted]' },
'lesson.pronounce_sentence': { hy: '[redacted]', ar: '[redacted]' },
'lesson.hide_transliteration': { hy: '[redacted]', ar: '[redacted]' },
'lesson.show_transliteration': { hy: '[redacted]', ar: '[redacted]' },
'lesson.hide_hint': { hy: '[redacted]', ar: '[redacted]' },
'lesson.show_hint': { hy: '[redacted]', ar: '[redacted]' },
'lesson.excellent': { hy: '[redacted]', ar: '[redacted]' },
'lesson.excellent_desc': { hy: '[redacted] +15 XP', ar: '[redacted] +15 XP' },
'lesson.good': { hy: '[redacted]', ar: '[redacted]' },
'lesson.good_desc': { hy: '[redacted] +8 XP', ar: '[redacted] +8 XP' },
'lesson.poor': { hy: '[redacted]', ar: '[redacted]' },
'lesson.poor_desc': { hy: '[redacted]', ar: '[redacted]' },
'lesson.continue': { hy: '[redacted]', ar: '[redacted]' },
'lesson.finish': { hy: '[redacted]', ar: '[redacted]' },
'lesson.skip': { hy: '[redacted] →', ar: '[redacted] →' },
'lesson.quit_title': { hy: '[redacted]', ar: '[redacted]' },
'lesson.quit_desc': { hy: '[redacted]', ar: '[redacted]
'lesson.stay': { hy: '[redacted]', ar: '[redacted]' },
'lesson.quit': { hy: '[redacted]', ar: '[redacted]' },
'lesson.out_of_lives': { hy: '[redacted]', ar: '[redacted]' },
'lesson.out_of_lives_desc': { hy: '[redacted]', ar: '[redacted]' },
'lesson.try_again': { hy: '[redacted]', ar: '[redacted]' },
'lesson.go_back': { hy: '[redacted]', ar: '[redacted]' },
'lesson.lesson_complete': { hy: '[redacted]', ar: '[redacted]' },
'lesson.lesson_complete_desc': { hy: '[redacted]', ar: '[redacted]' },
'lesson.xp_earned': { hy: 'XP [redacted]', ar: 'XP [redacted]' },
'lesson.accuracy': { hy: '[redacted]', ar: '[redacted]' },
'lesson.time': { hy: '[redacted]', ar: '[redacted]' },
'lesson.streak_bonus': { hy: '[redacted]', ar: '[redacted]' },
'lesson.streak_fire': { hy: '[redacted]', ar: '[redacted]' },
'lesson.translate_this': { hy: '[redacted]', ar: '[redacted]' },
'lesson.check': { hy: '[redacted]', ar: '[redacted]' },
'lesson.correct_answer_is': { hy: '[redacted]', ar: '[redacted]' },

// Profile Screen
'profile.stats': { hy: '[redacted]', ar: '[redacted]' },
'profile.streak': { hy: '[redacted]', ar: '[redacted]' },
'profile.lessons_done': { hy: '[redacted]', ar: '[redacted]' },
'profile.achievements': { hy: '[redacted]', ar: '[redacted]' },
'profile.goals': { hy: '[redacted]', ar: '[redacted]' },
'profile.see_all': { hy: '[redacted] →', ar: '[redacted] →' },
'profile.calendar': { hy: '[redacted]', ar: '[redacted]' },
'profile.learned': { hy: '[redacted]', ar: '[redacted]' },
'profile.missed': { hy: '[redacted]', ar: '[redacted]' },
'profile.ach1.title': { hy: '[redacted]', ar: '[redacted]' },
'profile.ach1.desc': { hy: '[redacted]', ar: '[redacted]' },
'profile.ach2.title': { hy: '[redacted]', ar: '[redacted]' },
'profile.ach2.desc': { hy: '7 [redacted]', ar: '[redacted] 7 [redacted]' },
'profile.ach3.title': { hy: '[redacted]', ar: '[redacted]' },
'profile.ach3.desc': { hy: '[redacted] 10 [redacted]', ar: '[redacted] 10 [redacted]' },
'profile.ach4.title': { hy: '[redacted]', ar: '[redacted]' },
'profile.ach4.desc': { hy: '[redacted] 22 [redacted]', ar: '[redacted] [redacted] 22' },
'profile.edit_name': { hy: '[redacted]', ar: '[redacted]' },
'profile.save': { hy: '[redacted]', ar: '[redacted]' },
'profile.language': { hy: '[redacted]', ar: '[redacted]' },
'profile.member_since': { hy: '[redacted] 2026-[redacted]', ar: '[redacted] 2026' },
'profile.bronze_league': { hy: '[redacted]', ar: '[redacted]' },
'profile.month_name': { hy: '[redacted] 2026', ar: '[redacted] 2026' },
'profile.days_month': { hy: '[redacted]', ar: '[redacted]' },
'profile.streak_label': { hy: '[redacted]', ar: '[redacted]' },
'profile.monthly_label': { hy: '[redacted]', ar: '[redacted]' },

// Chat Screen
'chat.title': { hy: 'AI [redacted]', ar: '[redacted]' },
'chat.status': { hy: '[redacted] · [redacted]', ar: '[redacted] · [redacted]' },
```

```
  'chat.clear': { hy: '[redacted]', ar: '[redacted]' },
  'chat.settings': { hy: '[redacted]', ar: '[redacted]' },
  'chat.help': { hy: '[redacted]', ar: '[redacted]' },
  'chat.input_placeholder': { hy: '[redacted]', ar: '[redacted]' },
  'chat.hide_translation': { hy: '[redacted]', ar: '[redacted]' },
  'chat.show_translation': { hy: '[redacted]', ar: '[redacted]' },
  'chat.new_words': { hy: '[redacted]', ar: '[redacted]' },
  'chat.quick_hello': { hy: '[redacted]', ar: '[redacted]' },
  'chat.quick_thanks': { hy: '[redacted]', ar: '[redacted]' },
  'chat.quick_test': { hy: '[redacted]', ar: '[redacted]' },
  'chat.quick_help': { hy: '[redacted]', ar: '[redacted]' },
  'chat.quick_new_words': { hy: '[redacted]', ar: '[redacted]' },
  'chat.quick_correction': { hy: '[redacted]', ar: '[redacted]' },

  // About Modal
  'about.title': { hy: '[redacted]', ar: '[redacted]' },
  'about.p1': {
    hy: '[redacted]',
    ar: '[redacted]',
  },
  'about.p2': {
    hy: '[redacted]',
    ar: '[redacted]',
  },
  'about.p3': {
    hy: '[redacted]',
    ar: '[redacted]',
  },
  'about.features_title': { hy: '[redacted]', ar: '[redacted]' },
  'about.feature1': { hy: '[redacted]', ar: '[redacted]' },
  'about.feature2': { hy: '[redacted]', ar: '[redacted]' },
  'about.feature3': { hy: '[redacted]', ar: '[redacted]' },
  'about.feature4': { hy: '[redacted]', ar: '[redacted]' },
  'about.goal': {
    hy: '[redacted]',
    ar: '[redacted]',
  },
  'about.team_title': { hy: '[redacted]', ar: '[redacted]' },
  'about.team_desc': {
    hy: '[redacted]',
    ar: '[redacted]',
  },
  'about.team_members': {
    hy: '[redacted]',
    ar: '[redacted]',
  },
};

const LanguageContext = createContext&lt;LanguageContextType | undefined&gt;(undefined);

export function LanguageProvider({ children }: { children: React.ReactNode }) {
  const [language, setLanguage] = useState&lt;Language&gt;('hy');

  // Load language from localStorage if available
  useEffect(() =&gt; {
    const savedLang = localStorage.getItem('app_language') as Language;
    if (savedLang &amp;&amp; (savedLang === 'hy' || savedLang === 'ar')) {
      setLanguage(savedLang);
    }
  }, []);

  const handleSetLanguage = (lang: Language) =&gt; {
    setLanguage(lang);
    localStorage.setItem('app_language', lang);
    document.documentElement.dir = lang === 'ar' ? 'rtl' : 'ltr';
  };

  useEffect(() =&gt; {
    document.documentElement.dir = language === 'ar' ? 'rtl' : 'ltr';
  }, [language]);

  const t = (key: string) =&gt; {
```

```
    return translations[key]?.[language] || key;
  };

  return (
    &lt;LanguageContext.Provider value={{ language, setLanguage: handleSetLanguage, t }}&gt;
      &lt;div dir={language === 'ar' ? 'rtl' : 'ltr'} className="w-full h-full"&gt;
        {children}
      &lt;/div&gt;
    &lt;/LanguageContext.Provider&gt;
  );
}

export function useLanguage() {
  const context = useContext(LanguageContext);
  if (!context) {
    throw new Error('useLanguage must be used within a LanguageProvider');
  }
  return context;
}
```

■ FICHIER : src\data\lessons.ts

```
import { LessonData, LessonStep, MatchPair, QuizOption } from './types';

import { u1 } from './lessons/u1';
import { u2 } from './lessons/u2';
import { u3 } from './lessons/u3';
import { u4 } from './lessons/u4';
import { u5 } from './lessons/u5';
import { u6 } from './lessons/u6';
import { u7 } from './lessons/u7';
import { u8 } from './lessons/u8';
import { u9 } from './lessons/u9';
import { u10 } from './lessons/u10';
import { u11 } from './lessons/u11';
import { u12 } from './lessons/u12';
import { u13 } from './lessons/u13';
import { u14 } from './lessons/u14';
import { u15 } from './lessons/u15';
import { u16 } from './lessons/u16';
import { u17 } from './lessons/u17';
import { u18 } from './lessons/u18';
import { u19 } from './lessons/u19';
import { u20 } from './lessons/u20';

import { u21 } from './lessons/u21';
import { u22 } from './lessons/u22';

export const lessonsData: Record&lt;string, LessonData&gt; = {
  u1,
  u2,
  u3,
  u4,
  u5,
  u6,
  u7,
  u8,
  u9,
  u10,
  u11,
  u12,
  u13,
  u14,
  u15,
  u16,
  u17,
  u18,
  u19,
  u20,
```

```
  u21,
  u22,
};

export type { LessonData, LessonStep, MatchPair, QuizOption };
```

■ FICHIER : src\data\lessons\u1.ts

```
import { LessonData, pair } from '../types';

export const u1: LessonData = {
  id: "u1",
  title: "■■■■■■ ■ ■■■■■■■■■■ ■■■■■■■■■",
  titleAr: "■■■■■■■ ■■■■■■■■■■ ■■■■■■■■■■",
  xpReward: 50,
  steps: [
    {
      id: 0,
      type: "listen",
      arabic: "■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■ ■",
      armenian: "■■■■■■■■■■ ■■■■■■■■■■■ — 28 ■■■■",
      transliteration: "alif · b■ · t■ · th■ · j■m · ■■ · kh■ · d■l · dh■l · r■ · zayn · s■n · sh■n · ■■d · ■
      hint: "■■■■■. ■■■. ■■■. ■■■■■ ■■■■ — ■■■■■■ ■■■■■ ■■■■...",
      hintIcon: "■",
    },
    {
      id: 1,
      type: "listen",
      arabic: "■■ / ■■ / ■■",
      armenian: "ba / bi / bu",
      transliteration: "ba / bi / bu",
      hint: "■■■■■ = a  ·  ■■■■■ = i  ·  ■■■■ = u",
      hintIcon: "■",
      highlightChar: "■",
    },
    {
      id: 2,
      type: "quiz",
      arabic: "■■",
      armenian: "■■",
      transliteration: "ba",
      meaning: "■■■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■■■■ (ba):",
      options: [
        { text: "bi", correct: false },
        { text: "bu", correct: false },
        { text: "ba", correct: true },
        { text: "b", correct: false },
      ],
    },
    {
      id: 21,
      type: "quiz",
      arabic: "■■",
      armenian: "■■",
      transliteration: "ba",
      meaning: "■■■■■■■■■■■■■■: ■■■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■■■■ (ba):",
      options: [
        { text: "bi", correct: false },
        { text: "ba", correct: true },
      ],
    },
    {
      id: 3,
      type: "quiz",
      arabic: "■■",
      armenian: "■■",
      transliteration: "bi",
      meaning: "■■■■■■■■■ ■■■■■ ■■■■■■■■■■ ■■■■■■ (bi):",
      options: [
```

```
        { text: "■■", correct: false },
        { text: "■■", correct: true },
        { text: "■■", correct: false },
        { text: "■■", correct: false },
      ],
    },
    {
      id: 31,
      type: "quiz",
      arabic: "■■",
      armenian: "■■",
      transliteration: "bi",
      meaning: "■■■■■■■■■■■■: ■■■■■■■■ ■■■■■ ■■■■■■■■■ ■■■■■■ (bi):",
      options: [
        { text: "■■", correct: false },
        { text: "■■", correct: true },
      ],
    },
    {
      id: 4,
      type: "listen",
      arabic: "■■■■■■■■",
      armenian: "■■■■■■■■■",
      transliteration: "maktab",
      hint: "■ + ■■ + ■■ + ■ = ■■■■■■■■",
      hintIcon: "■",
      highlightChar: "■■■",
    },
    {
      id: 5,
      type: "match",
      arabic: "■■■■ ■■■■■■■■",
      armenian: "■■■■■■■■■ ■■■■■■■■■ ■■■■",
      transliteration: "",
      meaning: "■■■■■■■ ■■■■■■■■■ ■■■■■■ ■■■■ ■■■■■■■■■■■■■■ ■■■■:",
      pairs: [
        pair("■■", "ba"),
        pair("■■", "bi"),
        pair("■■", "bu"),
      ],
    },
    {
      id: 6,
      type: "write",
      arabic: "■■■■■■■■",
      armenian: "■■■■■■■■■",
      transliteration: "maktab",
      meaning: "■■■■■■ 'maktab' ■■■■■ ■■■■■■■■■ ■■■■■■■■■■■■■■■■■■:",
      hint: "■ + ■■ + ■■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u10.ts

```
import { LessonData, pair } from '../types';

export const u10: LessonData = {
  id: "u10",
  title: "■■■■■■■■ (Family)",
  titleAr: "■■■■■■■■",
  xpReward: 60,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "■■■ / ■■■■■ / ■■■ / ■■■■■■ / ■■■■■ / ■■■■■■■",
      armenian: "■■■■■ / ■■■■■ / ■■■■■■■ / ■■■■■■ / ■■■■■■ / ■■■■■■",
      transliteration: "ab / umm / akh / ukht / jadd / jadda",
```

```
  hint: "[redacted]",
  hintIcon: "[redacted]",
},
{
  id: 2,
  type: "listen",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "ab[redacted]",
  hint: "[redacted]",
  hintIcon: "[redacted]",
},
{
  id: 3,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "umm",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 31,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "umm",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: true },
  ],
},
{
  id: 4,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "akh[redacted]",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 41,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "akh[redacted]",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "",
  meaning: "[redacted]",
  pairs: [
    pair("[redacted]", "[redacted]"),
    pair("[redacted]", "[redacted]"),
```

```
        pair("■■■", "■■■■■■■"),
        pair("■■■■■■", "■■■■■■"),
      ],
    },
    {
      id: 6,
      type: "listen",
      arabic: "■■■■■■■■ ■■■■■ ■■■■■■■■■■■■",
      armenian: "■■■■■■ ■■■■ ■■■■■■■ ■ ■■■■■■ ■■■■■■",
      transliteration: "■ind■ akhun wa-ukht■n",
      hint: "■■■■■■■■■ = dual feminine (2 ■■■■■■)",
      hintIcon: "■",
    },
    {
      id: 7,
      type: "quiz",
      arabic: "■■■■■ ■■■■■■ ■■■■■■■■■■",
      armenian: "■■■■■■ ■■■■■■■ ■■■■■■:",
      transliteration: "kam akhan ■indak?",
      meaning: "■■■■■ ■ ■■■■■■■■■■ '■■■■■':",
      options: [
        { text: "■■■■ (what)", correct: false },
        { text: "■■■■■ (how many)", correct: true },
        { text: "■■■■■■ (where)", correct: false },
      ],
    },
    {
      id: 71,
      type: "quiz",
      arabic: "■■■■■ ■■■■■■ ■■■■■■■■■■",
      armenian: "■■■■■■ ■■■■■■■ ■■■■■■:",
      transliteration: "kam akhan ■indak?",
      meaning: "■■■■■■■■■■■■■: ■■■■■ ■ ■■■■■■■■■■ '■■■■■':",
      options: [
        { text: "■■■■ (what)", correct: false },
        { text: "■■■■■ (how many)", correct: true },
      ],
    },
    {
      id: 8,
      type: "speak",
      arabic: "■■■■■■■■ ■■■■■ ■■■■■■■■ ■■■■■■■ ■■■■■■■",
      armenian: "■■■■■■ ■■■■■, ■■■■■ ■ ■■■■ ■■■■■■■:",
      transliteration: "■ind■ abun wa-ummun wa-akhun w■■id",
      meaning: "■■■■■■■■■■■■ ■■■■ ■■■■■■■■■■■■■■■■■:",
      hintIcon: "■■",
    },
    {
      id: 9,
      type: "write",
      arabic: "■■■■■",
      armenian: "■■■■■",
      transliteration: "umm",
      meaning: "■■■■■■ '■■■■■' ■■■■■■■■■:",
      hint: "■ + ■ + ■■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u11.ts

```
import { LessonData, pair } from '../types';

export const u11: LessonData = {
  id: "u11",
  title: "■■■■■■■■ ■ ■■■■■ (Food &amp;amp; Drink)",
  titleAr: "■■■■■■■ ■■■■■■■■",
  xpReward: 60,
  steps: [
```

```
{
  id: 1,
  type: "listen",
  arabic: "██████ / █████ / ██████ / ████████ / ███████ / ███████",
  armenian: "████ / █████ / ████ / ██████ / ██████ / ████",
  transliteration: "khubz / m■■ / laban / tuff■■ / aruzz / daj■j",
  hint: "██████ ■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■",
  hintIcon: "■",
},
{
  id: 2,
  type: "listen",
  arabic: "██████ ███████ — ████████ ████████",
  armenian: "■■ ██████ ■■ — ███████ ■■ ■■■■",
  transliteration: "an■ j■■i■ — ur■du khubzan",
  hint: "███████ = ██████ ■■ · ████████ = ███████ ■■",
  hintIcon: "■",
},
{
  id: 3,
  type: "quiz",
  arabic: "■■■■■",
  armenian: "■■■■■",
  transliteration: "m■■",
  meaning: "■■■■■ ■ ■■■■■■■■■■ '■■■■■':",
  options: [
    { text: "■■■■", correct: false },
    { text: "■■■■■", correct: true },
    { text: "■■■■", correct: false },
    { text: "■■■■■■", correct: false },
  ],
},
{
  id: 31,
  type: "quiz",
  arabic: "■■■■■",
  armenian: "■■■■■",
  transliteration: "m■■",
  meaning: "■■■■■■■■■■■■: ■■■■■ ■ ■■■■■■■■■■ '■■■■■':",
  options: [
    { text: "■■■■", correct: false },
    { text: "■■■■■", correct: true },
  ],
},
{
  id: 4,
  type: "quiz",
  arabic: "████████ ██████ ██████ ■■■■■ ████████",
  armenian: "■■ ██████ ■■■■■ ■■ ████████",
  transliteration: "ur■du k■ba m■■in min fa■lik",
  meaning: "■■■■■ ■ ■■■■■■■■■■ '■■■■■ ████████':",
  options: [
    { text: "■■■■■■■■■■■■■■■■■■ (thank you)", correct: false },
    { text: "■■■■■■■■ ■■ (please)", correct: true },
    { text: "■■■■■■■■ (apology)", correct: false },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "████████",
  armenian: "█████████",
  transliteration: "",
  meaning: "■■■■■■■ ■■■■■■■■■ ■■■■■■■■■ ■■■■■■■■■ ■■■■:",
  pairs: [
    pair("■■■■■■", "■■■■"),
    pair("■■■■■", "■■■■■"),
    pair("■■■■■■■■■", "■■■■■■"),
    pair("████████", "■■■■"),
  ],
},
{
```

```
      id: 6,
      type: "listen",
      arabic: "██████ ██████████ ███████",
      armenian: "███ ████████ ██████ ■",
      transliteration: "h■dh■ ■-■a■■mu ladh■dh",
      hint: "███████ = ██████ · ██████████ = ████████",
      hintIcon: "■",
    },
    {
      id: 7,
      type: "quiz",
      arabic: "████████...",
      armenian: "███████ ■■...",
      transliteration: "ur■du...",
      meaning: "████████ █████ ██████████████████:",
      options: [
        { text: "████████ ██████████", correct: true },
        { text: "██████████ ████████", correct: false },
      ],
    },
    {
      id: 8,
      type: "speak",
      arabic: "██████ ███████ ██████████ ████████ ███████",
      armenian: "■■ ██████ ■■ ■ ■■■ ■■ ■■■■■ ■■ ███████:",
      transliteration: "an■ j■■i■ wa-ur■du khubzan wa-m■■",
      meaning: "████████████ ■■■■ ██████████████████:",
      hintIcon: "■■",
    },
    {
      id: 9,
      type: "write",
      arabic: "██████",
      armenian: "■■■■",
      transliteration: "khubz",
      meaning: "██████ '■■■' ■■■■■■■■:",
      hint: "■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u12.ts

```
import { LessonData, pair } from '../types';

export const u12: LessonData = {
  id: "u12",
  title: "██████ ■ ██████████ (Places)",
  titleAr: "████████ ██████ ■■■■■",
  xpReward: 70,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "■■■■ / ██████ / ███████ / ████████ / ███████ / ██████████",
      armenian: "■■■■ / ■■■■ / ■■■■■ / ■■■■■ / ■■■■■ / ■■■■■■■",
      transliteration: "f■ / ■al■ / bayna / am■ma / khalfa / bi-j■nibi",
      hint: "██████████ ██████████",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "██████████ ██████ ██████████",
      armenian: "██████ ██████████ ■■■■ ■",
      transliteration: "al-kit■bu ■al■ l-maktabi",
      hint: "██████ = ■■■■ · ██████████ = ██████████",
      hintIcon: "■",
    },
```

```
{
  id: 3,
  type: "quiz",
  arabic: "██████████ ████ ████████████",
  armenian: "██████ ████████████ ■ (████)",
  transliteration: "al-kit■bu f■ l-■aq■ba",
  meaning: "█████ ■ ██████████ '████':",
  options: [
    { text: "████ (on)", correct: false },
    { text: "████ (in/inside)", correct: true },
    { text: "█████ (in front of)", correct: false },
  ],
},
{
  id: 31,
  type: "quiz",
  arabic: "██████████ ████ ████████████",
  armenian: "██████ ████████████ ■ (████)",
  transliteration: "al-kit■bu f■ l-■aq■ba",
  meaning: "████████████: █████ ■ ██████████ '████':",
  options: [
    { text: "████ (on)", correct: false },
    { text: "████ (in/inside)", correct: true },
  ],
},
{
  id: 4,
  type: "quiz",
  arabic: "████████████ ████████ ████████",
  armenian: "███████ ████ █████ ■",
  transliteration: "al-masjidu am■ma l-bayt",
  meaning: "███████ ■ ███████:",
  options: [
    { text: "█████ █████ (█████)", correct: true },
    { text: "████ ████", correct: false },
    { text: "████ ████████", correct: false },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "█████ █████",
  armenian: "██████████",
  transliteration: "",
  meaning: "███████ ████████ ████ ███████ ████:",
  pairs: [
    pair("████", "████ (in)"),
    pair("██████", "████ (on)"),
    pair("█████████", "█████ (in front)"),
    pair("███████", "█████ (behind)"),
  ],
},
{
  id: 6,
  type: "listen",
  arabic: "███████ ██████████ — ██████████ ███████ █████████ ███████████████",
  armenian: "███████ ■ ██████: — ██████ ██████ ■ ██████ █████ ■:",
  transliteration: "ayna l-kit■b? — al-kit■bu bayna l-qalami wa-d-daftar",
  hint: "███████ = ██████ · ███████ = █████",
  hintIcon: "■",
},
{
  id: 7,
  type: "quiz",
  arabic: "███████ ███████████████",
  armenian: "███████ ■ ███████:",
  transliteration: "ayna l-madrasa?",
  meaning: "█████ ■ ██████████ '███████':",
  options: [
    { text: "████ (what)", correct: false },
    { text: "██████ (where)", correct: true },
    { text: "████ (when)", correct: false },
```

```
      ],
    },
    {
      id: 8,
      type: "speak",
      arabic: "██████████ ██████ ████████████ ████████████ ████ ██████████████",
      armenian: "██████ ██████████ ████ ■, ██████ ██████████████ ■:",
      transliteration: "al-kit■bu ■al■ l-maktabi wa-l-qalamu f■ l-■aq■ba",
      meaning: "████████████ ████ ██████████████████:",
      hintIcon: "■■",
    },
    {
      id: 9,
      type: "write",
      arabic: "██████",
      armenian: "████ (on)",
      transliteration: "■al■",
      meaning: "██████ '████' ██████████:",
      hint: "■ + ■ + ■",
    },
    {
      id: 10,
      type: "match",
      arabic: "████████",
      armenian: "██████",
      transliteration: "",
      meaning: "███████ █████ — ███████ ███████:",
      pairs: [
        pair("████████████", "██████"),
        pair("██████████", "█████████"),
        pair("████████", "█████"),
        pair("██████████", "██████"),
      ],
    },
  ],
};
```

■ FICHIER : src\data\lessons\u13.ts

```
import { LessonData, pair } from '../types';

export const u13: LessonData = {
  id: "u13",
  title: "███████ ██████ (Marmni maser)",
  titleAr: "██████ ██████",
  xpReward: 60,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "██████ / ███ / ██████ / ██████ / ██████ / ███",
      armenian: "██████ / █████ / ███ / ███ / ██████ / ██████",
      transliteration: "ra■s / yad / rijl / ■ayn / udhun / fam",
      hint: "███████ ████████ ███████",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "████████ ████████████",
      armenian: "███████ ███████ ■",
      transliteration: "ra■s■ yu■limu-n■",
      hint: "████████████ = ███████ ■ ■■ ...-■",
      hintIcon: "■",
    },
    {
      id: 3,
      type: "quiz",
      arabic: "███",
```

```
  armenian: "[redacted]",
  transliteration: "yad",
  meaning: "[redacted] [redacted] '[redacted]' [redacted] [redacted]:",
  options: [
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 31,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "yad",
  meaning: "[redacted]: [redacted] [redacted] '[redacted]':",
  options: [
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: true },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "[redacted] [redacted]",
  armenian: "[redacted] [redacted]",
  transliteration: "",
  meaning: "[redacted] [redacted] [redacted]:",
  pairs: [
    pair("[redacted]", "[redacted]"),
    pair("[redacted]", "[redacted]"),
    pair("[redacted]", "[redacted]"),
    pair("[redacted]", "[redacted]"),
  ],
},
{
  id: 6,
  type: "listen",
  arabic: "[redacted] [redacted] [redacted] [redacted]",
  armenian: "[redacted] [redacted] [redacted] [redacted]",
  transliteration: "[redacted]ind[redacted] alamun f[redacted] [redacted]ahr[redacted]",
  hint: "[redacted] = [redacted] · [redacted] = [redacted]",
  hintIcon: "[redacted]",
},
{
  id: 7,
  type: "quiz",
  arabic: "[redacted] [redacted]",
  armenian: "[redacted] [redacted] [redacted]:",
  transliteration: "ayna tu[redacted]limuk?",
  meaning: "[redacted] [redacted] [redacted] '[redacted] [redacted]':",
  options: [
    { text: "[redacted] [redacted] [redacted]:", correct: true },
    { text: "[redacted] [redacted]:", correct: false },
    { text: "[redacted] [redacted] [redacted]:", correct: false },
  ],
},
{
  id: 8,
  type: "speak",
  arabic: "[redacted] [redacted]",
  armenian: "[redacted] [redacted] [redacted]:",
  transliteration: "ra[redacted]s[redacted] yu[redacted]limu-n[redacted]",
  meaning: "[redacted] [redacted]:",
  hintIcon: "[redacted]",
},
{
  id: 9,
  type: "write",
  arabic: "[redacted]",
  armenian: "[redacted]",
```

```
      transliteration: "yad",
      meaning: "██████ '█████' ██████████:",
      hint: "■ + ■■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u14.ts

```
import { LessonData, pair } from '../types';

export const u14: LessonData = {
  id: "u14",
  title: "███████ (Yeghanak)",
  titleAr: "██████",
  xpReward: 60,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "████████ / ███████ / ██████ / ██████ / █████ / ██████ / ███████",
      armenian: "██████ / ██████ / ██████ / ██████ / █████ / ████ / ██████",
      transliteration: "mushmis / gh■■im / ma■ar / thalj / r■■ / ■■rr / b■rid",
      hint: "████████████ ████████████████████",
      hintIcon: "■■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "████████ ████████ ████████",
      armenian: "██████ ████████ ██████ ■",
      transliteration: "al-jawwu l-yawma mushmis",
      hint: "███████ = ███████ · ████████ = ██████",
      hintIcon: "■■",
    },
    {
      id: 3,
      type: "quiz",
      arabic: "██████",
      armenian: "██████",
      transliteration: "ma■ar",
      meaning: "█████ ■ '██████' ███████:",
      options: [
        { text: "██████", correct: false },
        { text: "██████", correct: true },
        { text: "██████", correct: false },
        { text: "█████", correct: false },
      ],
    },
    {
      id: 31,
      type: "quiz",
      arabic: "██████",
      armenian: "██████",
      transliteration: "ma■ar",
      meaning: "████████████: █████ ■ '██████' ███████:",
      options: [
        { text: "██████", correct: false },
        { text: "██████", correct: true },
      ],
    },
    {
      id: 5,
      type: "match",
      arabic: "██████",
      armenian: "███████",
      transliteration: "",
      meaning: "███████ ████████████ ████████████:",
      pairs: [
```

```
        pair("████████", "██████"),
        pair("██████", "██████"),
        pair("██████", "██████"),
        pair("███████", "██████"),
      ],
    },
    {
      id: 6,
      type: "listen",
      arabic: "███████ ████████ ███████ — ██████████ ██████████",
      armenian: "██████████ ■ ████████ █████: — ██████ ██████:",
      transliteration: "kayfa l-jawwu ghadan? — sayak■nu b■ridan",
      hint: "██████████ = ██████ (██████)",
      hintIcon: "■■",
    },
    {
      id: 7,
      type: "quiz",
      arabic: "██████████ ████████",
      armenian: "████ ██████",
      transliteration: "sayak■nu ■■rran",
      meaning: "█████ ■ '██████████ ████████' ███████:",
      options: [
        { text: "████ ██████", correct: true },
        { text: "██████ ██████", correct: false },
        { text: "██████ ██████", correct: false },
      ],
    },
    {
      id: 8,
      type: "speak",
      arabic: "████████ ████████ ███████ ██████████",
      armenian: "██████ ████████ ████████ ■ ■ ██████:",
      transliteration: "al-jawwu l-yawma gh■■im wa-b■rid",
      meaning: "████████ ██████████████████:",
      hintIcon: "■■",
    },
    {
      id: 9,
      type: "write",
      arabic: "██████",
      armenian: "██████",
      transliteration: "ma■ar",
      meaning: "██████ '██████' ██████████:",
      hint: "■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u15.ts

```
import { LessonData, pair } from '../types';

export const u15: LessonData = {
  id: "u15",
  title: "██████████ (Dprotsum)",
  titleAr: "■■ ████████",
  xpReward: 60,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "██████████ / ██████ / ███████ / ██████ / ██████████ / ██████████ / ██████████",
      armenian: "██████ / ████████ / █████ / █████ / ██████████ / ██████████ / ████████",
      transliteration: "madrasa / fa■l / kit■b / qalam / sabb■ra / mu■allim / tilm■dh",
      hint: "██████████ ██████████",
      hintIcon: "■",
    },
    {
```

```
  id: 2,
  type: "listen",
  arabic: "████████████ ██████████ ██████████",
  armenian: "████████████ ████████████ ■ ██████",
  transliteration: "al-mu■allimu yashra■u d-darsa",
  hint: "██████████ = ████████████ ■ · ██████████ = ██████",
  hintIcon: "████",
},
{
  id: 3,
  type: "quiz",
  arabic: "██████████",
  armenian: "██████████",
  transliteration: "mu■allim",
  meaning: "██████ ■ '██████████' ████████:",
  options: [
    { text: "█████████", correct: false },
    { text: "██████████", correct: true },
    { text: "██████", correct: false },
    { text: "███████", correct: false },
  ],
},
{
  id: 31,
  type: "quiz",
  arabic: "██████████",
  armenian: "██████████",
  transliteration: "mu■allim",
  meaning: "████████████: ██████ ■ '██████████':",
  options: [
    { text: "█████████", correct: false },
    { text: "██████████", correct: true },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "█████████",
  armenian: "███████",
  transliteration: "",
  meaning: "████████ ████████████ ████████████:",
  pairs: [
    pair("████████████", "███████"),
    pair("████████", "██████"),
    pair("███████", "██████"),
    pair("██████████", "█████████"),
  ],
},
{
  id: 6,
  type: "listen",
  arabic: "█████████ ████████████████████",
  armenian: "███ ████████ ███ ████████████",
  transliteration: "u■ibbu r-riy■■iyy■t",
  hint: "█████████ = ████████ ■■ · ████████████████████ = ████████████",
  hintIcon: "■",
},
{
  id: 7,
  type: "quiz",
  arabic: "███████ ██████████ █████████",
  armenian: "████ █████████ ■■ ████████:",
  transliteration: "ayyu m■ddatin tu■ibb?",
  meaning: "██████ ■ '███████ ██████████' ████████:",
  options: [
    { text: "████ █████████", correct: true },
    { text: "████████ ■", correct: false },
    { text: "███████", correct: false },
  ],
},
{
  id: 8,
```

```
      type: "speak",
      arabic: "██████ ██████████ ██████████ ██████████████████",
      armenian: "██ ████████ ██ ▄ ███████ ██ ████████████",
      transliteration: "an■ tilm■dhun wa-u■ibbu r-riy■■iyy■t",
      meaning: "█████████ ██████████████████:",
      hintIcon: "■■",
    },
    {
      id: 9,
      type: "write",
      arabic: "██████",
      armenian: "█████",
      transliteration: "qalam",
      meaning: "██████ '█████' ██████████:",
      hint: "■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u16.ts

```
import { LessonData, pair } from '../types';

export const u16: LessonData = {
  id: "u16",
  title: "███████ ■■ (Khosum em)",
  titleAr: "███████ ██████",
  xpReward: 80,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "█████████ — ████████ ████████████",
      armenian: "█████ — ██████ ████████",
      transliteration: "ahlan — ahlan wa-sahlan",
      hint: "████████████████ ██████████████",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "██████ ███████ ... ████████ ██████ ████████████████ ████████████ ████████████████",
      armenian: "■■ ████████ ■ ... ■ ■■ ████████████ ■■ ■ ██████████ ■■ ███████████",
      transliteration: "an■ ism■ … wa-an■ min Arm■niy■ wa-adrusu l-■arabiyya",
      hint: "████████████ ██████████████████ ████████████ ███████",
      hintIcon: "■■",
    },
    {
      id: 3,
      type: "quiz",
      arabic: "███████ ████████ '■■ ■■■■ ■■' ██████████████████",
      armenian: "███████ ■■ ███████ '■■ ■■■■ ■■' ██████████:",
      transliteration: "kayfa taq■lu 'yes hay em' bi-l-■arabiyya?",
      meaning: "█████████ ■ '■■ ■■■■ ■■' ██████████:",
      options: [
        { text: "███████ ██████████████", correct: true },
        { text: "███████ ████████████", correct: false },
        { text: "███████ ███████████", correct: false },
      ],
    },
    {
      id: 31,
      type: "quiz",
      arabic: "████████ ████████ '■■ ■■■■ ■■' ██████████████████",
      armenian: "████████ ■■ ███████ '■■ ■■■■ ■■' ██████████:",
      transliteration: "kayfa taq■lu 'yes hay em' bi-l-■arabiyya?",
      meaning: "██████████████: ████████ ■ '■■ ■■■■ ■■' ██████████:",
      options: [
        { text: "███████ ██████████████", correct: true },
```

```
        { text: "██████ ██████████", correct: false },
      ],
    },
    {
      id: 5,
      type: "match",
      arabic: "███████",
      armenian: "████████████████",
      transliteration: "",
      meaning: "███████ ████████████████████ ██████ ██████████ ███:",
      pairs: [
        pair("████████", "█████"),
        pair("█████ ███████ ████████", "██████████ ██"),
        pair("██████████", "██████████ ██"),
        pair("████████", "████████████████████"),
      ],
    },
    {
      id: 6,
      type: "listen",
      arabic: "█████ ████████████ ████████████████ — ███████ ██████████",
      armenian: "██████████ ████████ ██: — ████, ██ ██████",
      transliteration: "hal tatakallamu l-■arabiyya? — na■am, qal■lan",
      hint: "██████████ = ██ ██████ · ███████ = ████",
      hintIcon: "██",
    },
    {
      id: 7,
      type: "quiz",
      arabic: "█████ ████████████ ████████████████",
      armenian: "██████████ ████████ ██:",
      transliteration: "hal tatakallamu l-■arabiyya?",
      meaning: "██████ ■ ███████████ ████ ███████:",
      options: [
        { text: "██████████ ████████ ██:", correct: true },
        { text: "████████ ██:", correct: false },
        { text: "████████████ ██ ██████████:", correct: false },
      ],
    },
    {
      id: 8,
      type: "speak",
      arabic: "██████████ ███████ ████████ ... ████████ ██████ ████████████████",
      armenian: "██████, ██ ████████ ■ ... ■ ██ ████████████ ██",
      transliteration: "ahlan, an■ ism■ … wa-an■ min Arm■niy■",
      meaning: "████████████ ██████████ ■ ████████████:",
      hintIcon: "██",
    },
    {
      id: 9,
      type: "speak",
      arabic: "█████ ████████████ ████████████████ — ████████ ████████████ ██████████",
      armenian: "██████████ ████████ ██: — ████, ██ ██████, ██████████ ██",
      transliteration: "hal tatakallamu l-■arabiyya? — na■am, qal■lan, adrusu",
      meaning: "████████ ████████████████████:",
      hintIcon: "██",
    },
    {
      id: 10,
      type: "write",
      arabic: "███████",
      armenian: "████",
      transliteration: "na■am",
      meaning: "███████ '████' ██████████:",
      hint: "■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u17.ts

```
import { LessonData, pair } from '../types';

export const u17: LessonData = {
  id: "u17",
  title: "■■■■■■■■■",
  titleAr: "■■■■■■■",
  xpReward: 65,
  steps: [
    {
      id: 1, type: "listen",
      arabic: "■■■■■ / ■■■■■■■■ / ■■■■■■ / ■■■■■■ / ■■■■■■■ / ■■■■■■■■",
      armenian: "■■■■■■ / ■■■■■■■ / ■■■■ / ■■■■■ / ■■■■■ / ■■■■■■■ ■■",
      transliteration: "s■q / dukk■n / thaman / gh■lin / rakh■■ / ur■du",
      hint: "■■■■■■■■■■ ■■■■■■■■■ ■■■■■■", hintIcon: "■",
    },
    {
      id: 2, type: "listen",
      arabic: "■■■■■■■ ■■■■■■■ — ■■■■■■ ■■■■■■■■■■■ ■■■■■■■■■",
      armenian: "■■■■■■■■ ■■■■■ ■■: — ■■ ■■■■■ ■■■■■■■ ■:",
      transliteration: "bikam h■dh■? — h■dh■ bi-■asharat dar■him",
      hint: "■■■■■■■ = ■■■■■■■■ (how much) · ■■■■■■■■■ = ■■■■■■■■■■", hintIcon: "■",
    },
    {
      id: 3, type: "quiz",
      arabic: "■■■■■■", armenian: "■■■■■",
      transliteration: "gh■lin",
      meaning: "■■■■■ ■ '■■■■■■' ■■■■■■■:",
      options: [
        { text: "■■■■■", correct: false },
        { text: "■■■■■", correct: true },
        { text: "■■■■■■", correct: false },
        { text: "■■■■", correct: false },
      ],
    },
    {
      id: 31, type: "quiz",
      arabic: "■■■■■■", armenian: "■■■■■", transliteration: "gh■lin",
      meaning: "■■■■■■■■■■■■■: ■■■■■ ■ '■■■■■■':",
      options: [{ text: "■■■■■", correct: false }, { text: "■■■■■", correct: true }],
    },
    {
      id: 4, type: "listen",
      arabic: "■■■■■■■■ ■■■■■ ■■■■■■■■■■■ ■■■■■■",
      armenian: "■■■■■■■ ■■ ■■ ■■■■■",
      transliteration: "ur■du an ashtariya h■dh■",
      hint: "■■■■■■■■■■■■ = ■■■■■ (to buy)", hintIcon: "■■",
    },
    {
      id: 5, type: "match",
      arabic: "■■■■■■■", armenian: "■■■■■■■■■", transliteration: "",
      meaning: "■■■■■■ ■■■■■■■■■■ ■■■■■■■:",
      pairs: [
        pair("■■■■■", "■■■■■■"),
        pair("■■■■■■", "■■■■■"),
        pair("■■■■■■■", "■■■■■"),
        pair("■■■■■■■■", "■■■■■■■ ■■"),
      ],
    },
    {
      id: 6, type: "quiz",
      arabic: "■■■■■■■ ■■■■■■■", armenian: "■■■■■■■■ ■■■■■ ■■:",
      transliteration: "bikam h■dh■?",
      meaning: "■■■■■ ■ '■■■■■■■ ■■■■■■■' ■■■■■■■:",
      options: [
        { text: "■■■■■■■■ ■■■■■ ■■: ✓", correct: true },
        { text: "■■■■■ ■ ■■: ✗", correct: false },
        { text: "■■■■■■■ ■ ■■: ✗", correct: false },
      ],
    },
```

```
    {
      id: 7, type: "speak",
      arabic: "■■■■■■■ ■■■■■■■ — ■■■■■■ ■■■■■■",
      armenian: "■■■■■■■■ ■■■■■ ■■: — ■■ ■■■■■ ■",
      transliteration: "bikam h■dh■? — h■dh■ gh■lin",
      meaning: "■■■■■■■ ■■■■■■■■■■■■■■■■:", hintIcon: "■■",
    },
    {
      id: 8, type: "write",
      arabic: "■■■■■■", armenian: "■■■■■",
      transliteration: "gh■lin",
      meaning: "■■■■■■ '■■■■■' ■■■■■■■■■:",
      hint: "■ + ■ + ■■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u18.ts

```
import { LessonData, pair } from '../types';

export const u18: LessonData = {
  id: "u18",
  title: "■■■■■■■■■■■■■■■■",
  titleAr: "■■■■■■■■■■",
  xpReward: 65,
  steps: [
    {
      id: 1, type: "listen",
      arabic: "■■■■■■■ / ■■■■■■■ / ■■■■■■■■ / ■■■■■■■ / ■■■■■■■■■■■■ / ■■■■■■■■■",
      armenian: "■■ / ■■■■ / ■■■■■ / ■■■■■ / ■■■■■■ / ■■■■",
      transliteration: "yam■n / yas■r / am■ma / khalfa / mustaq■m / idhhab",
      hint: "■■■■■■■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■■■■", hintIcon: "■",
    },
    {
      id: 2, type: "listen",
      arabic: "■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■",
      armenian: "■■■■ ■■■, ■■■■ ■■■■",
      transliteration: "idhhab yam■nan thumma yas■ran",
      hint: "■■■■■■■■■■ = ■■■■ (go!) · ■■■■■■ = ■■■■ (then)", hintIcon: "■■",
    },
    {
      id: 3, type: "quiz",
      arabic: "■■■■■■■■", armenian: "■■■",
      transliteration: "yam■n",
      meaning: "■■■■■ ■ '■■■■■■■' ■■■■■■■:",
      options: [
        { text: "■■■■ ✗", correct: false },
        { text: "■■■ ✓", correct: true },
        { text: "■■■■■ ✗", correct: false },
        { text: "■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 31, type: "quiz",
      arabic: "■■■■■■■■", armenian: "■■■", transliteration: "yam■n",
      meaning: "■■■■■■■■■■■■■: ■■■■■ ■ '■■■■■■■■':",
      options: [{ text: "■■■■", correct: false }, { text: "■■■", correct: true }],
    },
    {
      id: 4, type: "listen",
      arabic: "■■■■■■■ ■■■■■■■■■■■■■■ — ■■■■■■■ ■■■■■■■■■■■■■■■",
      armenian: "■■■■■■■ ■ ■■■■■■■■■■: — ■■■■ ■■■■■■",
      transliteration: "ayna l-ma■a■■a? — imshi mustaq■man",
      hint: "■■■■■■■■■■■■ = ■■■■■■■■■ · ■■■■■■■ = ■■■■■■■/■■■■ (walk!)", hintIcon: "■",
    },
    {
      id: 5, type: "match",
      arabic: "■■■■■■■■■■■", armenian: "■■■■■■■■■■■■■■■■", transliteration: "",
```

```
      meaning: "██████ ████████████████:",
      pairs: [
        pair("███████", "██"),
        pair("███████", "████"),
        pair("████████", "█████"),
        pair("███████", "█████"),
      ],
    },
    {
      id: 6, type: "quiz",
      arabic: "█████████", armenian: "████",
      transliteration: "idhhab",
      meaning: "█████ ■ '█████████' ███████:",
      options: [
        { text: "████ ✓", correct: true },
        { text: "████ ✗", correct: false },
        { text: "████████ ✗", correct: false },
      ],
    },
    {
      id: 7, type: "speak",
      arabic: "█████████ ████████ ██████ ██████████████",
      armenian: "████ ■■, ████ ██████",
      transliteration: "idhhab yam■nan thumma mustaq■man",
      meaning: "███████ ████████████████:", hintIcon: "■■",
    },
    {
      id: 8, type: "write",
      arabic: "███████", armenian: "■■",
      transliteration: "yam■n",
      meaning: "██████ '■■' █████████:",
      hint: "■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u19.ts

```
import { LessonData, pair } from '../types';

export const u19: LessonData = {
  id: "u19",
  title: "█████ ██████",
  titleAr: "████████ ███████",
  xpReward: 65,
  steps: [
    {
      id: 1, type: "listen",
      arabic: "██████████████ / ████████ / ████████ / ████████ / ████████",
      armenian: "██████████ ■■ / ██████████ ■■ / ████████ ■■ / ████████ ■■ / ██████ ■■",
      transliteration: "astayqi■u / af■iru / a■malu / adrusu / an■mu",
      hint: "███████ ████████████████ ██████", hintIcon: "■",
    },
    {
      id: 2, type: "listen",
      arabic: "██████████████ ██████████ ████████████ ████████",
      armenian: "████████ ■■ ████ ██████ ██████████",
      transliteration: "astayqi■u s-s■■ata s-s■bi■a ■ab■■an",
      hint: "██████████ ████████████ = ████ ██████ (7 o'clock) · ████████ = ██████████", hintIcon: "■",
    },
    {
      id: 3, type: "listen",
      arabic: "█████████",
      armenian: "■■ ████████ ■■",
      transliteration: "a■malu",
      hint: "█████████ = I work · ███████ ███████", hintIcon: "■",
    },
    {
      id: 4, type: "quiz",
```

```
      arabic: "████████", armenian: "██████ ██",
      transliteration: "an■mu",
      meaning: "█████ ■ '████████' ███████:",
      options: [
        { text: "██████████ ■■ ✗", correct: false },
        { text: "██████ ■■ ✓", correct: true },
        { text: "███████ ■■ ✗", correct: false },
        { text: "██████ ■■ ✗", correct: false },
      ],
    },
    {
      id: 41, type: "quiz",
      arabic: "████████", armenian: "██████ ■■", transliteration: "an■mu",
      meaning: "█████████████: █████ ■ '████████':",
      options: [{ text: "██████████ ■■", correct: false }, { text: "██████ ■■", correct: true }],
    },
    {
      id: 5, type: "match",
      arabic: "████████", armenian: "█████ ██████", transliteration: "",
      meaning: "██████ ███████ ██████ ████████████:",
      pairs: [
        pair("██████████████", "██████████ ■■"),
        pair("██████████", "██████████ ■■"),
        pair("██████████", "██████████ ■■"),
        pair("████████", "██████ ■■"),
      ],
    },
    {
      id: 6, type: "quiz",
      arabic: "██████████", armenian: "███████████",
      transliteration: "■ab■■an",
      meaning: "█████ ■ '██████████' ███████:",
      options: [
        { text: "███████████ ✓", correct: true },
        { text: "████████ ✗", correct: false },
        { text: "██████████ ✗", correct: false },
      ],
    },
    {
      id: 7, type: "speak",
      arabic: "████████████████ ██████████ ████████████ ███████████ ████████████",
      armenian: "███████████ ███████████ ■■, ██████████ ■■ ■ ██████ ██████ ■■",
      transliteration: "astayqi■u ■ab■■an wa-a■malu wa-an■mu mubakkiran",
      meaning: "███████ ████████████████████:", hintIcon: "■■",
    },
    {
      id: 8, type: "write",
      arabic: "████████", armenian: "██████ ■■",
      transliteration: "an■mu",
      meaning: "██████ '██████ ■■' ██████████:",
      hint: "■ + ■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u2.ts

```
import { LessonData, pair } from '../types';

export const u2: LessonData = {
  id: "u2",
  title: "████████████████ (■, ■, ■)",
  titleAr: "███████ ████████",
  xpReward: 50,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "■■■ / ■■■ / ■■■",
```

```
  armenian: "b■ / b■ / b■",
  transliteration: "b■ / b■ / b■",
  hint: "■■■■ + ■ = ■  ·  ■■■ + ■ = ■  ·  ■■■■ + ■ = ■",
  hintIcon: "■",
},
{
  id: 2,
  type: "quiz",
  arabic: "■■■",
  armenian: "■■■ (b■)",
  transliteration: "b■",
  meaning: "■■■ ■■■■■■■■■ ■■■■■■■■■■■ ■ 'b■' ■■■■■■■■■■■ (■■ ■):",
  options: [
    { text: "■■■", correct: true },
    { text: "■■■", correct: false },
    { text: "■■■", correct: false },
  ],
},
{
  id: 21,
  type: "quiz",
  arabic: "■■■",
  armenian: "■■■ (b■)",
  transliteration: "b■",
  meaning: "■■■■■■■■■■■■: ■■■ ■■■■■■■■■ ■■■■■■■■■■■■ ■ 'b■':",
  options: [
    { text: "■■■", correct: true },
    { text: "■■■", correct: false },
  ],
},
{
  id: 3,
  type: "listen",
  arabic: "■■■■■",
  armenian: "■■■■■",
  transliteration: "b■b",
  hint: "■■ ■ — ■■■■ + ■■■ ■■",
  hintIcon: "■",
  highlightChar: "■■■",
},
{
  id: 4,
  type: "listen",
  arabic: "■■■■■",
  armenian: "■■■■■■",
  transliteration: "n■r",
  hint: "■■ ■ — ■■■ + ■■■ ■■",
  hintIcon: "■",
  highlightChar: "■■■",
},
{
  id: 5,
  type: "quiz",
  arabic: "■■■■■■",
  armenian: "■■■■■■",
  transliteration: "bint",
  meaning: "■■■■■■■■ '■■■■■■' ■■■■■ ■■■■■ ■■■■■■■■■■■:",
  options: [
    { text: "■■■■ (■■■■)", correct: false },
    { text: "■■■ (■■■)", correct: true },
  ],
},
{
  id: 6,
  type: "match",
  arabic: "■■■",
  armenian: "■■■■■■■■■■■■■■■■ ■■■■",
  transliteration: "",
  meaning: "■■■■■■■ ■■■■■■■■■■■■■■■ ■■■■■ ■■ ■■■■■■■ ■■■■:",
  pairs: [
    pair("■■■■■", "■■■■■"),
    pair("■■■■■", "■■■■■■"),
```

```
        pair("███████", "█████"),
        pair("██████", "█████"),
      ],
    },
    {
      id: 7,
      type: "speak",
      arabic: "█████",
      armenian: "█████",
      transliteration: "b■b",
      meaning: "██████████ 'b■b' ██████ ███████ ■-■:",
      hintIcon: "■■",
    },
    {
      id: 8,
      type: "write",
      arabic: "█████",
      armenian: "██████",
      transliteration: "n■r",
      meaning: "██████ '██████' ██████████:",
      hint: "■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u20.ts

```
import { LessonData, pair } from '../types';

export const u20: LessonData = {
  id: "u20",
  title: "██████████ █████",
  titleAr: "███████ ██████",
  xpReward: 100,
  steps: [
    {
      id: 1, type: "listen",
      arabic: "██████████ ██████████ — ██████ ████ ████████████████",
      armenian: "██████████ ██████████ — █████ ████, ████ ████████ ■■",
      transliteration: "mur■ja■a sh■mila — kullu m■ ta■allamtahu",
      hint: "██████████ █████ — u1-■■ u19 ███████", hintIcon: "■",
    },
    {
      id: 2, type: "quiz",
      arabic: "███████ ████████", armenian: "████████ ■■:",
      transliteration: "kayfa ■■luk?",
      meaning: "█████ ■ █████ ██████████ '████ ■■':",
      options: [
        { text: "██████ ████████ ✓", correct: true },
        { text: "██████ ███████ ✗", correct: false },
        { text: "██████ ███████ ✗", correct: false },
      ],
    },
    {
      id: 21, type: "quiz",
      arabic: "███████ ████████", armenian: "████████ ■■:", transliteration: "kayfa ■■luk?",
      meaning: "████████████: '████ ■■' ██████████:",
      options: [{ text: "██████ ████████ ✓", correct: true }, { text: "██████ ███████ ✗", correct: false }],
    },
    {
      id: 3, type: "quiz",
      arabic: "██████████ ██████ ███████████", armenian: "██████ ██████████ ████ ■",
      transliteration: "al-kit■bu ■al■ l-maktabi",
      meaning: "█████ ■ '██████' ███████:",
      options: [
        { text: "████ (in) ✗", correct: false },
        { text: "████ (on) ✓", correct: true },
        { text: "█████ (front) ✗", correct: false },
      ],
```

```
    },
    {
      id: 31, type: "quiz",
      arabic: "██████", armenian: "███ (on)", transliteration: "■al■",
      meaning: "████████████: ████ ■ '██████':",
      options: [{ text: "███", correct: false }, { text: "███", correct: true }],
    },
    {
      id: 4, type: "match",
      arabic: "███████", armenian: "███████████", transliteration: "",
      meaning: "██████████ ████████████████████ — ██████:",
      pairs: [
        pair("██████████", "█████"),
        pair("████████", "████████████████████"),
        pair("██████", "███"),
        pair("███", "■■"),
      ],
    },
    {
      id: 5, type: "quiz",
      arabic: "██████ ████████ ██████████████", armenian: "■■ ████████ ■■ ████████",
      transliteration: "an■ adrusu l-■arabiyya",
      meaning: "████ ■ '████████' ███████:",
      options: [
        { text: "██████ ■■ ✗", correct: false },
        { text: "██████████ ■■ ✓", correct: true },
        { text: "██████ ■■ ✗", correct: false },
      ],
    },
    {
      id: 6, type: "speak",
      arabic: "█████████ ██████ ███████ ... ██████ ████ ██████████████ ███████████ ██████████████",
      armenian: "█████, ■■ ███████ ■ ..., ■■ ███████████ ■■ ■ ████████ ■■ ████████",
      transliteration: "ahlan, an■ ism■ … wa-an■ min Arm■niy■ wa-adrusu l-■arabiyya",
      meaning: "███████████ ████████████ ████:", hintIcon: "■■",
    },
    {
      id: 7, type: "speak",
      arabic: "████ ████████████ ████████████████ — ███████ ██████████ █████████!",
      armenian: "██████████ ████████ ■■: — ███, ■■ ████, ██████████ ■■:",
      transliteration: "hal tatakallamu l-■arabiyya? — na■am, qal■lan, adrusu!",
      meaning: "██████████ ████████████████:", hintIcon: "■",
    },
    {
      id: 8, type: "write",
      arabic: "████████", armenian: "████████████████████",
      transliteration: "shukran",
      meaning: "██████ '████████████████████' █████████:",
      hint: "■ + ■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u21.ts

```
import { LessonData, pair } from '../types';

export const u21: LessonData = {
  id: "u21",
  title: "████ ■ ██████ ██████████",
  titleAr: "████████ █████████",
  xpReward: 60,
  steps: [
    {
      id: 1, type: "listen",
      arabic: "███████████",
      armenian: "█████",
      transliteration: "al-a■d■d",
      hint: "3-10 ██████ ███ ████████ ███████ ■ ████████ ████████ ████████ (███ ██████):",
```

```
      hintIcon: "■",
    },
    {
      id: 2, type: "quiz",
      arabic: "■■■■■■■■ ■ ■ ■ ■ (■■■■■■)",
      armenian: "■■ ■■■■■■ ... (■■■■■■)",
      transliteration: "■ind■ ... kutub",
      meaning: "■■■■■■■■ ■■■■■■■■■■■■■■■■ ■■■■■ ■■■■■:",
      options: [
        { text: "■■■■■■■■■■ ■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■■■■ ■■■■■■■■ ✗", correct: false },
        { text: "■■■■■■■■■■ ■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 3, type: "quiz",
      arabic: "■■■■ ■■■■■■■■■ ■ ■ ■ ■ (■■■■■■■■)",
      armenian: "■■■■■■■■■■■■ ■■■■ ... (■■■■■■■■■■■■)",
      transliteration: "f■ l-qismi ... ■ull■b",
      meaning: "■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■:",
      options: [
        { text: "■■■■■■■■■ ■■■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■ ■■■■■■■■■ ✗", correct: false },
        { text: "■■■■■■■■■■ ■■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 4, type: "quiz",
      arabic: "■■■■■■■■■ ■ ■ ■ ■ (■■■■■■■■)",
      armenian: "■■ ■■■■■■■■ ... (■■■■■■)",
      transliteration: "qara■tu ... dur■s",
      meaning: "■■■■■■■ ■■■■■ ■■■■■■■■■■:",
      options: [
        { text: "■■■■■■■■■ ■■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■■■ ■■■■■■■ ✗", correct: false },
        { text: "■■■■■■■■■■ ■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 5, type: "match",
      arabic: "■■■■■■■",
      armenian: "■■■■■■■■■■■■■■■■■■■■",
      transliteration: "",
      meaning: "■■■■■■ ■■■■■■ ■■■■■■■ ■■■:",
      pairs: [
        pair("■■■■■■■■■■ ■■■■■■■■■", "■■■■■ ■■■■■■■■"),
        pair("■■■■■■■■■ ■■■■■■■", "■■■■■ ■■■■■"),
        pair("■■■■■■■■■■ ■■■■■■■■■", "■■■■ ■■"),
        pair("■■■■■■■■■ ■■■■■■■■", "■■■■ ■■■■■■■■"),
      ],
    },
    {
      id: 6, type: "write",
      arabic: "■■■■■■■■■■ ■■■■■■■",
      armenian: "■■■■■ ■■■■■",
      transliteration: "thal■thatu kutubin",
      meaning: "■■■■■■ 'thal■thatu kutubin' ■■■■■■■■■:",
      hint: "■■■■■■ ■■■■",
    },
    {
      id: 7, type: "quiz",
      arabic: "■■■■■■ ■■■■■■: ■■■■■■■■■ ■■■■■■■■",
      armenian: "■■■■■■■ ■■■■■■■ «■■■■■ ■■■■■»:",
      transliteration: "Ta■■■■■: khamsatu kit■bin",
      meaning: "■■■■■■■■ ■ ■■■■■:",
      options: [
        { text: "■■■■■■■■■ ■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■ ■■■■■■■ ✗", correct: false },
      ]
    }
  ],
};
```

■ FICHIER : src\data\lessons\u22.ts

```
import { LessonData, pair } from '../types';

export const u22: LessonData = {
  id: "u22",
  title: "■■■■■■■■■■■■ ■■■■■■■■■■■■",
  titleAr: "■■■■■■■■ ■■■■■■■■■■",
  xpReward: 60,
  steps: [
    {
      id: 1, type: "listen",
      arabic: "■■■■■■■■■■■■ ■■■■■■■■■■■■■■■■■",
      armenian: "■■■■■■■■■■■■ ■■■■■■■■■■■■ (Harakat)",
      transliteration: "al-■arak■tu l-i■r■biyya",
      hint: "■■■■■ ■ (fatha), ■■■■■■■■ ■ (damma), ■■■■■■■ ■ (fatha), ■■■■■■/■■■■■■■■■■ ■ (kasra)",
      hintIcon: "■",
    },
    {
      id: 2, type: "quiz",
      arabic: "■■■■■■■ ■ ■ ■ ■ ■■■■■■■■■■",
      armenian: "■■■■■■■■■■ ■■■■■ ■■■■■:",
      transliteration: "kataba ... ad-darsa",
      meaning: "■■■■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■■ ■■■■■■■■■ (■■■■■■■):",
      options: [
        { text: "■■■■■■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■■■■■■ ✗", correct: false },
        { text: "■■■■■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 3, type: "quiz",
      arabic: "■■■■■■■ ■■■■■■■■■ ■■■■■■ ■ ■ ■ ■",
      armenian: "■■■■■ ■■■■■ ■■■■■■:",
      transliteration: "dhahaba l-waladu il■ ...",
      meaning: "■■■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■■ ■■■■■■■■■ (■■■■■■ -■■ ■■■■■):",
      options: [
        { text: "■■■■■■■■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■■■■■■■■ ✗", correct: false },
        { text: "■■■■■■■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 4, type: "quiz",
      arabic: "■■■■■■■ ■■■■■■■■■■■■ ■ ■ ■ ■",
      armenian: "■■■■■■■■■■ ■■■■■ ■■■■■■■:",
      transliteration: "qara■a ■-■■libu ...",
      meaning: "■■■■■■■■■ ■■■■■ ■■■■■■■■■■■■■■■■■ ■■■■■■■■■ (■■■■■■ ■■■■■■):",
      options: [
        { text: "■■■■■■■■■■■ ✓", correct: true },
        { text: "■■■■■■■■■■■ ✗", correct: false },
        { text: "■■■■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 5, type: "match",
      arabic: "■■■■■■■ ■■■■■■■■■■",
      armenian: "■■■■■ ■■■■■",
      transliteration: "",
      meaning: "■■■■■■ ■■■■■■■■■■■■■■■■■■■■ ■■■■■ ■■■■■■■:",
      pairs: [
        pair("■■■■■■...", "■■■■■■■■ (kit■bun)"),
        pair("■■■■■■■■■■...", "■■■■■■■■■ (■■liban)"),
        pair("■■■■■■■■■■ ■■■...", "■■■■■■■■■■ (mudarrisin)"),
      ],
    },
    {
      id: 6, type: "quiz",
      arabic: "■■■■■■ ■■■■■■: ■■■■■■ ■■■■■■■■■■■■",
```

```
      armenian: "■■■■■■■ ■■■■■■ (■■■■■ ■■■■■■■■■):",
      transliteration: "Ta■■■■■: j■■a ■-■■liba",
      meaning: "■■■■■ ■ ■■■■■ ■■■■ ■■■■■■■■■ ■■■■■■:",
      options: [
        { text: "■■■■■■ ■■■■■■■■■■■ ✓", correct: true },
        { text: "■■■■■■ ■■■■■■■■■■■ ✗", correct: false },
      ],
    },
    {
      id: 7, type: "write",
      arabic: "■■■■ ■■■■■■■■■",
      armenian: "■■■■■",
      transliteration: "f■ l-bayti",
      meaning: "■■■■■■ 'f■ l-bayti':",
      hint: "■■ ■■■■■■■",
    }
  ],
};
```

■ FICHIER : src\data\lessons\u3.ts

```
import { LessonData, pair } from '../types';

export const u3: LessonData = {
  id: "u3",
  title: "■■■■■■■■ (■■■■■■■ ■■■■■■.)",
  titleAr: "■■■■■■■■",
  xpReward: 50,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "■■■■■■■■ / ■■■■■■■■■ / ■■■■■■■■",
      armenian: "kit■bun / kit■ban / kit■bin",
      transliteration: "kit■bun / kit■ban / kit■bin",
      hint: "■ = ■■■■ (nominative) · ■ = ■■■■ (accusative) · ■ = ■■■■ (genitive)",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "quiz",
      arabic: "■■■■■■■■",
      armenian: "■■■■■ (■■■■■■■, ■■■■■.)",
      transliteration: "kit■bun",
      meaning: "■■■■■■■■ ■■■■■ ■■■■■■■■■ (■■■■ — nominative):",
      options: [
        { text: "■■■■■■■■", correct: true },
        { text: "■■■■■■■■■", correct: false },
        { text: "■■■■■■■■", correct: false },
      ],
    },
    {
      id: 21,
      type: "quiz",
      arabic: "■■■■■■■■",
      armenian: "■■■■■ (■■■■■■■, ■■■■■.)",
      transliteration: "kit■bun",
      meaning: "■■■■■■■■■■■■■: ■■■■■■■■ ■■■■■ ■■■■■■■■■ (■■■■):",
      options: [
        { text: "■■■■■■■■", correct: true },
        { text: "■■■■■■■■■", correct: false },
      ],
    },
    {
      id: 3,
      type: "quiz",
      arabic: "■■■■■■■■■■",
      armenian: "■■■■■ (■■■■■■■, ■■■■■.)",
      transliteration: "kit■ban",
```

```
  meaning: "████████ █████ ██████████ (████ — accusative):",
  options: [
    { text: "████████", correct: false },
    { text: "██████████", correct: true },
    { text: "████████", correct: false },
  ],
},
{
  id: 31,
  type: "quiz",
  arabic: "██████████",
  armenian: "█████ (████████, ██████.)",
  transliteration: "kit■ban",
  meaning: "██████████████: ████████ █████ ██████████ (████):",
  options: [
    { text: "████████", correct: false },
    { text: "██████████", correct: true },
  ],
},
{
  id: 4,
  type: "quiz",
  arabic: "████████",
  armenian: "█████ (████████, █████████)",
  transliteration: "kit■bin",
  meaning: "████████ █████ ██████████ (██ — genitive):",
  options: [
    { text: "█████████", correct: false },
    { text: "██████████", correct: false },
    { text: "█████████", correct: true },
  ],
},
{
  id: 41,
  type: "quiz",
  arabic: "████████",
  armenian: "█████ (████████, █████████)",
  transliteration: "kit■bin",
  meaning: "██████████████: ████████ █████ ██████████ (██):",
  options: [
    { text: "██████████", correct: false },
    { text: "████████", correct: true },
  ],
},
{
  id: 5,
  type: "speak",
  arabic: "████████ — ██████████ — █████████",
  armenian: "████████ █████ ██████████████ ████ ██████.",
  transliteration: "kit■bun — kit■ban — kit■bin",
  meaning: "████████████ ████████████████ ████ ████ ██████.",
  hintIcon: "██",
},
{
  id: 6,
  type: "listen",
  arabic: "███████ █████████",
  armenian: "██ ██████ ■",
  transliteration: "h■dh■ kit■bun",
  hint: "████████ ████████████ ■ ■ -██ (nominative predicate)",
  hintIcon: "■",
},
{
  id: 7,
  type: "match",
  arabic: "███████",
  armenian: "██████████████ ████",
  transliteration: "",
  meaning: "████████ ██████████ ██████████ ████████████ ████████████████:",
  pairs: [
    pair("█████████", "kit■bun"),
    pair("██████████", "kit■ban"),
```

```
        pair("████████", "kit■bin"),
      ],
    },
    {
      id: 8,
      type: "write",
      arabic: "█████████",
      armenian: "█████ (████████, ██████.)",
      transliteration: "kit■bun",
      meaning: "███████ '██████' (anoroush) ██████████:",
      hint: "■ + ■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u4.ts

```
import { LessonData, pair } from '../types';

export const u4: LessonData = {
  id: "u4",
  title: "████ ■ ████████ (█████████)",
  titleAr: "███████ ■■ █████████",
  xpReward: 50,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "██████████ / ██████████ / ████████",
      armenian: "██████████ / ██████████ / █████████",
      transliteration: "ar-raf■ / an-na■b / al-jarr",
      hint: "████████████ ██████ ██████████ ████████████",
      hintIcon: "■■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "██████████ ██████████",
      armenian: "██████ ████████ ■ (████████████ ███████)",
      transliteration: "al-waladu yal■abu",
      hint: "██████████ ████████████ ■ «■■» ████████████ (████) ███████ ████████ (███████)",
      hintIcon: "■",
      highlightChar: "■■",
    },
    {
      id: 3,
      type: "quiz",
      arabic: "██████████ ██████████",
      armenian: "██████ ████████ ■",
      transliteration: "al-waladu yal■abu",
      meaning: "██████ ██████████████████ ██████ █████████ (██████) ███████ ████████:",
      options: [
        { text: "■■ (██████)", correct: false },
        { text: "■■ (████)", correct: true },
        { text: "■■ (██████)", correct: false },
      ],
    },
    {
      id: 31,
      type: "quiz",
      arabic: "██████████ ██████████",
      armenian: "██████ ████████ ■",
      transliteration: "al-waladu yal■abu",
      meaning: "██████████████: ██████ ██████████████████ ██████ █████████ (██████) ███████ ████████:",
      options: [
        { text: "■■ (██████)", correct: false },
        { text: "■■ (████)", correct: true },
      ],
    },
```

```
{
  id: 5,
  type: "listen",
  arabic: "██████████ ██████████",
  armenian: "██ ██████ ████████ (████████████ ██████████)",
  transliteration: "ra■aytu l-walada",
  hint: "██████████ ████████████ ■ «a» ████████████ (██████) ██████ ████████",
  hintIcon: "■",
  highlightChar: "■■",
},
{
  id: 6,
  type: "quiz",
  arabic: "██████████ ██████████",
  armenian: "██ ██████ ████████",
  transliteration: "ra■aytu l-walada",
  meaning: "██████ ██████████████████ ██████ ████████ (██████) ██████ ████████:",
  options: [
    { text: "■■ (██████ – ■■■■)", correct: true },
    { text: "■■ (■■■■ – ■■■■)", correct: false },
    { text: "■■ (██████ – ■■)", correct: false },
  ],
},
{
  id: 7,
  type: "listen",
  arabic: "██████████ ██████ ██████████",
  armenian: "■■ ██████ ██████ (████████ – ██████████)",
  transliteration: "dhahabtu il■ l-bayti",
  hint: "██████████ ████████████ ■ «i» ████████████ (██████) ████████ ██████████",
  hintIcon: "■",
  highlightChar: "■■",
},
{
  id: 8,
  type: "quiz",
  arabic: "██████ ██████████",
  armenian: "██████ ██████",
  transliteration: "il■ l-bayti",
  meaning: "██████ – ████████████ ■ ■■ ████████:",
  options: [
    { text: "■■■■ (■■)", correct: false },
    { text: "■■■■ (■■)", correct: false },
    { text: "■■ (■■)", correct: true },
  ],
},
{
  id: 9,
  type: "match",
  arabic: "█████████",
  armenian: "████████████ ██████",
  transliteration: "",
  meaning: "████████████████████ ██████████ ████████████ ██████ ████████ ████████████████:",
  pairs: [
    pair("██████████", "██████████"),
    pair("██████████", "██████████"),
    pair("██████████", "█████████/████████"),
  ],
},
{
  id: 10,
  type: "speak",
  arabic: "██████████ ██████████",
  armenian: "██████ ████████ ■",
  transliteration: "al-waladu yal■abu",
  meaning: "████████████ ■ ██████████████████ ████████ «■■» (■■■■) ████████████████████:",
  hintIcon: "■■",
},
{
  id: 11,
  type: "write",
  arabic: "██████████",
```

```
      armenian: "■■■■■",
      transliteration: "al-waladu",
      meaning: "■■■■■■ '■■■■■' ■■■■■■■■■■ ■■■■■ ■■■■■■■■■ ■■■■■■■■■■■■■■■■:",
      hint: "■ + ■ + ■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u5.ts

```
import { LessonData, pair } from '../types';

export const u5: LessonData = {
  id: "u5",
  title: "■■■■■ 1-10 (■■■■■■■■)",
  titleAr: "■■■■■■■■ ■■ 1 ■■■■ 10",
  xpReward: 50,
  steps: [
    { id: 1, type: "listen", arabic: "■■■■■■■", armenian: "■■■■ (1)", transliteration: "w■■id", hintIcon: "1■■"
    { id: 2, type: "listen", arabic: "■■■■■■■■■", armenian: "■■■■■■ (2)", transliteration: "ithn■n", hintIcon:
    { id: 3, type: "listen", arabic: "■■■■■■■■■", armenian: "■■■■■ (3)", transliteration: "thal■tha", hintIcon:
    { id: 4, type: "listen", arabic: "■■■■■■■■■■", armenian: "■■■■■ (4)", transliteration: "arba■a", hintIcon:
    { id: 5, type: "listen", arabic: "■■■■■■■■", armenian: "■■■■■ (5)", transliteration: "khamsa", hintIcon: "5■
    { id: 6, type: "listen", arabic: "■■■■■■■", armenian: "■■■■ (6)", transliteration: "sitta", hintIcon: "6■■"
    { id: 7, type: "listen", arabic: "■■■■■■■■", armenian: "■■■■ (7)", transliteration: "sab■a", hintIcon: "7■■"

    {
      id: 8,
      type: "match",
      arabic: "■■■■■■ ■■■■■■ ■■■■■■",
      armenian: "■■■■ 1-■ ■■■■■■",
      transliteration: "",
      meaning: "■■■■■■■■■■■■■■■■■■■ ■■■■■■■",
      pairs: [
        pair("■■■■■■■ (1)", "■■■■ (1)"),
        pair("■■■■■■■■■ (2)", "■■■■■■ (2)"),
        pair("■■■■■■■■■ (3)", "■■■■■ (3)"),
        pair("■■■■■■■■■■ (4)", "■■■■■ (4)"),
      ],
    },

    { id: 9, type: "listen", arabic: "■■■■■■■■■■■■", armenian: "■■■■ (8)", transliteration: "tham■niya", hintIcon
    { id: 10, type: "listen", arabic: "■■■■■■■■■", armenian: "■■■■ (9)", transliteration: "tis■a", hintIcon: "9■■
    { id: 11, type: "listen", arabic: "■■■■■■■■", armenian: "■■■■■ (10)", transliteration: "■ashara", hintIcon:

    {
      id: 12,
      type: "match",
      arabic: "■■■■■■ ■■■■■■ ■■■■■■■",
      armenian: "■■■■ 2-■ ■■■■■■",
      transliteration: "",
      meaning: "■■■■■■■■■■■■■■■■■■■ ■■■■■■■",
      pairs: [
        pair("■■■■■■■■ (5)", "■■■■■ (5)"),
        pair("■■■■■■■ (6)", "■■■■ (6)"),
        pair("■■■■■■■■ (7)", "■■■■ (7)"),
      ],
    },

    {
      id: 13,
      type: "match",
      arabic: "■■■■■■ ■■■■■■ ■■■■■■■",
      armenian: "■■■■ 3-■ ■■■■■■",
      transliteration: "",
      meaning: "■■■■■■■■■■■■■■■■■■■ ■■■■■■■",
      pairs: [
        pair("■■■■■■■■■■■ (8)", "■■■■ (8)"),
        pair("■■■■■■■■ (9)", "■■■■ (9)"),
```

```
        pair("████████ (10)", "████ (10)"),
      ],
    },

    {
      id: 14,
      type: "quiz",
      arabic: "████████",
      armenian: "████ (10)",
      transliteration: "■ashara",
      meaning: "████ ■ '████████' ████ ████ ████████████████:",
      options: [
        { text: "████ (5)", correct: false },
        { text: "███ (7)", correct: false },
        { text: "████ (10)", correct: true },
      ],
    },
    {
      id: 141,
      type: "quiz",
      arabic: "████████",
      armenian: "████ (10)",
      transliteration: "■ashara",
      meaning: "████████████: ████ ■ '████████' ████ ████ ████████████████:",
      options: [
        { text: "████ (5)", correct: false },
        { text: "████ (10)", correct: true },
      ],
    },
    {
      id: 15,
      type: "write",
      arabic: "███████",
      armenian: "███ (1)",
      transliteration: "w■■id",
      meaning: "██████ '1' ████████:",
      hint: "■ + ■ + ■ + ■",
    },
    {
      id: 16,
      type: "speak",
      arabic: "████████ ██████████ ████████",
      armenian: "███, ██████, ████",
      transliteration: "w■■id, ithn■n, thal■tha",
      meaning: "██████████ ██████ 1-■■ 3-■:",
      hintIcon: "■■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u6.ts

```
import { LessonData, pair } from '../types';

export const u6: LessonData = {
  id: "u6",
  title: "██████ ■ ████████████",
  titleAr: "████████ ████████",
  xpReward: 50,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "██████ ████████",
      armenian: "■■ ████████ ■■",
      transliteration: "an■ adrusu",
      hint: "██████ (■■) ██████████ ■ ■... ████ ███████",
      hintIcon: "■",
    },
```

```
{
  id: 2,
  type: "listen",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "huwa yadrusu",
  hint: "[redacted]",
  hintIcon: "[redacted]",
},
{
  id: 3,
  type: "listen",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "anta taktubu",
  hint: "[redacted]",
  hintIcon: "[redacted]",
},
{
  id: 4,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "hiya tadrusu",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 41,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "hiya tadrusu",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 5,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "yadrusu",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: true },
    { text: "[redacted]", correct: false },
  ],
},
{
  id: 51,
  type: "quiz",
  arabic: "[redacted]",
  armenian: "[redacted]",
  transliteration: "yadrusu",
  meaning: "[redacted]",
  options: [
    { text: "[redacted]", correct: false },
    { text: "[redacted]", correct: true },
  ],
},
{
  id: 6,
  type: "match",
```

```
      arabic: "████████",
      armenian: "██████████████ ████",
      transliteration: "",
      meaning: "███████ ██████████ ██████ ██████ ██████ ████ (██████):",
      pairs: [
        pair("██████", "██████████"),
        pair("██████", "██████████"),
        pair("██████", "██████████"),
        pair("████████", "██████████"),
      ],
    },
    {
      id: 7,
      type: "speak",
      arabic: "██████ ██████████ ██████ ██████████",
      armenian: "███ ██████████ ███, ■ ███ ██████████ ■",
      transliteration: "an■ adrusu wa huwa yadrusu",
      meaning: "████████████ ████████ ██████████████████:",
      hintIcon: "███",
    },
    {
      id: 8,
      type: "write",
      arabic: "██████████",
      armenian: "(███ ████████) ██████████ ■",
      transliteration: "yadrusu",
      meaning: "███████ '███ (████████) ██████████ ■' ██████████:",
      hint: "■ + ■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u7.ts

```
import { LessonData, pair } from '../types';

export const u7: LessonData = {
  id: "u7",
  title: "████████████ (█████████)",
  titleAr: "█████████ ██████████",
  xpReward: 50,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "████████████",
      armenian: "██████",
      transliteration: "mar■aban",
      hint: "████████████████ ██████████",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "██████████████ ████████████",
      armenian: "████████████████ ████ (██████ ████)",
      transliteration: "as-sal■mu ■alaykum",
      hint: "██████████████ ■ ██████████ ██████████",
      hintIcon: "■",
    },
    {
      id: 3,
      type: "listen",
      arabic: "████████ ██████████",
      armenian: "██████████ ███ ████",
      transliteration: "kayfa ■■luk",
      hint: "██████████ ██████████████ ███████",
      hintIcon: "■",
    },
```

```
{
  id: 4,
  type: "listen",
  arabic: "██████ ██████████ ████████",
  armenian: "██ ████ ██, ██████████████████",
  transliteration: "an■ bi-khayr, shukran",
  hint: "████████████ ████████ ████████-██",
  hintIcon: "■",
},
{
  id: 5,
  type: "quiz",
  arabic: "████████████",
  armenian: "██████",
  transliteration: "mar■aban",
  meaning: "██████ ■ ████████████ ████████████:",
  options: [
    { text: "██████████████", correct: false },
    { text: "██████", correct: true },
    { text: "████████████████████", correct: false },
  ],
},
{
  id: 51,
  type: "quiz",
  arabic: "████████████",
  armenian: "██████",
  transliteration: "mar■aban",
  meaning: "██████████████: ██████ ■ ████████████ ████████████:",
  options: [
    { text: "██████████████", correct: false },
    { text: "██████", correct: true },
  ],
},
{
  id: 6,
  type: "quiz",
  arabic: "█████████",
  armenian: "████████████████████",
  transliteration: "shukran",
  meaning: "██████ ■ ████████████ █████████:",
  options: [
    { text: "████████", correct: false },
    { text: "█████████", correct: false },
    { text: "████████████████████", correct: true },
  ],
},
{
  id: 61,
  type: "quiz",
  arabic: "█████████",
  armenian: "████████████████████",
  transliteration: "shukran",
  meaning: "██████████████: ██████ ■ ████████████ █████████:",
  options: [
    { text: "████████", correct: false },
    { text: "████████████████████", correct: true },
  ],
},
{
  id: 7,
  type: "match",
  arabic: "███████",
  armenian: "██████████████ ████",
  transliteration: "",
  meaning: "████████ ████████████████████ ████████████████ ████:",
  pairs: [
    pair("████████████", "██████"),
    pair("████████ █████████", "█████████ ███ ████"),
    pair("█████████", "████████████████████"),
  ],
},
```

```
    {
      id: 8,
      type: "speak",
      arabic: "■■■■■■■■■■■ ■■■■■■■ ■■■■■■■■",
      armenian: "■■■■■, ■■■■■■■■ ■■ ■■■■:",
      transliteration: "mar■aban, kayfa ■■luk",
      meaning: "■■■■■■■■■■■■ ■■■■ ■■■■■■■■■■■■■■ ■■■■■■■:",
      hintIcon: "■■",
    },
    {
      id: 9,
      type: "write",
      arabic: "■■■■■■■■",
      armenian: "■■■■■■■■■■■■■■■■■■■",
      transliteration: "shukran",
      meaning: "■■■■■■ '■■■■■■■■■■■■■■■■■■' ■■■■■■■■■:",
      hint: "■ + ■ + ■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u8.ts

```
import { LessonData, pair } from '../types';

export const u8: LessonData = {
  id: "u8",
  title: "■■■■■■■■ (■■■■■■)",
  titleAr: "■■■■■■ ■■■■■■■",
  xpReward: 50,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "■■■■■■■■",
      armenian: "■■■■■■",
      transliteration: "al-yawm",
      hint: "O■ — ■■■■■■",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "■■■■■■",
      armenian: "■■■■■",
      transliteration: "ghadan",
      hint: "■■■■■■ (Future)",
      hintIcon: "■",
    },
    {
      id: 3,
      type: "listen",
      arabic: "■■■■■■",
      armenian: "■■■■■",
      transliteration: "ams",
      hint: "■■■■■■■ (Past)",
      hintIcon: "■",
    },
    {
      id: 4,
      type: "quiz",
      arabic: "■■■■■■■■",
      armenian: "■■■■■■",
      transliteration: "al-yawm",
      meaning: "■■■■■ ■ ■■■■■■■■■■ ■■■■■■■■:",
      options: [
        { text: "■■■■■", correct: false },
        { text: "■■■■■■", correct: true },
        { text: "■■■■■", correct: false },
```

```
  ],
},
{
  id: 41,
  type: "quiz",
  arabic: "████████",
  armenian: "██████",
  transliteration: "al-yawm",
  meaning: "████████████: ████ ■ ██████████ ████████:",
  options: [
    { text: "████", correct: false },
    { text: "██████", correct: true },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "██████",
  armenian: "████████████",
  transliteration: "",
  meaning: "██████████████████ ███████:",
  pairs: [
    pair("████████", "██████"),
    pair("██████", "████"),
    pair("██████", "████"),
    pair("██████", "██"),
  ],
},
{
  id: 6,
  type: "listen",
  arabic: "████████ ████ ██████████",
  armenian: "██████ ██████████ ■",
  transliteration: "al-yawmu huwa l-kham■s",
  hint: "██████████ = ██████████",
  hintIcon: "■",
},
{
  id: 7,
  type: "quiz",
  arabic: "█████ ████████████",
  armenian: "█████ ████████ ■:",
  transliteration: "kami s-s■■a?",
  meaning: "████████ ████████ ████:",
  options: [
    { text: "███████ ████████", correct: false },
    { text: "█████ ████████████", correct: true },
    { text: "███████ ███████████", correct: false },
  ],
},
{
  id: 71,
  type: "quiz",
  arabic: "█████ ████████████",
  armenian: "█████ ████████ ■:",
  transliteration: "kami s-s■■a?",
  meaning: "████████████: ████████ ████████ ████:",
  options: [
    { text: "███████ ████████", correct: false },
    { text: "█████ ████████████", correct: true },
  ],
},
{
  id: 8,
  type: "speak",
  arabic: "████████ ████ ██████████",
  armenian: "██████ ██████████ ■:",
  transliteration: "al-yawmu huwa l-kham■s",
  meaning: "████████████ ████ ████████████████:",
  hintIcon: "■■",
},
{
```

```
      id: 9,
      type: "write",
      arabic: "██████",
      armenian: "█████",
      transliteration: "ams",
      meaning: "██████ '█████' ██████████:",
      hint: "■ + ■ + ■",
    },
  ],
};
```

■ FICHIER : src\data\lessons\u9.ts

```
import { LessonData, pair } from '../types';

export const u9: LessonData = {
  id: "u9",
  title: "████████ (Colors)",
  titleAr: "████████",
  xpReward: 60,
  steps: [
    {
      id: 1,
      type: "listen",
      arabic: "████████ / ████████ / ████████ / ████████ / ████████ / ████████",
      armenian: "███████ / ██████ / ████████ / ██████ / ███████ / ■■",
      transliteration: "a■mar / akh■ar / azraq / a■far / abya■ / aswad",
      hint: "████████ ██████ ██████████",
      hintIcon: "■",
    },
    {
      id: 2,
      type: "listen",
      arabic: "████████████████ ██████████",
      armenian: "████████ ███████ ■",
      transliteration: "as-sayy■ratu ■amr■■",
      hint: "███████ ████████ ██████████ ██████████ (■amr■■)",
      hintIcon: "■",
    },
    {
      id: 3,
      type: "quiz",
      arabic: "████████",
      armenian: "███████",
      transliteration: "a■mar",
      meaning: "████ ███████ ■ '████████':",
      options: [
        { text: "■ ██████ (Kanach)", correct: false },
        { text: "■ ███████ (Karmir)", correct: true },
        { text: "■ ████████ (Kapuyt)", correct: false },
        { text: "■ ██████ (Deghin)", correct: false },
      ],
    },
    {
      id: 31,
      type: "quiz",
      arabic: "████████",
      armenian: "███████",
      transliteration: "a■mar",
      meaning: "████████████: ████ ███████ ■ '████████':",
      options: [
        { text: "■ ██████ (Kanach)", correct: false },
        { text: "■ ███████ (Karmir)", correct: true },
      ],
    },
    {
      id: 4,
      type: "quiz",
      arabic: "██████████ ████████",
```

```
  armenian: "██████ ████████ ■",
  transliteration: "al-kit■bu azraq",
  meaning: "█████ ██████ ████ ██████:",
  options: [
    { text: "███████ (Karmir)", correct: false },
    { text: "████████ (Kapuyt)", correct: true },
    { text: "██████ (Kanach)", correct: false },
  ],
},
{
  id: 41,
  type: "quiz",
  arabic: "██████████ ████████",
  armenian: "██████ ████████ ■",
  transliteration: "al-kit■bu azraq",
  meaning: "████████████: ████ ██████ ████ ██████:",
  options: [
    { text: "███████ (Karmir)", correct: false },
    { text: "████████ (Kapuyt)", correct: true },
  ],
},
{
  id: 5,
  type: "match",
  arabic: "████████",
  armenian: "████████",
  transliteration: "",
  meaning: "███████ ████████ ██████ ███████████████:",
  pairs: [
    pair("████████", "███████"),
    pair("████████", "██████"),
    pair("████████", "████████"),
    pair("████████", "██████"),
  ],
},
{
  id: 6,
  type: "listen",
  arabic: "███ ███████ ██████████ — ██████████ ████████",
  armenian: "████ ███████ ■ ████████: — ████████ ████████ ■:",
  transliteration: "m■ lawnu s-sam■■? — as-sam■■u zarq■■",
  hint: "████ ■■ ████████ ███████ ██████",
  hintIcon: "■■",
},
{
  id: 7,
  type: "quiz",
  arabic: "███ ███████ ████████████",
  armenian: "████ ███████ ■ ████:",
  transliteration: "m■ lawnu sh-shajar■?",
  meaning: "████████ ████ ██████████:",
  options: [
    { text: "████████ (████████)", correct: false },
    { text: "████████ (██████)", correct: true },
    { text: "████████ (██████)", correct: false },
  ],
},
{
  id: 71,
  type: "quiz",
  arabic: "███ ███████ ████████████",
  armenian: "████ ███████ ■ ████:",
  transliteration: "m■ lawnu sh-shajar■?",
  meaning: "████████████: ████████ ████ ██████████:",
  options: [
    { text: "████████ (████████)", correct: false },
    { text: "████████ (██████)", correct: true },
  ],
},
{
  id: 8,
  type: "write",
```

```
      arabic: "████████",
      armenian: "███████",
      transliteration: "a■mar",
      meaning: "██████ '███████' █████████:",
      hint: "■ + ■ + ■ + ■",
    },
    {
      id: 9,
      type: "speak",
      arabic: "███████████████ █████████ ████████████ ████████",
      armenian: "████████ ███████ ■, ██████ ████████:",
      transliteration: "as-sayy■ratu ■amr■■ wa-l-kit■bu azraq",
      meaning: "████████ ██████████████████:",
      hintIcon: "■■",
    },
  ],
};
```

■ FICHIER : src\data\types.ts

```
export interface QuizOption {
  text: string;
  correct: boolean;
}

export interface MatchPair {
  arabic: string;
  armenian: string;
}

export interface LessonStep {
  id: number;
  type: "listen" | "speak" | "quiz" | "match" | "write";
  arabic: string;
  armenian: string;
  transliteration: string;
  hint?: string;
  hintIcon?: string;
  highlightIndex?: number;
  highlightChar?: string;
  audio?: string;
  options?: QuizOption[];
  pairs?: MatchPair[];
  meaning?: string;
}

export interface LessonData {
  id: string;
  title: string;
  titleAr: string;
  steps: LessonStep[];
  xpReward: number;
}

export const pair = (arabic: string, armenian: string): MatchPair =&gt; ({
  arabic,
  armenian,
});
```

■ FICHIER : src\hooks\useArabicTTS.ts

```
/**
 * useArabicTTS — synthèse vocale arabe fiable, web + mobile (Android/iOS).
 *
 * Ordre de priorité :
 *   1. Audio PRÉ-GÉNÉRÉ (manifest.json) — instantané, fiable, hors-ligne. RECOMMANDÉ.
 *   2. Web Speech API native — avec lang='ar-SA'. Sur Android, fonctionne même
```

```
 *      sans voix arabe explicite si Google TTS est installé (cas courant).
 *      Correctifs Android intégrés (anti-race cancel/speak + resume).
 *   3. Repli serveur /api/tts (WAV) — DÉSACTIVÉ dans Capacitor (pas de serveur).
 *      Actif uniquement en mode web.
 *
 * Correctifs Android intégrés :
 *   - Pas de cancel() juste avant speak() (sinon Android ignore speak silencieusement).
 *   - resume() de relance après speak() (Android met l'utterance en pause).
 *   - Déblocage de l'audio au 1er geste utilisateur (sinon play() après await est bloqué).
 */

import { useCallback, useEffect, useRef, useState } from 'react';

type Platform = 'android' | 'ios' | 'desktop';

/** Détecte si l'app tourne dans un conteneur Capacitor (APK natif). */
function isCapacitorApp(): boolean {
  return !!(window as unknown as Record&lt;string, unknown&gt;)['Capacitor'];
}

function detectPlatform(): Platform {
  const ua = navigator.userAgent;
  if (/Android/i.test(ua)) return 'android';
  if (/iPhone|iPad|iPod/i.test(ua)) return 'ios';
  return 'desktop';
}

/** Charge les voix système de façon asynchrone (obligatoire sur Chrome/Android). */
function getVoicesAsync(timeoutMs = 3000): Promise&lt;SpeechSynthesisVoice[]&gt; {
  return new Promise((resolve) =&gt; {
    if (!('speechSynthesis' in window)) return resolve([]);
    const existing = window.speechSynthesis.getVoices();
    if (existing.length &gt; 0) return resolve(existing);

    let settled = false;
    const finish = (v: SpeechSynthesisVoice[]) =&gt; {
      if (settled) return;
      settled = true;
      window.speechSynthesis.removeEventListener('voiceschanged', handler);
      resolve(v);
    };
    const handler = () =&gt; finish(window.speechSynthesis.getVoices());
    window.speechSynthesis.addEventListener('voiceschanged', handler);
    setTimeout(() =&gt; finish(window.speechSynthesis.getVoices()), timeoutMs);
  });
}

const sleep = (ms: number) =&gt; new Promise&lt;void&gt;((r) =&gt; setTimeout(r, ms));

export function useArabicTTS() {
  const [isPlaying, setIsPlaying] = useState(false);
  const platform = useRef&lt;Platform&gt;('desktop');
  const audioCache = useRef&lt;Map&lt;string, string&gt;&gt;(new Map());
  const currentAudio = useRef&lt;HTMLAudioElement | null&gt;(null);
  const arabicVoice = useRef&lt;SpeechSynthesisVoice | null&gt;(null);
  const manifest = useRef&lt;Record&lt;string, string&gt;&gt;({});
  const audioUnlocked = useRef(false);

  useEffect(() =&gt; {
    platform.current = detectPlatform();
    let cancelled = false;

    getVoicesAsync().then((voices) =&gt; {
      if (cancelled) return;
      // Préfère une voix arabe « locale » (hors-ligne) si disponible.
      arabicVoice.current =
        voices.find((v) =&gt; v.lang.startsWith('ar') &amp;&amp; v.localService) ??
        voices.find((v) =&gt; v.lang.startsWith('ar')) ??
        null;
    });

    fetch('/audio/manifest.json')
```

```
    .then((r) =&gt; (r.ok ? r.json() : {}))
    .then((m) =&gt; { if (!cancelled) manifest.current = m; })
    .catch(() =&gt; {});

  // Débloque l'audio mobile au 1er geste utilisateur (résout NotAllowedError).
  const unlock = () =&gt; {
    if (audioUnlocked.current) return;
    try {
      const a = new Audio();
      a.muted = true;
      a.play().catch(() =&gt; {});
      audioUnlocked.current = true;
    } catch { /* noop */ }
    // Réveille aussi le moteur de synthèse sur Android.
    if ('speechSynthesis' in window) {
      try { window.speechSynthesis.resume(); } catch { /* noop */ }
    }
    window.removeEventListener('pointerdown', unlock);
    window.removeEventListener('touchstart', unlock);
  };
  window.addEventListener('pointerdown', unlock, { once: false });
  window.addEventListener('touchstart', unlock, { once: false });

  return () =&gt; {
    cancelled = true;
    stop();
    window.removeEventListener('pointerdown', unlock);
    window.removeEventListener('touchstart', unlock);
    for (const url of audioCache.current.values()) URL.revokeObjectURL(url);
    audioCache.current.clear();
  };
  // eslint-disable-next-line react-hooks/exhaustive-deps
}, []);

const stop = useCallback(() =&gt; {
  if ('speechSynthesis' in window) window.speechSynthesis.cancel();
  if (currentAudio.current) {
    currentAudio.current.pause();
    currentAudio.current = null;
  }
  setIsPlaying(false);
}, []);

const playUrl = useCallback(async (url: string, speed: number) =&gt; {
  const audio = new Audio(url);
  currentAudio.current = audio;
  audio.playbackRate = speed;
  audio.onended = () =&gt; setIsPlaying(false);
  audio.onerror = () =&gt; setIsPlaying(false);
  await audio.play();
}, []);

/** Repli serveur : récupère un WAV depuis /api/tts puis le joue. */
const playServerTTS = useCallback(async (text: string, speed: number) =&gt; {
  try {
    const cacheKey = `${text}__server`;
    let url = audioCache.current.get(cacheKey);
    if (!url) {
      const res = await fetch(`/api/tts?text=${encodeURIComponent(text)}&amp;amp;lang=ar`);
      if (!res.ok) throw new Error(`TTS serveur ${res.status}`);
      const blob = await res.blob();
      url = URL.createObjectURL(blob);
      audioCache.current.set(cacheKey, url);
    }
    await playUrl(url, speed);
  } catch (err) {
    console.error('[TTS] repli serveur échoué :', err);
    setIsPlaying(false);
  }
}, [playUrl]);

/** Synthèse native Android-safe (sans course cancel/speak, avec resume). */
```

```
const playNative = useCallback((text: string, speed: number) =&gt; {
  const u = new SpeechSynthesisUtterance(text);
  u.lang = 'ar-SA';
  u.rate = speed &lt; 1 ? 0.6 : 0.9;
  u.pitch = 1;
  u.volume = 1;
  if (arabicVoice.current) u.voice = arabicVoice.current;

  const inCapacitor = isCapacitorApp();
  // Sur Android, on réduit le délai de garde : si le moteur TTS ne répond pas
  // rapidement dans Capacitor, on ne peut pas tomber sur le serveur (absent).
  const guardMs = inCapacitor ? 4000 : 2500;

  let fellBack = false;
  const guard = setTimeout(() =&gt; {
    if (!fellBack) {
      fellBack = true;
      window.speechSynthesis.cancel();
      if (!inCapacitor) playServerTTS(text, speed);
      else setIsPlaying(false);
    }
  }, guardMs);

  u.onstart = () =&gt; clearTimeout(guard);
  u.onend = () =&gt; { clearTimeout(guard); if (!fellBack) setIsPlaying(false); };
  u.onerror = () =&gt; {
    clearTimeout(guard);
    if (!fellBack) {
      fellBack = true;
      if (!inCapacitor) playServerTTS(text, speed);
      else setIsPlaying(false);
    }
  };

  // IMPORTANT Android : pas de cancel() juste avant ; on speak() puis on resume().
  window.speechSynthesis.speak(u);
  // Android met parfois l'utterance en pause immédiatement → on la relance.
  setTimeout(() =&gt; { try { window.speechSynthesis.resume(); } catch { /* noop */ } }, 80);
}, [playServerTTS]);

const speak = useCallback(async (text: string, speed = 1.0) =&gt; {
  const clean = text?.trim();
  if (!clean) return;
  stop();
  setIsPlaying(true);

  const inCapacitor = isCapacitorApp();

  // 1) Audio pré-généré (le plus fiable, surtout sur Android).
  const preGen = manifest.current[clean];
  if (preGen) {
    try { await playUrl(preGen, speed); return; } catch { /* on continue */ }
  }

  // 2) Voix native Web Speech API.
  //    - Sur le web : uniquement si une voix arabe est détectée.
  //    - Sur Android Capacitor : on tente quand même avec lang='ar-SA' car
  //      le TTS Google peut lire l'arabe sans voix explicite dans la liste.
  const canUseNative =
    'speechSynthesis' in window &amp;&amp;
    (arabicVoice.current !== null || (inCapacitor &amp;&amp; platform.current === 'android'));

  if (canUseNative) {
    try { playNative(clean, speed); return; }
    catch (err) { console.error('[TTS] native échouée :', err); setIsPlaying(false); }
  }

  // 3) Repli serveur — DÉSACTIVÉ dans Capacitor (aucun serveur Express dans l'APK).
  if (!inCapacitor) {
    await playServerTTS(clean, speed);
  } else {
    console.warn('[TTS] Texte absent du manifest et pas de voix arabe installée :', clean);
```

```
      setIsPlaying(false);
    }
  }, [playNative, playServerTTS, playUrl, stop]);

  return { speak, stop, isPlaying, platform: platform.current };
}
```

■ FICHIER : src\index.css

```
@import url('https://fonts.googleapis.com/css2?family=Cairo:wght@400;600;700&amp;family=Noto+Sans+Armenian:wght@4
@import "tailwindcss";

@theme {
  --font-sans: "Noto Sans Armenian", ui-sans-serif, system-ui, sans-serif;
  --font-arabic: "Cairo", sans-serif;
}

body {
  background-color: #e5e7eb; /* gray-200 for outer background */
  margin: 0;
}

#root {
  display: flex;
  flex-direction: column;
  /* Utilise min-height au lieu de height pour permettre le scroll si besoin */
  min-height: 100vh;
  max-width: 1200px;
  margin: 0 auto;
  position: relative;
  /* Supprime overflow: hidden pour permettre le défilement vertical sur mobile */
  overflow-x: hidden;
  background-color: white; /* Fond de l'appli (différent du gris du body) */
}

@media (min-width: 768px) {
  #root {
    padding: 2rem;
    flex-direction: row;
    gap: 20px;
    /* Optionnel : ajoute une ombre pour un effet "carte" sur ordinateur */
    box-shadow: 0 10px 25px -5px rgba(0, 0, 0, 0.1);
    margin-top: 2rem;
    margin-bottom: 2rem;
    border-radius: 12px;
  }
}

@keyframes ripple {
  to {
    transform: scale(4);
    opacity: 0;
  }
}

@utility animate-ripple {
  animation: ripple 0.6s linear forwards;
}

/* ■■■ Flashcard 3D flip ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■ */
@utility perspective-1000 {
  perspective: 1000px;
}

@utility transform-style-3d {
  transform-style: preserve-3d;
}

@utility backface-hidden {
```

```
  backface-visibility: hidden;
}

@utility rotate-y-180 {
  transform: rotateY(180deg);
}
```

■ FICHIER : src\main.tsx

```
import {StrictMode} from 'react';
import {createRoot} from 'react-dom/client';
import App from './App.tsx';
import './index.css';

createRoot(document.getElementById('root')!).render(
  &lt;StrictMode&gt;
    &lt;App /&gt;
  &lt;/StrictMode&gt;,
);
```

■ FICHIER : src\screens\ChatScreen.tsx

```
import React, { useState, useRef, useEffect, useCallback } from 'react';
import {
  Send,
  Bot,
  User,
  Mic,
  Volume2,
  Copy,
  Check,
  Sparkles,
  BookOpen,
  Languages,
  ChevronDown,
  Smile,
  MoreVertical,
  Phone,
  Video,
  ArrowLeft,
  ThumbsUp,
  ThumbsDown,
  Lightbulb,
  GraduationCap,
  Clock,
  Trash2,
  Settings,
  Info,
  MicOff
} from 'lucide-react';
import { useLanguage } from '../contexts/LanguageContext';
import { useArabicTTS } from '../hooks/useArabicTTS';

// ===== Types =====
interface Message {
  id: number;
  text: string;
  sender: 'user' | 'ai';
  timestamp: Date;
  translation?: string;
  transliteration?: string;
  isTyping?: boolean;
  rating?: 'up' | 'down' | null;
  corrections?: string[];
  vocabulary?: VocabWord[];
  audioPlaying?: boolean;
  type?: 'text' | 'quiz' | 'tip' | 'correction';
```

```
}

interface VocabWord {
  arabic: string;
  armenian: string;
  transliteration: string;
}

interface QuickReply {
  text: string;
  arabic: string;
}

// ===== AI Response Logic =====
const SYSTEM_PROMPT = `You are "Armin", a friendly and encouraging Arabic tutor for Armenian speakers.

## Your Role
- Teach Modern Standard Arabic (MSA) to Armenian-speaking beginners (A1 level).
- Always respond in BOTH Arabic and Armenian (████████).
- Keep a warm, patient, and motivating tone — celebrate small wins.

## Response Format (ALWAYS follow this structure)
1. **Arabic text** — written clearly with full harakat (██████) when possible.
2. **Armenian translation** — accurate and natural.
3. **Transliteration** — in Latin letters for pronunciation help.
4. **Vocabulary** — at the end, list 2-4 new words in this format:
   [VOCAB]
   word_arabic | word_armenian | transliteration
   [/VOCAB]

## Teaching Rules
- If the user makes an Arabic error, gently correct it. Show the wrong form ■ then the correct form ✓.
- Use simple sentences. Avoid complex grammar explanations.
- Occasionally ask the user a question to keep them engaged.
- Use emojis sparingly to make responses friendly (■ ■ ■ ■).
- If the user writes in Armenian, respond fully in Armenian + Arabic.
- If the user writes in Arabic (even incorrectly), praise the attempt first.

## Context
The user is learning Arabic through a gamified app. They have completed structured lessons.
This chat is for free practice and questions.`;

// Clé API Gemini exposée via Vite (VITE_ prefix)
const GEMINI_API_KEY = import.meta.env.VITE_GEMINI_API_KEY as string;
const GEMINI_API_URL =
  'https://generativelanguage.googleapis.com/v1beta/models/gemini-2.0-flash:generateContent';

const sendMessageToAI = async (
  userText: string,
  history: Message[]
): Promise&lt;Omit&lt;Message, 'id' | 'timestamp'&gt;&gt; =&gt; {
  if (!GEMINI_API_KEY) {
    throw new Error('VITE_GEMINI_API_KEY manquante dans le fichier .env');
  }

  const contents = [
    ...history.filter(m =&gt; !m.isTyping).map(m =&gt; ({
      role: m.sender === 'user' ? 'user' : 'model',
      parts: [{ text: m.text }],
    })),
    { role: 'user', parts: [{ text: userText }] },
  ];

  const body = {
    system_instruction: { parts: [{ text: SYSTEM_PROMPT }] },
    contents,
    generationConfig: { temperature: 1, maxOutputTokens: 1024 },
  };

  const res = await fetch(`${GEMINI_API_URL}?key=${GEMINI_API_KEY}`, {
    method: 'POST',
    headers: { 'Content-Type': 'application/json' },
```

```
    body: JSON.stringify(body),
  });

  if (!res.ok) {
    let errorMsg = `Gemini API error ${res.status}`;
    try {
      const errorData = await res.json();
      if (errorData.error?.message) errorMsg += `: ${errorData.error.message}`;
    } catch (e) {}
    throw new Error(errorMsg);
  }

  const data = await res.json();
  let text: string =
    data?.candidates?.[0]?.content?.parts?.[0]?.text ?? '...';

  let vocabulary: VocabWord[] | undefined;
  const vocabMatch = text.match(/\[VOCAB\]([\s\S]*?)\[\/VOCAB\]/);
  if (vocabMatch) {
    const vocabLines = vocabMatch[1].trim().split('\n');
    vocabulary = vocabLines.map((line: string) =&gt; {
      const parts = line.split('|').map(p =&gt; p.trim());
      return {
        arabic: parts[0] || '',
        armenian: parts[1] || '',
        transliteration: parts[2] || '',
      };
    }).filter((v: VocabWord) =&gt; v.arabic &amp;&amp; v.armenian);

    // Remove the vocab block from the text
    text = text.replace(/\[VOCAB\][\s\S]*?\[\/VOCAB\]/, '').trim();
  }

  return {
    sender: 'ai',
    text,
    type: 'text',
    vocabulary,
  };
};

// ===== Typing Indicator Component =====
function TypingIndicator() {
  return (
    &lt;div className="flex justify-start"&gt;
      &lt;div className="flex items-end gap-2"&gt;
        &lt;div className="w-8 h-8 rounded-full bg-gradient-to-br from-emerald-400 to-teal-500 flex items-center
          &lt;Bot size={16} className="text-white" /&gt;
        &lt;/div&gt;
        &lt;div className="bg-white border border-gray-100 rounded-2xl rounded-bl-sm px-5 py-3.5 shadow-sm"&gt;
          &lt;div className="flex gap-1.5 items-center"&gt;
            &lt;div className="w-2.5 h-2.5 bg-gray-400 rounded-full animate-bounce" style={{ animationDelay: '0ms
            &lt;div className="w-2.5 h-2.5 bg-gray-400 rounded-full animate-bounce" style={{ animationDelay: '150
            &lt;div className="w-2.5 h-2.5 bg-gray-400 rounded-full animate-bounce" style={{ animationDelay: '300
          &lt;/div&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// ===== Vocabulary Card Component =====
function VocabularyCard({ words }: { words: VocabWord[] }) {
  const [flipped, setFlipped] = useState&lt;number | null&gt;(null);
  const { t } = useLanguage();

  return (
    &lt;div className="mt-3 space-y-2"&gt;
      &lt;p className="text-xs font-semibold text-emerald-700 flex items-center gap-1"&gt;
        &lt;BookOpen size={12} /&gt;
        {t('chat.new_words')}
      &lt;/p&gt;
```

```
      &lt;div className="flex flex-wrap gap-2"&gt;
        {words.map((word, idx) =&gt; (
          &lt;button
            key={idx}
            onClick={() =&gt; setFlipped(flipped === idx ? null : idx)}
            className={`px-3 py-2 rounded-xl text-xs font-medium transition-all duration-300 transform ${
              flipped === idx
                ? 'bg-emerald-500 text-white scale-105 shadow-md'
                : 'bg-emerald-50 text-emerald-800 hover:bg-emerald-100 border border-emerald-200'
            }`}
          &gt;
            {flipped === idx ? (
              &lt;span className="block text-center"&gt;
                &lt;span className="block text-sm"&gt;{word.armenian}&lt;/span&gt;
                &lt;span className="block text-[10px] opacity-80 mt-0.5"&gt;{word.transliteration}&lt;/span&gt;
              &lt;/span&gt;
            ) : (
              &lt;span className="text-sm"&gt;{word.arabic}&lt;/span&gt;
            )}
          &lt;/button&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// ===== Message Bubble Component =====
function MessageBubble({
  message,
  onRate,
  onCopy,
  onSpeak,
}: {
  key?: React.Key;
  message: Message;
  onRate: (id: number, rating: 'up' | 'down') =&gt; void;
  onCopy: (text: string) =&gt; void;
  onSpeak: (text: string) =&gt; void;
}) {
  const [copied, setCopied] = useState(false);
  const [showTranslation, setShowTranslation] = useState(false);
  const { t } = useLanguage();
  const isUser = message.sender === 'user';

  const handleCopy = () =&gt; {
    onCopy(message.text);
    setCopied(true);
    setTimeout(() =&gt; setCopied(false), 2000);
  };

  const getTypeIcon = () =&gt; {
    switch (message.type) {
      case 'quiz':
        return &lt;GraduationCap size={14} className="text-purple-500" /&gt;;
      case 'tip':
        return &lt;Lightbulb size={14} className="text-amber-500" /&gt;;
      case 'correction':
        return &lt;Languages size={14} className="text-red-500" /&gt;;
      default:
        return null;
    }
  };

  const getTypeBadge = () =&gt; {
    switch (message.type) {
      case 'quiz':
        return (
          &lt;span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-[10px] font-bold bg-pu
            &lt;GraduationCap size={10} /&gt; QUIZ
          &lt;/span&gt;
        );
      case 'tip':
```

```
      return (
        &lt;span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-[10px] font-bold bg-am
          &lt;Lightbulb size={10} /&gt; TIP
        &lt;/span&gt;
      );
    case 'correction':
      return (
        &lt;span className="inline-flex items-center gap-1 px-2 py-0.5 rounded-full text-[10px] font-bold bg-re
          &lt;Languages size={10} /&gt; CORRECTION
        &lt;/span&gt;
      );
    default:
      return null;
  }
};

return (
  &lt;div className={`flex ${isUser ? 'justify-end' : 'justify-start'} group`}&gt;
    &lt;div className={`flex items-end gap-2 max-w-[85%] ${isUser ? 'flex-row-reverse' : 'flex-row'}`}&gt;
      {/* Avatar */}
      {!isUser &amp;&amp; (
        &lt;div className="w-8 h-8 rounded-full bg-gradient-to-br from-emerald-400 to-teal-500 flex items-cente
          &lt;Bot size={16} className="text-white" /&gt;
        &lt;/div&gt;
      )}
      {isUser &amp;&amp; (
        &lt;div className="w-8 h-8 rounded-full bg-gradient-to-br from-blue-500 to-indigo-600 flex items-center
          &lt;User size={16} className="text-white" /&gt;
        &lt;/div&gt;
      )}

      &lt;div className="flex flex-col gap-1"&gt;
        {/* Message Bubble */}
        &lt;div
          className={`relative px-4 py-3 rounded-2xl transition-all duration-200 ${
            isUser
              ? 'bg-gradient-to-br from-blue-500 to-indigo-600 text-white rounded-br-sm shadow-md shadow-blue-2
              : 'bg-white border border-gray-100 text-gray-800 rounded-bl-sm shadow-sm'
          }`}
        &gt;
          {!isUser &amp;&amp; getTypeBadge()}
          &lt;p className="text-sm leading-relaxed whitespace-pre-line"&gt;{message.text}&lt;/p&gt;

          {/* Translation toggle */}
          {!isUser &amp;&amp; message.translation &amp;&amp; (
            &lt;button
              onClick={() =&gt; setShowTranslation(!showTranslation)}
              className="mt-2 flex items-center gap-1 text-[11px] text-emerald-600 hover:text-emerald-700 font-
            &gt;
              &lt;Languages size={12} /&gt;
              {showTranslation ? t('chat.hide_translation') : t('chat.show_translation')}
            &lt;/button&gt;
          )}

          {showTranslation &amp;&amp; message.translation &amp;&amp; (
            &lt;div className="mt-2 pt-2 border-t border-gray-100"&gt;
              &lt;p className="text-xs text-gray-500 italic"&gt;{message.translation}&lt;/p&gt;
              {message.transliteration &amp;&amp; (
                &lt;p className="text-[11px] text-gray-400 mt-1 font-mono"&gt;{message.transliteration}&lt;/p&g
              )}
            &lt;/div&gt;
          )}

          {/* Vocabulary */}
          {!isUser &amp;&amp; message.vocabulary &amp;&amp; message.vocabulary.length &gt; 0 &amp;&amp; (
            &lt;VocabularyCard words={message.vocabulary} /&gt;
          )}
        &lt;/div&gt;

        {/* Timestamp &amp; Actions */}
        &lt;div className={`flex items-center gap-2 px-1 ${isUser ? 'justify-end' : 'justify-start'}`}&gt;
          &lt;span className="text-[10px] text-gray-400"&gt;
```

```
            {message.timestamp.toLocaleTimeString('hy-AM', { hour: '2-digit', minute: '2-digit' })}
          &amp;lt;/span&amp;gt;

          {/* Action buttons for AI messages */}
          {!isUser &amp;&amp; (
            &lt;div className="flex items-center gap-1 opacity-0 group-hover:opacity-100 transition-opacity dur
              &lt;button
                onClick={handleCopy}
                className="p-1 rounded-full hover:bg-gray-100 transition-colors"
                title="Copy"
              &gt;
                {copied ? &lt;Check size={12} className="text-green-500" /&gt; : &lt;Copy size={12} className="
              &lt;/button&gt;
              &lt;button
                onClick={() =&gt; onSpeak(message.text)}
                className="p-1 rounded-full hover:bg-gray-100 transition-colors"
                title="Listen"
              &gt;
                &lt;Volume2 size={12} className="text-gray-400" /&gt;
              &lt;/button&gt;
              &lt;button
                onClick={() =&gt; onRate(message.id, 'up')}
                className={`p-1 rounded-full hover:bg-gray-100 transition-colors ${message.rating === 'up' ? 't
              &gt;
                &lt;ThumbsUp size={12} /&gt;
              &lt;/button&gt;
              &lt;button
                onClick={() =&gt; onRate(message.id, 'down')}
                className={`p-1 rounded-full hover:bg-gray-100 transition-colors ${message.rating === 'down' ?
              &gt;
                &lt;ThumbsDown size={12} /&gt;
              &lt;/button&gt;
            &lt;/div&gt;
          )}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
  );
}

// ===== Main Chat Screen =====
export default function ChatScreen() {
  const { t } = useLanguage();
  const { speak } = useArabicTTS();

  // ===== Quick Replies =====
  const quickReplies: QuickReply[] = [
    { text: t('chat.quick_hello'), arabic: '[illegible]' },
    { text: t('chat.quick_thanks'), arabic: '[illegible] [illegible]' },
    { text: t('chat.quick_test'), arabic: '[illegible]' },
    { text: t('chat.quick_help'), arabic: '[illegible]' },
    { text: t('chat.quick_new_words'), arabic: '[illegible] [illegible]' },
    { text: t('chat.quick_correction'), arabic: '[illegible]' },
  ];
  const [messages, setMessages] = useState&lt;Message[]&gt;([
    {
      id: 1,
      text: '[illegible] [illegible] [illegible] [illegible] [illegible] [illegible]!\n\n[illegible] [illegible] [illegible]. [illegible]:\n\n[illegible] [illegible] [illegible] [illegible]
      sender: 'ai',
      timestamp: new Date(),
      translation: '[illegible] [illegible] [illegible] [illegible]:\n\n[illegible] [illegible] [illegible] [illegible] [illegible]:',
      transliteration: 'Marhaban bik fi dars al-lugha al-arabiyya!',
      type: 'text',
      vocabulary: [
        { arabic: '[illegible]', armenian: '[illegible]', transliteration: 'Marhaban' },
        { arabic: '[illegible]', armenian: '[illegible]', transliteration: "Mu'allim" },
        { arabic: '[illegible] [illegible]', armenian: '[illegible]', transliteration: 'Al-lugha al-arabiyya' },
      ],
    },
  ]);
  const [input, setInput] = useState('');
```

```
const [isTyping, setIsTyping] = useState(false);
const [showQuickReplies, setShowQuickReplies] = useState(true);
const [showScrollDown, setShowScrollDown] = useState(false);
const [showMenu, setShowMenu] = useState(false);

const messagesRef = useRef&lt;Message[]&gt;(messages);
useEffect(() =&gt; {
  messagesRef.current = messages;
}, [messages]);

const messagesEndRef = useRef&lt;HTMLDivElement&gt;(null);
const chatContainerRef = useRef&lt;HTMLDivElement&gt;(null);
const inputRef = useRef&lt;HTMLInputElement&gt;(null);

const [isRecording, setIsRecording] = useState(false);
const [isTranscribing, setIsTranscribing] = useState(false);
const mediaRecorderRef = useRef&lt;MediaRecorder | null&gt;(null);
const audioChunksRef = useRef&lt;BlobPart[]&gt;([]);

const handleRecord = async () =&gt; {
  if (isRecording) {
    if (mediaRecorderRef.current &amp;&amp; mediaRecorderRef.current.state === 'recording') {
      mediaRecorderRef.current.stop();
    }
    return;
  }

  setIsRecording(true);
  audioChunksRef.current = [];

  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    const mediaRecorder = new MediaRecorder(stream, { audioBitsPerSecond: 16000 });
    mediaRecorderRef.current = mediaRecorder;

    mediaRecorder.ondataavailable = (event) =&gt; {
      if (event.data.size &gt; 0) {
        audioChunksRef.current.push(event.data);
      }
    };

    mediaRecorder.onstop = () =&gt; {
      setIsRecording(false);
      stream.getTracks().forEach((track) =&gt; track.stop());

      const audioBlob = new Blob(audioChunksRef.current, { type: mediaRecorder.mimeType });
      const reader = new FileReader();
      reader.readAsDataURL(audioBlob);
      reader.onloadend = async () =&gt; {
        const result = reader.result as string;
        if (!result || !result.includes(',')) return;
        const base64Data = result.split(',')[1];

        setIsTranscribing(true);
        try {
           const transcribeBody = {
             contents: [{
               parts: [
                 { text: 'Please transcribe this audio. The expected language is Arabic or Armenian. Only outpu
                 { inline_data: { data: base64Data, mime_type: mediaRecorder.mimeType || 'audio/webm' } }
               ]
             }]
           };
           const res = await fetch(`${GEMINI_API_URL}?key=${GEMINI_API_KEY}`, {
             method: 'POST',
             headers: { 'Content-Type': 'application/json' },
             body: JSON.stringify(transcribeBody),
           });
           const data = await res.json();
           const transcribed = data?.candidates?.[0]?.content?.parts?.[0]?.text?.trim();
           if (transcribed) {
               setInput(prev =&gt; (prev ? prev + ' ' : '') + transcribed);
```

```
          }
        } catch(err) {
          console.error(err);
        } finally {
          setIsTranscribing(false);
        }
      };
    };

    mediaRecorder.start();

    // Limit recording to 15 seconds
    setTimeout(() =&gt; {
      if (mediaRecorderRef.current &amp;&amp; mediaRecorderRef.current.state === 'recording') {
          mediaRecorderRef.current.stop();
      }
    }, 15000);
  } catch (err) {
    console.error('Mic error:', err);
    setIsRecording(false);
  }
};

const scrollToBottom = useCallback(() =&gt; {
  messagesEndRef.current?.scrollIntoView({ behavior: 'smooth' });
}, []);

useEffect(() =&gt; {
  scrollToBottom();
}, [messages, isTyping, scrollToBottom]);

const handleScroll = () =&gt; {
  if (!chatContainerRef.current) return;
  const { scrollTop, scrollHeight, clientHeight } = chatContainerRef.current;
  setShowScrollDown(scrollHeight - scrollTop - clientHeight &gt; 100);
};

const handleSend = async (text?: string) =&gt; {
  const messageText = text || input;
  const trimmed = messageText.trim();
  if (!trimmed) return;

  const userMsg: Message = {
    id: Date.now(),
    text: trimmed,
    sender: 'user',
    timestamp: new Date(),
  };

  setMessages((prev) =&gt; [...prev, userMsg]);
  setInput('');
  setShowQuickReplies(false);
  setIsTyping(true);

  const currentMessages = [...messagesRef.current, userMsg];

  try {
    const aiMsg = await sendMessageToAI(trimmed, currentMessages);
    setMessages((prev) =&gt; [
      ...prev,
      { ...aiMsg, id: Date.now() + 2, timestamp: new Date() },
    ]);
  } catch (err) {
    const errorMsg = err instanceof Error ? err.message : '[redacted], [redacted]: [redacted]';
    setMessages((prev) =&gt; [
      ...prev,
      {
        id: Date.now() + 2,
        sender: 'ai',
        timestamp: new Date(),
        text: `[redacted]: ${errorMsg}`,
        type: 'text',
```

```
      },
    ]);
  } finally {
    setIsTyping(false);
    setShowQuickReplies(true);
  }
};

const handleRate = (id: number, rating: 'up' | 'down') =&gt; {
  setMessages((prev) =&gt; prev.map((msg) =&gt; (msg.id === id ? { ...msg, rating } : msg)));
};

const handleCopy = (text: string) =&gt; {
  navigator.clipboard.writeText(text).catch(() =&gt; {});
};

const handleSpeak = (text: string) =&gt; {
  speak(text, 1.0);
};

const handleClearChat = () =&gt; {
  setMessages([
    {
      id: Date.now(),
      text: '■ ■■ ■■■ ■■■■■■■■■. ■■■■ ■■■■■ ■■ ■■■■■!\n\n■■■■■■■■ ■■■■■■■ ■: ■■■■■ ■■■■■■ ■■■■■■■:',
      sender: 'ai',
      timestamp: new Date(),
      type: 'text',
    },
  ]);
  setShowMenu(false);
};

return (
  &lt;div className="flex-1 flex flex-col bg-gradient-to-b from-gray-50 to-gray-100 relative overflow-hidden pb
    {/* Header */}
    &lt;div className="bg-white/95 backdrop-blur-md px-4 py-3 border-b border-gray-200/50 shadow-sm sticky top-
      &lt;div className="flex items-center justify-between max-w-2xl mx-auto"&gt;
        &lt;div className="flex items-center gap-3"&gt;
        &lt;button className="p-1.5 rounded-full hover:bg-gray-100 transition-colors lg:hidden"&gt;
          &lt;ArrowLeft size={20} className="text-gray-600" /&gt;
        &lt;/button&gt;
        &lt;div className="relative"&gt;
          &lt;div className="w-11 h-11 bg-gradient-to-br from-emerald-400 to-teal-500 rounded-full flex items-c
            &lt;Bot size={22} className="text-white" /&gt;
          &lt;/div&gt;
          &lt;div className="absolute -bottom-0.5 -right-0.5 w-3.5 h-3.5 bg-green-400 rounded-full border-2 bor
        &lt;/div&gt;
        &lt;div&gt;
          &lt;h1 className="font-bold text-gray-800 text-sm flex items-center gap-1.5"&gt;
            {t('chat.title')}
            &lt;Sparkles size={14} className="text-amber-500" /&gt;
          &lt;/h1&gt;
          &lt;p className="text-[11px] text-green-500 font-medium"&gt;{t('chat.status')}&lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div className="flex items-center gap-1"&gt;
        &lt;button className="p-2 rounded-full hover:bg-gray-100 transition-colors"&gt;
          &lt;Phone size={18} className="text-gray-500" /&gt;
        &lt;/button&gt;
        &lt;button className="p-2 rounded-full hover:bg-gray-100 transition-colors"&gt;
          &lt;Video size={18} className="text-gray-500" /&gt;
        &lt;/button&gt;
        &lt;div className="relative"&gt;
          &lt;button
            onClick={() =&gt; setShowMenu(!showMenu)}
            className="p-2 rounded-full hover:bg-gray-100 transition-colors"
          &gt;
            &lt;MoreVertical size={18} className="text-gray-500" /&gt;
          &lt;/button&gt;
          {showMenu &amp;&amp; (
            &lt;&gt;
```

```
          &lt;div className="fixed inset-0 z-40" onClick={() =&gt; setShowMenu(false)} /&gt;
          &lt;div className="absolute right-0 top-full mt-1 w-48 bg-white rounded-xl shadow-xl border borde
            &lt;button
              onClick={handleClearChat}
              className="w-full px-4 py-2.5 text-left text-sm text-gray-700 hover:bg-gray-50 flex items-cen
            &gt;
              &lt;Trash2 size={16} className="text-gray-400" /&gt;
              {t('chat.clear')}
            &lt;/button&gt;
            &lt;button className="w-full px-4 py-2.5 text-left text-sm text-gray-700 hover:bg-gray-50 flex
              &lt;Settings size={16} className="text-gray-400" /&gt;
              {t('chat.settings')}
            &lt;/button&gt;
            &lt;button className="w-full px-4 py-2.5 text-left text-sm text-gray-700 hover:bg-gray-50 flex
              &lt;Info size={16} className="text-gray-400" /&gt;
              {t('chat.help')}
            &lt;/button&gt;
          &lt;/div&gt;
        &lt;/&gt;
      )}
    &lt;/div&gt;
  &lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;

{/* Date Separator */}
&lt;div className="flex justify-center py-3"&gt;
  &lt;span className="bg-white/80 backdrop-blur-sm text-[11px] text-gray-500 px-3 py-1 rounded-full border
    &lt;Clock size={11} /&gt;
    {new Date().toLocaleDateString('hy-AM', {
      weekday: 'long',
      year: 'numeric',
      month: 'long',
      day: 'numeric',
    })}
  &lt;/span&gt;
&lt;/div&gt;

{/* Messages */}
&lt;div
  ref={chatContainerRef}
  onScroll={handleScroll}
  className="flex-1 overflow-y-auto px-4 space-y-4 pb-4 scroll-smooth w-full max-w-2xl mx-auto"
  style={{ scrollbarWidth: 'thin', scrollbarColor: '#d1d5db transparent' }}
&gt;
  {messages.map((msg) =&gt; (
    &lt;MessageBubble
      key={msg.id}
      message={msg}
      onRate={handleRate}
      onCopy={handleCopy}
      onSpeak={handleSpeak}
    /&gt;
  ))}

  {isTyping &amp;&amp; &lt;TypingIndicator /&gt;}

  &lt;div ref={messagesEndRef} /&gt;
&lt;/div&gt;

{/* Scroll to bottom button */}
{showScrollDown &amp;&amp; (
  &lt;button
    onClick={scrollToBottom}
    className="absolute bottom-36 right-4 w-10 h-10 bg-white rounded-full shadow-lg border border-gray-200
  &gt;
    &lt;ChevronDown size={20} className="text-gray-600" /&gt;
  &lt;/button&gt;
)}

{/* Quick Replies */}
{showQuickReplies &amp;&amp; !isTyping &amp;&amp; (
```

```
        &lt;div className="px-4 pb-2 overflow-x-auto w-full max-w-2xl mx-auto"&gt;
          &lt;div className="flex gap-2 pb-1"&gt;
            {quickReplies.map((reply, idx) =&gt; (
              &lt;button
                key={idx}
                onClick={() =&gt; handleSend(reply.arabic)}
                className="flex-shrink-0 px-3.5 py-2 bg-white border border-emerald-200 rounded-full text-xs font
              &gt;
                &lt;span className="block text-sm"&gt;{reply.text}&lt;/span&gt;
              &lt;/button&gt;
            ))}
          &lt;/div&gt;
        &lt;/div&gt;
      )}

      {/* Input Area */}
      &lt;div className="bg-white/95 backdrop-blur-md border-t border-gray-200/50 px-3 py-3 z-20 w-full"&gt;
        &lt;div className="flex items-end gap-2 max-w-2xl mx-auto"&gt;
          &lt;button className="p-2.5 rounded-full hover:bg-gray-100 transition-colors flex-shrink-0 mb-0.5"&gt;
            &lt;Smile size={22} className="text-gray-400" /&gt;
          &lt;/button&gt;
          &lt;div className="flex-1 relative"&gt;
            &lt;input
              ref={inputRef}
              type="text"
              disabled={isTyping}
              value={input}
              onChange={(e) =&gt; setInput(e.target.value)}
              onKeyDown={(e) =&gt; {
                if (e.key === 'Enter' &amp;&amp; !e.shiftKey) {
                  e.preventDefault();
                  handleSend();
                }
              }}
              placeholder={t('chat.input_placeholder')}
              className="w-full bg-gray-100 rounded-2xl px-4 py-3 text-sm focus:ring-2 focus:ring-emerald-500 foc
              dir="auto"
            /&gt;
          &lt;/div&gt;
          {input.trim() ? (
            &lt;button
              onClick={() =&gt; handleSend()}
              disabled={isTyping}
              className={`w-11 h-11 bg-gradient-to-br from-emerald-500 to-teal-600 text-white rounded-full flex i
            &gt;
              &lt;Send size={18} className="ml-0.5" /&gt;
            &lt;/button&gt;
          ) : (
            &lt;button
              onClick={handleRecord}
              disabled={isTyping || isTranscribing}
              className={`w-11 h-11 rounded-full flex items-center justify-center transition-all shadow-md flex-s
              {isTranscribing ? (
                  &lt;div className="w-5 h-5 border-2 border-white border-t-transparent rounded-full animate-spin
              ) : isRecording ? &lt;MicOff size={18} /&gt; : &lt;Mic size={18} /&gt;}
            &lt;/button&gt;
          )}
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\screens\HomeScreen.tsx

```
import React, { useState } from 'react';
import { Star, Check, HelpCircle } from 'lucide-react';
import { useLanguage } from '../contexts/LanguageContext';
import AboutModal from '../components/AboutModal';
```

```
import { lessonsData } from '../data/lessons';

// ■■■ Types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

type NodeStatus = 'completed' | 'current';

interface LessonNodeProps {
  title: string;
  status: NodeStatus;
  onClick: () =&gt; void;
  isRight?: boolean;
  color?: UnitColor;
}

interface UnitSectionProps {
  title: string;
  subtitle: string;
  progress: string;
  color: UnitColor;
  children: React.ReactNode;
}

type UnitColor = 'green' | 'blue' | 'purple' | 'orange';

// ■■■ Color Maps ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

const COLOR_MAP: Record&lt;
  UnitColor,
  {
    header: string;
    headerBorder: string;
    headerText: string;
    nodeBg: string;
    nodeShadow: string;
    currentRing: string;
    currentText: string;
    progressBadge: string;
    progressText: string;
    pathDone: string;
  }
&gt; = {
  green: {
    header: 'bg-green-50 border-green-200',
    headerBorder: 'border-l-4 border-l-green-500',
    headerText: 'text-green-700',
    nodeBg: 'bg-green-500',
    nodeShadow: 'shadow-[0_4px_0_#15803d]',
    currentRing: 'ring-4 ring-green-200 border-green-500',
    currentText: 'text-green-500',
    progressBadge: 'bg-green-100 text-green-700',
    progressText: 'text-green-600',
    pathDone: 'bg-green-400',
  },
  blue: {
    header: 'bg-blue-50 border-blue-200',
    headerBorder: 'border-l-4 border-l-blue-500',
    headerText: 'text-blue-700',
    nodeBg: 'bg-blue-500',
    nodeShadow: 'shadow-[0_4px_0_#1d4ed8]',
    currentRing: 'ring-4 ring-blue-200 border-blue-500',
    currentText: 'text-blue-500',
    progressBadge: 'bg-blue-100 text-blue-700',
    progressText: 'text-blue-600',
    pathDone: 'bg-blue-400',
  },
  purple: {
    header: 'bg-purple-50 border-purple-200',
    headerBorder: 'border-l-4 border-l-purple-500',
    headerText: 'text-purple-700',
    nodeBg: 'bg-purple-500',
    nodeShadow: 'shadow-[0_4px_0_#7e22ce]',
    currentRing: 'ring-4 ring-purple-200 border-purple-500',
```

```
    currentText: 'text-purple-500',
    progressBadge: 'bg-purple-100 text-purple-700',
    progressText: 'text-purple-600',
    pathDone: 'bg-purple-400',
  },
  orange: {
    header: 'bg-orange-50 border-orange-200',
    headerBorder: 'border-l-4 border-l-orange-500',
    headerText: 'text-orange-700',
    nodeBg: 'bg-orange-500',
    nodeShadow: 'shadow-[0_4px_0_#c2410c]',
    currentRing: 'ring-4 ring-orange-200 border-orange-500',
    currentText: 'text-orange-500',
    progressBadge: 'bg-orange-100 text-orange-700',
    progressText: 'text-orange-600',
    pathDone: 'bg-orange-400',
  },
};

// ■■■ HomeScreen ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

export default function HomeScreen({
  onStartLesson,
  completedUnits,
  totalXP,
  streak,
}: {
  onStartLesson: (id: string) =&gt; void;
  completedUnits: string[];
  totalXP: number;
  streak: number;
}) {
  const { t } = useLanguage();
  const [showAboutModal, setShowAboutModal] = useState(false);

  const getStatus = (id: string): NodeStatus =&gt; {
    if (completedUnits.includes(id)) return 'completed';
    return 'current';
  };

  return (
    &lt;div className="flex-1 overflow-y-auto pb-24 md:pb-6 bg-gray-50 relative"&gt;
      {showAboutModal &amp;&amp; &lt;AboutModal onClose={() =&gt; setShowAboutModal(false)} /&gt;}

      {/* ■■ Header ■■ */}
      &lt;div className="bg-green-600 text-white px-6 pt-10 pb-8 md:rounded-b-3xl shadow-lg sticky top-0 z-10 w-f
        &lt;div className="flex justify-between items-center max-w-2xl mx-auto"&gt;
          &lt;div className="flex items-center gap-3"&gt;
            &lt;div&gt;
              &lt;h1 className="text-[clamp(1.5rem,4vw,2.25rem)] font-bold tracking-tight"&gt;{t('home.title')}&l
              &lt;p className="text-green-200 text-[clamp(0.875rem,2vw,1rem)] mt-0.5"&gt;{t('home.subtitle')}&lt;
            &lt;/div&gt;
            &lt;button
              onClick={() =&gt; setShowAboutModal(true)}
              className="mt-1 flex items-center justify-center w-8 h-8 rounded-full bg-white/20 hover:bg-white/30
              aria-label="About the App"
            &gt;
              &lt;HelpCircle size={18} className="text-white" /&gt;
            &lt;/button&gt;
          &lt;/div&gt;
          &lt;div className="flex items-center gap-3"&gt;
            {/* XP pill */}
            &lt;div className="bg-white/15 px-3 py-1.5 rounded-full flex items-center gap-1.5 text-sm font-semibo
              &lt;span className="text-yellow-300"&gt;■&lt;/span&gt;
              &lt;span&gt;{totalXP} XP&lt;/span&gt;
            &lt;/div&gt;
            {/* Streak pill */}
            &lt;div className="bg-white/15 px-3 py-1.5 rounded-full flex items-center gap-1.5 text-sm font-semibo
              &lt;span className="text-orange-300"&gt;■&lt;/span&gt;
              &lt;span&gt;{streak}&lt;/span&gt;
            &lt;/div&gt;
          &lt;/div&gt;
```

```
  &lt;/div&gt;

  {/* Progress bar */}
  &lt;div className="mt-4 bg-white/20 rounded-full h-2 max-w-2xl mx-auto"&gt;
    &lt;div
      className="bg-white rounded-full h-2 transition-all duration-500"
      style={{ width: `${(completedUnits.length / 22) * 100}%` }}
    /&gt;
  &lt;/div&gt;
  &lt;p className="text-green-200 text-xs mt-1.5 max-w-2xl mx-auto"&gt;
    {completedUnits.length} / 22 {t('home.lessons_completed')}
  &lt;/p&gt;
&lt;/div&gt;

&lt;div className="p-5 space-y-8 max-w-2xl mx-auto w-full"&gt;
  {/* ■■ Unit 1 ■■ */}
  &lt;UnitSection
    title={t('home.unit1.title')}
    subtitle={t('home.unit1.subtitle')}
    progress={completedUnits.includes('u1') ? '1/1' : '0/1'}
    color="green"
  &gt;
    &lt;LessonNode title={t('home.node.u1')} status={getStatus('u1')} onClick={() =&gt; onStartLesson('u1')
  &lt;/UnitSection&gt;

  {/* ■■ Unit 2 ■■ */}
  &lt;UnitSection
    title={t('home.unit2.title')}
    subtitle={t('home.unit2.subtitle')}
    progress={completedUnits.includes('u2') ? '1/1' : '0/1'}
    color="blue"
  &gt;
    &lt;LessonNode title={t('home.node.u2')} status={getStatus('u2')} onClick={() =&gt; onStartLesson('u2')
  &lt;/UnitSection&gt;

  {/* ■■ Unit 3 ■■ */}
  &lt;UnitSection
    title={t('home.unit3.title')}
    subtitle={t('home.unit3.subtitle')}
    progress={completedUnits.includes('u3') ? '1/1' : '0/1'}
    color="purple"
  &gt;
    &lt;LessonNode
      title={t('home.node.u3')}
      status={getStatus('u3')}
      onClick={() =&gt; onStartLesson('u3')}
      color="purple"
    /&gt;
  &lt;/UnitSection&gt;

  {/* ■■ Unit 4 ■■ */}
  &lt;UnitSection
    title={t('home.unit4.title')}
    subtitle={t('home.unit4.subtitle')}
    progress={completedUnits.includes('u4') ? '1/1' : '0/1'}
    color="orange"
  &gt;
    &lt;LessonNode title={t('home.node.u4')} status={getStatus('u4')} onClick={() =&gt; onStartLesson('u4')
  &lt;/UnitSection&gt;

  {/* ■■ Unit 5 ■■ */}
  &lt;UnitSection
    title={t('home.unit5.title')}
    subtitle={t('home.unit5.subtitle')}
    progress={completedUnits.includes('u5') ? '1/1' : '0/1'}
    color="green"
  &gt;
    &lt;LessonNode title={t('home.node.u5')} status={getStatus('u5')} onClick={() =&gt; onStartLesson('u5')
  &lt;/UnitSection&gt;

  {/* ■■ Unit 6 ■■ */}
  &lt;UnitSection
```

```
  title={t('home.unit6.title')}
  subtitle={t('home.unit6.subtitle')}
  progress={completedUnits.includes('u6') ? '1/1' : '0/1'}
  color="blue"
&gt;
  &lt;LessonNode title={t('home.node.u6')} status={getStatus('u6')} onClick={() =&gt; onStartLesson('u6')
&lt;/UnitSection&gt;

{/* ■■ Unit 7 ■■ */}
&lt;UnitSection
  title={t('home.unit7.title')}
  subtitle={t('home.unit7.subtitle')}
  progress={completedUnits.includes('u7') ? '1/1' : '0/1'}
  color="purple"
&gt;
  &lt;LessonNode title={t('home.node.u7')} status={getStatus('u7')} onClick={() =&gt; onStartLesson('u7')
&lt;/UnitSection&gt;

{/* ■■ Unit 8 ■■ */}
&lt;UnitSection
  title={t('home.unit8.title')}
  subtitle={t('home.unit8.subtitle')}
  progress={completedUnits.includes('u8') ? '1/1' : '0/1'}
  color="orange"
&gt;
  &lt;LessonNode title={t('home.node.u8')} status={getStatus('u8')} onClick={() =&gt; onStartLesson('u8')
&lt;/UnitSection&gt;

{/* ■■ Unit 9 ■■ */}
&lt;UnitSection
  title={t('home.unit9.title')}
  subtitle={t('home.unit9.subtitle')}
  progress={completedUnits.includes('u9') ? '1/1' : '0/1'}
  color="green"
&gt;
  &lt;LessonNode title={t('home.node.u9')} status={getStatus('u9')} onClick={() =&gt; onStartLesson('u9')
&lt;/UnitSection&gt;

{/* ■■ Unit 10 ■■ */}
&lt;UnitSection
  title={t('home.unit10.title')}
  subtitle={t('home.unit10.subtitle')}
  progress={completedUnits.includes('u10') ? '1/1' : '0/1'}
  color="blue"
&gt;
  &lt;LessonNode title={t('home.node.u10')} status={getStatus('u10')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 11 ■■ */}
&lt;UnitSection
  title={t('home.unit11.title')}
  subtitle={t('home.unit11.subtitle')}
  progress={completedUnits.includes('u11') ? '1/1' : '0/1'}
  color="purple"
&gt;
  &lt;LessonNode title={t('home.node.u11')} status={getStatus('u11')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 12 ■■ */}
&lt;UnitSection
  title={t('home.unit12.title')}
  subtitle={t('home.unit12.subtitle')}
  progress={completedUnits.includes('u12') ? '1/1' : '0/1'}
  color="orange"
&gt;
  &lt;LessonNode title={t('home.node.u12')} status={getStatus('u12')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 13 ■■ */}
&lt;UnitSection
  title={t('home.unit13.title')}
  subtitle={t('home.unit13.subtitle')}
```

```
  progress={completedUnits.includes('u13') ? '1/1' : '0/1'}
  color="green"
&gt;
  &lt;LessonNode title={t('home.node.u13')} status={getStatus('u13')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 14 ■■ */}
&lt;UnitSection
  title={t('home.unit14.title')}
  subtitle={t('home.unit14.subtitle')}
  progress={completedUnits.includes('u14') ? '1/1' : '0/1'}
  color="blue"
&gt;
  &lt;LessonNode title={t('home.node.u14')} status={getStatus('u14')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 15 ■■ */}
&lt;UnitSection
  title={t('home.unit15.title')}
  subtitle={t('home.unit15.subtitle')}
  progress={completedUnits.includes('u15') ? '1/1' : '0/1'}
  color="purple"
&gt;
  &lt;LessonNode title={t('home.node.u15')} status={getStatus('u15')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 16 ■■ */}
&lt;UnitSection
  title={t('home.unit16.title')}
  subtitle={t('home.unit16.subtitle')}
  progress={completedUnits.includes('u16') ? '1/1' : '0/1'}
  color="orange"
&gt;
  &lt;LessonNode title={t('home.node.u16')} status={getStatus('u16')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 17 ■■ */}
&lt;UnitSection
  title={t('home.unit17.title')}
  subtitle={t('home.unit17.subtitle')}
  progress={completedUnits.includes('u17') ? '1/1' : '0/1'}
  color="green"
&gt;
  &lt;LessonNode title={t('home.node.u17')} status={getStatus('u17')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 18 ■■ */}
&lt;UnitSection
  title={t('home.unit18.title')}
  subtitle={t('home.unit18.subtitle')}
  progress={completedUnits.includes('u18') ? '1/1' : '0/1'}
  color="blue"
&gt;
  &lt;LessonNode title={t('home.node.u18')} status={getStatus('u18')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 19 ■■ */}
&lt;UnitSection
  title={t('home.unit19.title')}
  subtitle={t('home.unit19.subtitle')}
  progress={completedUnits.includes('u19') ? '1/1' : '0/1'}
  color="purple"
&gt;
  &lt;LessonNode title={t('home.node.u19')} status={getStatus('u19')} onClick={() =&gt; onStartLesson('u1
&lt;/UnitSection&gt;

{/* ■■ Unit 20 ■■ */}
&lt;UnitSection
  title={t('home.unit20.title')}
  subtitle={t('home.unit20.subtitle')}
  progress={completedUnits.includes('u20') ? '1/1' : '0/1'}
  color="orange"
```

```
        &gt;
          &lt;LessonNode title={t('home.node.u20')} status={getStatus('u20')} onClick={() =&gt; onStartLesson('u2
        &lt;/UnitSection&gt;

        {/* ■■ Unit 21 ■■ */}
        &lt;UnitSection
          title={t('home.unit21.title')}
          subtitle={t('home.unit21.subtitle')}
          progress={completedUnits.includes('u21') ? '1/1' : '0/1'}
          color="green"
        &gt;
          &lt;LessonNode title={t('home.node.u21')} status={getStatus('u21')} onClick={() =&gt; onStartLesson('u2
        &lt;/UnitSection&gt;

        {/* ■■ Unit 22 ■■ */}
        &lt;UnitSection
          title={t('home.unit22.title')}
          subtitle={t('home.unit22.subtitle')}
          progress={completedUnits.includes('u22') ? '1/1' : '0/1'}
          color="blue"
        &gt;
          &lt;LessonNode title={t('home.node.u22')} status={getStatus('u22')} onClick={() =&gt; onStartLesson('u2
        &lt;/UnitSection&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// ■■■ UnitSection ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

function UnitSection({ title, subtitle, progress, color, children }: UnitSectionProps) {
  const { t } = useLanguage();
  const theme = COLOR_MAP[color];
  const [done, total] = progress.split('/').map(Number);
  const isComplete = done === total;

  return (
    &lt;div&gt;
      {/* Section header card */}
      &lt;div
        className={`rounded-2xl border px-4 py-3 mb-5 ${theme.header} ${theme.headerBorder}`}
      &gt;
        &lt;div className="flex items-center justify-between"&gt;
          &lt;div&gt;
            &lt;h2 className={`text-base font-bold ${theme.headerText}`}&gt;{title}&lt;/h2&gt;
            &lt;p className="text-xs text-gray-500 mt-0.5"&gt;{subtitle}&lt;/p&gt;
          &lt;/div&gt;
          &lt;div className="flex items-center gap-2"&gt;
            {isComplete &amp;&amp; (
              &lt;span className="text-xs bg-white border border-green-300 text-green-600 px-2 py-0.5 rounded-ful
                ✓ {t('home.completed')}
              &lt;/span&gt;
            )}
            &lt;span
              className={`text-xs font-bold px-2.5 py-1 rounded-full ${theme.progressBadge}`}
            &gt;
              {progress}
            &lt;/span&gt;
          &lt;/div&gt;
        &lt;/div&gt;

        {/* Mini progress bar */}
        &lt;div className="mt-2 bg-white/60 rounded-full h-1.5"&gt;
          &lt;div
            className={`${theme.nodeBg} rounded-full h-1.5 transition-all duration-700`}
            style={{ width: total &gt; 0 ? `${(done / total) * 100}%` : '0%' }}
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Zigzag path */}
      &lt;div className="relative flex flex-col items-center gap-7"&gt;
```

```
        {/* Vertical guide line */}
        &lt;div className="absolute top-8 bottom-8 w-1.5 bg-gray-200 rounded-full -z-10" /&gt;

        {React.Children.map(children, (child) =&gt; child)}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// ■■■ LessonNode ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

function LessonNode({
  title,
  status,
  onClick,
  isRight = false,
  color = 'green',
}: LessonNodeProps) {
  const { t } = useLanguage();
  const theme = COLOR_MAP[color];

  const buttonClass = status === 'completed'
    ? `${theme.nodeBg} ${theme.nodeShadow} text-white active:shadow-none active:translate-y-1`
    : `bg-white border-2 ${theme.currentRing} ${theme.currentText} animate-bounce`;

  const icon = status === 'completed'
    ? &lt;Check strokeWidth={3} size={26} /&gt;
    : &lt;Star fill="currentColor" size={26} /&gt;;

  return (
    &lt;div
      className={`relative flex items-center w-full ${
        isRight ? 'justify-end pr-[15%]' : 'justify-start pl-[15%]'
      }`}
    &gt;
      &lt;div className="flex flex-col items-center gap-2"&gt;
        &lt;button
          onClick={onClick}
          aria-label={`${title} – ${status}`}
          className={`
            w-[60px] h-[60px] rounded-full flex items-center justify-center
            transition-transform duration-150
            hover:scale-105 active:scale-95
            ${buttonClass}
          `}
          style={status === 'current' ? { animationDuration: '2.5s' } : undefined}
        &gt;
          {icon}
        &lt;/button&gt;

        &lt;span className="text-[11px] font-bold text-center leading-tight max-w-[80px] text-gray-700"&gt;
          {title}
        &lt;/span&gt;

        {/* "START" badge on current node */}
        {status === 'current' &amp;&amp; (
          &lt;span
            className={`text-[10px] font-bold uppercase tracking-wide px-2 py-0.5 rounded-full ${theme.progressBa
          &gt;
            {t('home.start')}
          &lt;/span&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\screens\LessonScreen.tsx

```
import React, { useState, useEffect, useCallback, useRef, useId, useMemo } from 'react';
import {
  X,
  Mic,
  MicOff,
  Play,
  Pause,
  CheckCircle,
  AlertCircle,
  Turtle,
  Volume2,
  ChevronRight,
  ChevronLeft,
  Star,
  Trophy,
  Heart,
  Zap,
  RotateCcw,
  BookOpen,
  Eye,
  EyeOff,
  Lightbulb,
  ArrowRight,
  Sparkles,
  Target,
  Award,
  Clock,
  Hash,
  Edit3,
  Link2,
  type LucideIcon,
} from 'lucide-react';
import { useLanguage } from '../contexts/LanguageContext';
import { useArabicTTS } from '../hooks/useArabicTTS';
import { lessonsData, type LessonData, type LessonStep, type QuizOption } from '../data/lessons';

// ===== Sound Effects (Real Web Audio + Vibration) =====
const lazyAudioContext = (() =&gt; {
  let ctx: AudioContext | null = null;
  return () =&gt; {
    if (!ctx) {
      ctx = new (window.AudioContext || (window as Record&lt;string, any&gt;).webkitAudioContext)();
    }
    return ctx;
  };
})();

const playSound = (type: 'correct' | 'wrong' | 'complete' | 'click' | 'speak') =&gt; {
  try {
    if ('vibrate' in navigator) {
      switch (type) {
        case 'correct': navigator.vibrate(50); break;
        case 'wrong': navigator.vibrate([100, 50, 100]); break;
        case 'complete': navigator.vibrate([50, 30, 50, 30, 50]); break;
        default: navigator.vibrate(20);
      }
    }
  } catch (e) {}

  try {
    const ctx = lazyAudioContext();
    if (ctx.state === 'suspended') ctx.resume();

    const t = ctx.currentTime;
    const osc = ctx.createOscillator();
    const gain = ctx.createGain();

    osc.connect(gain);
    gain.connect(ctx.destination);

    if (type === 'correct') {
      osc.type = 'sine';
```

```
      osc.frequency.setValueAtTime(523.25, t); // Do5
      osc.frequency.setValueAtTime(659.25, t + 0.1); // Mi5
      osc.frequency.setValueAtTime(783.99, t + 0.2); // Sol5
      gain.gain.setValueAtTime(0, t);
      gain.gain.linearRampToValueAtTime(0.3, t + 0.05);
      gain.gain.linearRampToValueAtTime(0, t + 0.3);
      osc.start(t);
      osc.stop(t + 0.3);
    } else if (type === 'wrong') {
      osc.type = 'sawtooth';
      osc.frequency.setValueAtTime(300, t);
      osc.frequency.linearRampToValueAtTime(150, t + 0.3);
      gain.gain.setValueAtTime(0, t);
      gain.gain.linearRampToValueAtTime(0.3, t + 0.05);
      gain.gain.linearRampToValueAtTime(0, t + 0.3);
      osc.start(t);
      osc.stop(t + 0.3);
    } else if (type === 'complete') {
      osc.type = 'square';
      osc.frequency.setValueAtTime(440, t);
      osc.frequency.setValueAtTime(554.37, t + 0.15);
      osc.frequency.setValueAtTime(659.25, t + 0.3);
      osc.frequency.setValueAtTime(880, t + 0.45);
      gain.gain.setValueAtTime(0, t);
      gain.gain.linearRampToValueAtTime(0.15, t + 0.05);
      gain.gain.linearRampToValueAtTime(0, t + 0.6);
      osc.start(t);
      osc.stop(t + 0.6);
    } else if (type === 'click') {
      osc.type = 'sine';
      osc.frequency.setValueAtTime(800, t);
      gain.gain.setValueAtTime(0, t);
      gain.gain.linearRampToValueAtTime(0.1, t + 0.02);
      gain.gain.linearRampToValueAtTime(0, t + 0.08);
      osc.start(t);
      osc.stop(t + 0.08);
    } else if (type === 'speak') {
      osc.type = 'sine';
      osc.frequency.setValueAtTime(600, t);
      osc.frequency.setValueAtTime(800, t + 0.1);
      gain.gain.setValueAtTime(0, t);
      gain.gain.linearRampToValueAtTime(0.1, t + 0.05);
      gain.gain.linearRampToValueAtTime(0, t + 0.2);
      osc.start(t);
      osc.stop(t + 0.2);
    }
  } catch (e) {}
};

// ===== Animated Counter Component =====
function AnimatedCounter({ value, duration = 500 }: { value: number; duration?: number }) {
  const [display, setDisplay] = useState(0);
  const previousValue = useRef(0);

  useEffect(() =&gt; {
    const start = previousValue.current;
    previousValue.current = value;
    if (start === value) return;

    const steps = Math.ceil(duration / 16);
    let step = 0;
    const timer = setInterval(() =&gt; {
      step++;
      const progress = step / steps;
      setDisplay(Math.round(start + (value - start) * Math.min(progress, 1)));
      if (step &gt;= steps) clearInterval(timer);
    }, 16);

    return () =&gt; clearInterval(timer);
  }, [value, duration]);

  return &lt;span&gt;{display}&lt;/span&gt;;
```

```
}

// ===== Circular Progress Component =====
function CircularProgress({ progress, size = 40, strokeWidth = 3 }: { progress: number; size?: number; strokeWidt
  const gradientId = useId();
  const radius = (size - strokeWidth) / 2;
  const circumference = radius * 2 * Math.PI;
  const offset = circumference - (progress / 100) * circumference;

  return (
    &lt;svg width={size} height={size} className="transform -rotate-90"&gt;
      &lt;circle cx={size / 2} cy={size / 2} r={radius} stroke="#e5e7eb" strokeWidth={strokeWidth} fill="none" /&
      &lt;circle
        cx={size / 2}
        cy={size / 2}
        r={radius}
        stroke={`url(#${gradientId})`}
        strokeWidth={strokeWidth}
        fill="none"
        strokeDasharray={circumference}
        strokeDashoffset={offset}
        strokeLinecap="round"
        className="transition-all duration-700 ease-out"
      /&gt;
      &lt;defs&gt;
        &lt;linearGradient id={gradientId} x1="0%" y1="0%" x2="100%" y2="0%"&gt;
          &lt;stop offset="0%" stopColor="#10b981" /&gt;
          &lt;stop offset="100%" stopColor="#059669" /&gt;
        &lt;/linearGradient&gt;
      &lt;/defs&gt;
    &lt;/svg&gt;
  );
}

// ===== Particle Effect Component =====
function ParticleExplosion({ active }: { active: boolean }) {
  const particles = useMemo(() =&gt; {
    return Array.from({ length: 20 }).map((_, i) =&gt; {
      const angle = (i / 20) * 360;
      const distance = 60 + Math.random() * 100;
      const x = Math.cos((angle * Math.PI) / 180) * distance;
      const y = Math.sin((angle * Math.PI) / 180) * distance;
      const colors = ['■', '■', '■', '■', '■', '■'];
      const emoji = colors[Math.floor(Math.random() * colors.length)];
      const size = 14 + Math.random() * 16;
      const duration = 0.6 + Math.random() * 0.5;
      return { x, y, emoji, size, duration };
    });
  }, []);

  if (!active) return null;

  return (
    &lt;div className="fixed inset-0 pointer-events-none z-50"&gt;
      {particles.map((p, i) =&gt; (
        &lt;div
          key={i}
          className="absolute left-1/2 top-1/2"
          style={{
            fontSize: p.size,
            animation: `particleFly ${p.duration}s ease-out forwards`,
            '--tx': `${p.x}px`,
            '--ty': `${p.y}px`,
            opacity: 0,
          } as React.CSSProperties}
        &gt;
          {p.emoji}
        &lt;/div&gt;
      ))}
      &lt;style&gt;{`
        @keyframes particleFly {
          0% { opacity: 1; transform: translate(0, 0) scale(0.5); }
```

```
          100% { opacity: 0; transform: translate(var(--tx, 50px), var(--ty, -50px)) scale(1.2); }
        }
      `}&lt;/style&gt;
    &lt;/div&gt;
  );
}

// ===== Hearts Display =====
function HeartsDisplay({ lives, maxLives = 3 }: { lives: number; maxLives?: number }) {
  return (
    &lt;div className="flex items-center gap-0.5"&gt;
      {Array.from({ length: maxLives }).map((_, i) =&gt; (
        &lt;Heart
          key={i}
          size={18}
          className={`transition-all duration-300 ${
            i &lt; lives ? 'text-red-500 fill-red-500 scale-100' : 'text-gray-300 scale-90'
          } ${i === lives ? 'animate-pulse' : ''}`}
        /&gt;
      ))}
    &lt;/div&gt;
  );
}

// ===== Waveform Animation =====
function WaveformAnimation({ active }: { active: boolean }) {
  const heights = useMemo(
    () =&gt; Array.from({ length: 20 }, () =&gt; 20 + Math.random() * 40),
    []
  );

  return (
    &lt;div className="flex items-center justify-center gap-1 h-16"&gt;
      {heights.map((h, i) =&gt; (
        &lt;div
          key={i}
          className={`w-1 rounded-full transition-all duration-150 ${
            active ? 'bg-gradient-to-t from-red-500 to-red-300' : 'bg-gray-300'
          }`}
          style={{
            height: active ? `${h}px` : '8px',
            animation: active ? `wave 0.5s ease-in-out ${i * 0.05}s infinite alternate` : 'none',
          }}
        /&gt;
      ))}
      &lt;style&gt;{`
        @keyframes wave {
          0% { height: 8px; }
          100% { height: 40px; }
        }
      `}&lt;/style&gt;
    &lt;/div&gt;
  );
}

// ===== Completion Screen =====
function CompletionScreen({
  xpEarned,
  accuracy,
  timeStr,
  onContinue,
}: {
  xpEarned: number;
  accuracy: number;
  timeStr: string;
  onContinue: () =&gt; void;
}) {
  const { t } = useLanguage();
  const [showStars, setShowStars] = useState(0);
  const [showXP, setShowXP] = useState(false);
  const [celebrate, setCelebrate] = useState(false);
```

```
const starCount = accuracy &gt;= 90 ? 3 : accuracy &gt;= 70 ? 2 : 1;

useEffect(() =&gt; {
  const timers = [
    setTimeout(() =&gt; setShowStars(1), 400),
    setTimeout(() =&gt; setShowStars(2), 800),
    setTimeout(() =&gt; setShowStars(3), 1200),
    setTimeout(() =&gt; setShowXP(true), 1600),
    setTimeout(() =&gt; setCelebrate(true), 500),
  ];
  playSound('complete');
  return () =&gt; timers.forEach(clearTimeout);
}, []);

return (
  &lt;div className="fixed inset-0 bg-gradient-to-b from-emerald-500 via-teal-500 to-cyan-600 z-50 flex flex-co
    &lt;ParticleExplosion active={celebrate} /&gt;

    &lt;div className="text-center space-y-8 animate-in fade-in zoom-in duration-500"&gt;
      {/* Trophy */}
      &lt;div className="relative"&gt;
        &lt;div className="w-28 h-28 bg-yellow-400/20 rounded-full flex items-center justify-center mx-auto bac
          &lt;Trophy size={56} className="text-yellow-300 drop-shadow-lg" /&gt;
        &lt;/div&gt;
        &lt;div className="absolute -top-2 -right-2"&gt;
          &lt;Sparkles size={28} className="text-yellow-300 animate-pulse" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Title */}
      &lt;div&gt;
        &lt;h1 className="text-3xl font-black text-white mb-2 drop-shadow-md"&gt;
          {t('lesson.lesson_complete')}
        &lt;/h1&gt;
        &lt;p className="text-emerald-100 text-lg"&gt;{t('lesson.lesson_complete_desc')}&lt;/p&gt;
      &lt;/div&gt;

      {/* Stars */}
      &lt;div className="flex items-center justify-center gap-3"&gt;
        {[1, 2, 3].map((i) =&gt; (
          &lt;div
            key={i}
            className={`transition-all duration-500 ${
              i &lt;= showStars
                ? 'scale-100 opacity-100'
                : 'scale-0 opacity-0'
            }`}
            style={{ transitionDelay: `${i * 200}ms` }}
          &gt;
            &lt;Star
              size={i === 2 ? 56 : 44}
              className={`${
                i &lt;= starCount
                  ? 'text-yellow-300 fill-yellow-300 drop-shadow-lg'
                  : 'text-white/30'
              } ${i === 2 ? '-mt-4' : ''}`}
            /&gt;
          &lt;/div&gt;
        ))}
      &lt;/div&gt;

      {/* Stats */}
      {showXP &amp;&amp; (
        &lt;div className="space-y-4 animate-in fade-in slide-in-from-bottom-4 duration-500"&gt;
          &lt;div className="bg-white/15 backdrop-blur-md rounded-2xl p-5 space-y-4 border border-white/20"&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;div className="flex items-center gap-2 text-white/90"&gt;
                &lt;Zap size={20} className="text-yellow-300" /&gt;
                &lt;span className="font-medium"&gt;{t('lesson.xp_earned')}&lt;/span&gt;
              &lt;/div&gt;
              &lt;span className="text-2xl font-black text-yellow-300"&gt;
                +&lt;AnimatedCounter value={xpEarned} /&gt;
```

```
              &lt;/span&gt;
            &lt;/div&gt;
            &lt;div className="h-px bg-white/20" /&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;div className="flex items-center gap-2 text-white/90"&gt;
                &lt;Target size={20} className="text-emerald-300" /&gt;
                &lt;span className="font-medium"&gt;{t('lesson.accuracy')}&lt;/span&gt;
              &lt;/div&gt;
              &lt;span className="text-2xl font-black text-emerald-300"&gt;
                &lt;AnimatedCounter value={accuracy} /&gt;%
              &lt;/span&gt;
            &lt;/div&gt;
            &lt;div className="h-px bg-white/20" /&gt;
            &lt;div className="flex items-center justify-between"&gt;
              &lt;div className="flex items-center gap-2 text-white/90"&gt;
                &lt;Clock size={20} className="text-blue-300" /&gt;
                &lt;span className="font-medium"&gt;{t('lesson.time')}&lt;/span&gt;
              &lt;/div&gt;
              &lt;span className="text-lg font-bold text-blue-300"&gt;{timeStr}&lt;/span&gt;
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;button
            onClick={onContinue}
            className="w-full py-4 bg-white text-emerald-600 rounded-2xl font-bold text-lg hover:bg-emerald-50
          &gt;
            {t('lesson.continue')}
          &lt;/button&gt;
        &lt;/div&gt;
      )}
    &lt;/div&gt;
  &lt;/div&gt;
 );
}

// ===== Main Lesson Screen =====
const normalize = (s: string) =&gt;
  s.trim()
   .replace(/[\u200f\u200e\u200b]/g, '')   // remove directional marks
   .replace(/[\u064B-\u065F]/g, '')         // strip all tashkeel (diacritics)
   .replace(/[.,!?■'"]/g, '')               // remove punctuation
   .replace(/\s+/g, ' ')
   .toLowerCase();

function getSimilarity(s1: string, s2: string): number {
  let longer = s1;
  let shorter = s2;
  if (s1.length &lt; s2.length) {
    longer = s2;
    shorter = s1;
  }
  const longerLength = longer.length;
  if (longerLength === 0) {
    return 1.0;
  }
  return (longerLength - editDistance(longer, shorter)) / parseFloat(longerLength.toString());
}

function editDistance(s1: string, s2: string): number {
  const costs = [];
  for (let i = 0; i &lt;= s1.length; i++) {
    let lastValue = i;
    for (let j = 0; j &lt;= s2.length; j++) {
      if (i === 0) costs[j] = j;
      else {
        if (j &gt; 0) {
          let newValue = costs[j - 1];
          if (s1.charAt(i - 1) !== s2.charAt(j - 1)) {
            newValue = Math.min(Math.min(newValue, lastValue), costs[j]) + 1;
          }
          costs[j - 1] = lastValue;
          lastValue = newValue;
```

```
        }
      }
    }
    if (i &gt; 0) costs[s2.length] = lastValue;
  }
  return costs[s2.length];
}

// Choisit le premier conteneur audio supporté par le navigateur courant.
function pickMimeType(): string | undefined {
  if (typeof MediaRecorder === 'undefined') return undefined;
  const candidates = [
    'audio/webm;codecs=opus', // Chrome / Android
    'audio/webm',
    'audio/mp4',              // Safari iOS 14.3+
    'audio/ogg;codecs=opus',
  ];
  return candidates.find((t) =&gt; MediaRecorder.isTypeSupported?.(t));
}

export default function LessonScreen({
  onBack,
  lessonId,
  onComplete,
}: {
  onBack: () =&gt; void;
  lessonId: string | null;
  onComplete: (lessonId: string, xpEarned: number) =&gt; void;
}) {
  const { t } = useLanguage();
  const [currentStep, setCurrentStep] = useState(0);
  const [recording, setRecording] = useState(false);
  const [isTranscribing, setIsTranscribing] = useState(false);
  const mediaRecorderRef = useRef&lt;MediaRecorder | null&gt;(null);
  const audioChunksRef = useRef&lt;BlobPart[]&gt;([]);
  const [feedback, setFeedback] = useState&lt;'excellent' | 'good' | 'poor' | null&gt;(null);
  const [lives, setLives] = useState(3);
  const [xp, setXp] = useState(0);
  const [streak, setStreak] = useState(0);
  const [showHint, setShowHint] = useState(false);
  const [showTransliteration, setShowTransliteration] = useState(true);
  const { speak, isPlaying } = useArabicTTS();
  const [selectedAnswer, setSelectedAnswer] = useState&lt;number | null&gt;(null);
  const [quizAnswered, setQuizAnswered] = useState(false);
  const [showCompletion, setShowCompletion] = useState(false);
  const startTimeRef = useRef(Date.now());
  const [correctAnswers, setCorrectAnswers] = useState(0);
  const [totalAnswered, setTotalAnswered] = useState(0);
  const [shakeWrong, setShakeWrong] = useState(false);
  const [showStreakBonus, setShowStreakBonus] = useState(false);
  const [exitConfirm, setExitConfirm] = useState(false);

  // Match State
  const [matchSelected, setMatchSelected] = useState&lt;{ arabic: string | null; armenian: string | null }&gt;({
  const [matchedPairs, setMatchedPairs] = useState&lt;string[]&gt;([]);
  // Write State
  const [writeInput, setWriteInput] = useState('');
  const [writeAnswered, setWriteAnswered] = useState(false);
  const [writeCorrect, setWriteCorrect] = useState(false);

  // We should shuffle pairs whenever step changes.
  const [shuffledArabics, setShuffledArabics] = useState&lt;string[]&gt;([]);
  const [shuffledArmenians, setShuffledArmenians] = useState&lt;string[]&gt;([]);

  const isValidLesson = lessonId &amp;&amp; lessonsData[lessonId];

  useEffect(() =&gt; {
    if (!isValidLesson) {
      onBack();
    }
  }, [isValidLesson, onBack]);
```

```
if (!isValidLesson) return null;

const lesson = lessonsData[lessonId!];
const steps = lesson.steps;
const step = steps[currentStep];
const progress = ((currentStep + 1) / steps.length) * 100;

// Setup match step arrays
useEffect(() =&gt; {
  if (step.type === 'match' &amp;&amp; step.pairs) {
    const arabics = step.pairs.map((p) =&gt; p.arabic).sort(() =&gt; Math.random() - 0.5);
    const armenians = step.pairs.map((p) =&gt; p.armenian).sort(() =&gt; Math.random() - 0.5);
    setShuffledArabics(arabics);
    setShuffledArmenians(armenians);
  }
}, [step]);

const handlePlayAudio = () =&gt; speak(step.arabic, 1.0);
const handlePlaySlow = () =&gt; speak(step.arabic, 0.7);

const handleSimulatedRecord = () =&gt; {
  if (recording) {
    setRecording(false);
    playSound('click');

    setTimeout(() =&gt; {
      const rand = Math.random();
      if (rand &gt; 0.5) {
        setFeedback('excellent');
        setXp((prev) =&gt; prev + 15);
        setStreak((prev) =&gt; prev + 1);
        setCorrectAnswers((prev) =&gt; prev + 1);
        playSound('correct');
        if (streak &gt; 0 &amp;&amp; (streak + 1) % 3 === 0) {
          setShowStreakBonus(true);
          setXp((prev) =&gt; prev + 5);
          setTimeout(() =&gt; setShowStreakBonus(false), 2000);
        }
      } else if (rand &gt; 0.25) {
        setFeedback('good');
        setXp((prev) =&gt; prev + 8);
        setCorrectAnswers((prev) =&gt; prev + 1);
        playSound('correct');
      } else {
        setFeedback('poor');
        setStreak(0);
        playSound('wrong');
        setShakeWrong(true);
        setTimeout(() =&gt; setShakeWrong(false), 500);
      }
      setTotalAnswered((prev) =&gt; prev + 1);
    }, 1200);
  } else {
    setRecording(true);
    setFeedback(null);
    playSound('speak');
  }
};

const handleRecord = async () =&gt; {
  // Stop si déjà en cours d'enregistrement.
  if (recording) {
    mediaRecorderRef.current?.stop();
    return;
  }
  if (isTranscribing) return; // évite le double-clic pendant l'analyse

  // Vérifie le support du navigateur.
  if (!navigator.mediaDevices?.getUserMedia || typeof MediaRecorder === 'undefined') {
    setFeedback('poor');
    playSound('wrong');
    return;
```

```
}

setRecording(true);
setFeedback(null);
playSound('speak');
audioChunksRef.current = [];

try {
  const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
  const mimeType = pickMimeType();
  const mediaRecorder = mimeType
    ? new MediaRecorder(stream, { mimeType })
    : new MediaRecorder(stream); // laisse le navigateur choisir si rien n'est supporté
  mediaRecorderRef.current = mediaRecorder;

  mediaRecorder.ondataavailable = (e) =&gt; {
    if (e.data.size &gt; 0) audioChunksRef.current.push(e.data);
  };

  mediaRecorder.onstop = () =&gt; {
    setRecording(false);
    stream.getTracks().forEach((t) =&gt; t.stop());

    const blob = new Blob(audioChunksRef.current, { type: mediaRecorder.mimeType });
    const reader = new FileReader();
    reader.readAsDataURL(blob);
    reader.onloadend = async () =&gt; {
      const result = reader.result as string;
      if (!result?.includes(',')) { setRecording(false); return; }
      const base64Data = result.split(',')[1];

      setIsTranscribing(true);
      try {
        const res = await fetch('/api/transcribe', {
          method: 'POST',
          headers: { 'Content-Type': 'application/json' },
          body: JSON.stringify({
            audioData: base64Data,
            mimeType: mediaRecorder.mimeType,
            expectedLanguage: 'Arabic',
          }),
        });
        if (!res.ok) throw new Error(`transcribe ${res.status}`);
        const { text: transcript } = await res.json();

        const normalizedTranscript = normalize(transcript || '');
        const expected = normalize(step.arabic);
        const similarity = getSimilarity(normalizedTranscript, expected);

        if (similarity &gt; 0.75 || (expected &amp;&amp; normalizedTranscript.includes(expected))) {
          setFeedback('excellent');
          setXp((p) =&gt; p + 15);
          setStreak((p) =&gt; p + 1);
          setCorrectAnswers((p) =&gt; p + 1);
          playSound('correct');
          if (streak &gt; 0 &amp;&amp; (streak + 1) % 3 === 0) {
            setShowStreakBonus(true);
            setXp((p) =&gt; p + 5);
            setTimeout(() =&gt; setShowStreakBonus(false), 2000);
          }
        } else if (
          similarity &gt; 0.4 ||
          expected.split(' ').some((w) =&gt; w.length &gt; 2 &amp;&amp; normalizedTranscript.includes(w))
        ) {
          setFeedback('good');
          setXp((p) =&gt; p + 8);
          setCorrectAnswers((p) =&gt; p + 1);
          playSound('correct');
        } else {
          setFeedback('poor');
          setStreak(0);
          playSound('wrong');
```

```
            setShakeWrong(true);
            setTimeout(() =&gt; setShakeWrong(false), 500);
          }
          setTotalAnswered((p) =&gt; p + 1);
        } catch (err) {
          console.error('[transcribe]', err);
          setFeedback('poor');
          setStreak(0);
          playSound('wrong');
          setShakeWrong(true);
          setTimeout(() =&gt; setShakeWrong(false), 500);
          setTotalAnswered((p) =&gt; p + 1);
        } finally {
          setIsTranscribing(false);
        }
      };
    };

    mediaRecorder.start();
    // Arrêt automatique au bout de 5,5 s max.
    setTimeout(() =&gt; {
      if (mediaRecorderRef.current?.state === 'recording') {
        mediaRecorderRef.current.stop();
      }
    }, 5500);
  } catch (err) {
    // Permission refusée ou micro indisponible.
    console.error('Micro inaccessible :', err);
    setRecording(false);
    setFeedback('poor');
    playSound('wrong');
    // Optionnel : afficher un message « activez le micro dans les réglages ».
  }
};

const handleQuizAnswer = (index: number) =&gt; {
  if (quizAnswered) return;
  setSelectedAnswer(index);
  setQuizAnswered(true);
  setTotalAnswered((prev) =&gt; prev + 1);

  const isCorrect = step.options![index].correct;
  if (isCorrect) {
    setXp((prev) =&gt; prev + 10);
    setStreak((prev) =&gt; prev + 1);
    setCorrectAnswers((prev) =&gt; prev + 1);
    playSound('correct');
    if (streak &gt; 0 &amp;&amp; (streak + 1) % 3 === 0) {
      setShowStreakBonus(true);
      setXp((prev) =&gt; prev + 5);
      setTimeout(() =&gt; setShowStreakBonus(false), 2000);
    }
  } else {
    setLives((prev) =&gt; Math.max(0, prev - 1));
    setStreak(0);
    playSound('wrong');
    setShakeWrong(true);
    setTimeout(() =&gt; setShakeWrong(false), 500);
  }
};

const handleMatchSelect = (type: 'arabic' | 'armenian', value: string) =&gt; {
  if (matchedPairs.includes(value)) return;

  setMatchSelected((prev) =&gt; {
    const next = { ...prev, [type]: value };

    if (next.arabic &amp;&amp; next.armenian) {
      // Evaluate the pair
      const isPair = step.pairs?.some((p) =&gt; p.arabic === next.arabic &amp;&amp; p.armenian === next.armenia
      if (isPair) {
        setXp((prevXp) =&gt; prevXp + 5);
```

```
        playSound('correct');
        setMatchedPairs((mp) =&gt; {
          const updated = [...mp, next.arabic!, next.armenian!];
          if (step.pairs &amp;&amp; updated.length &gt;= step.pairs.length * 2) {
            setStreak((prevStreak) =&gt; prevStreak + 1);
            setCorrectAnswers((prevCA) =&gt; prevCA + 1);
            setTotalAnswered((prevTA) =&gt; prevTA + 1);
          }
          return updated;
        });
      } else {
        setLives((prevLives) =&gt; Math.max(0, prevLives - 1));
        setStreak(0);
        playSound('wrong');
        setShakeWrong(true);
        setTimeout(() =&gt; setShakeWrong(false), 500);
      }
      return { arabic: null, armenian: null }; // Reset selection
    }
    playSound('click');
    return next;
  });
};

const handleWriteSubmit = () =&gt; {
  if (writeAnswered) return;
  setWriteAnswered(true);
  setTotalAnswered((prev) =&gt; prev + 1);

  const isCorrect = normalize(writeInput) === normalize(step.arabic);
  setWriteCorrect(isCorrect);

  if (isCorrect) {
    setXp((prev) =&gt; prev + 15);
    setStreak((prev) =&gt; prev + 1);
    setCorrectAnswers((prev) =&gt; prev + 1);
    playSound('correct');
  } else {
    setLives((prev) =&gt; Math.max(0, prev - 1));
    setStreak(0);
    playSound('wrong');
    setShakeWrong(true);
    setTimeout(() =&gt; setShakeWrong(false), 500);
  }
};

const handleNext = () =&gt; {
  if (currentStep &lt; steps.length - 1) {
    setCurrentStep((prev) =&gt; prev + 1);
    setFeedback(null);
    setSelectedAnswer(null);
    setQuizAnswered(false);
    setShowHint(false);
    setRecording(false);
    setMatchSelected({ arabic: null, armenian: null });
    setMatchedPairs([]);
    setWriteInput('');
    setWriteAnswered(false);
    setWriteCorrect(false);
    playSound('click');
  } else {
    setShowCompletion(true);
  }
};

const handleExit = () =&gt; {
  if (currentStep &gt; 0) {
    setExitConfirm(true);
  } else {
    onBack();
  }
};
```

```
const canProceed =
  step.type === 'listen' ||
  feedback !== null ||                                // speak: any feedback
  (step.type === 'quiz' &amp;&amp; quizAnswered) ||          // quiz: answered (right or wrong)
  (step.type === 'match' &amp;&amp; matchedPairs.length === (step.pairs?.length || 0) * 2) ||
  (step.type === 'write' &amp;&amp; writeAnswered);          // write: submitted (right or wrong)

if (showCompletion) {
  const elapsed = Math.round((Date.now() - startTimeRef.current) / 1000);
  const minutes = Math.floor(elapsed / 60);
  const seconds = elapsed % 60;
  const timeStr = `${minutes}:${String(seconds).padStart(2, '0')}`;

  return (
    &lt;CompletionScreen
      xpEarned={xp}
      accuracy={totalAnswered &gt; 0 ? Math.round((correctAnswers / totalAnswered) * 100) : 100}
      timeStr={timeStr}
      onContinue={() =&gt; {
        if (lessonId) onComplete(lessonId, xp);
        onBack();
      }}
    /&gt;
  );
}

if (lives === 0) {
  return (
    &lt;div className="fixed inset-0 bg-gradient-to-b from-red-500 to-red-700 z-50 flex flex-col items-center j
      &lt;div className="text-center space-y-6"&gt;
        &lt;div className="w-24 h-24 bg-red-400/30 rounded-full flex items-center justify-center mx-auto"&gt;
          &lt;Heart size={48} className="text-white" /&gt;
        &lt;/div&gt;
        &lt;h1 className="text-3xl font-black text-white"&gt;{t('lesson.out_of_lives')}&lt;/h1&gt;
        &lt;p className="text-red-100 text-lg"&gt;{t('lesson.out_of_lives_desc')}&lt;/p&gt;
        &lt;div className="space-y-3 pt-4"&gt;
          &lt;button
            onClick={() =&gt; {
              setLives(3);
              setCurrentStep(0);
              setXp(0);
              setStreak(0);
              setCorrectAnswers(0);
              setTotalAnswered(0);
              setFeedback(null);
              setSelectedAnswer(null);
              setQuizAnswered(false);
              setMatchSelected({ arabic: null, armenian: null });
              setMatchedPairs([]);
              setWriteInput('');
              setWriteAnswered(false);
              setWriteCorrect(false);
            }}
            className="w-full py-4 bg-white text-red-600 rounded-2xl font-bold text-lg"
          &gt;
            &lt;RotateCcw size={20} className="inline mr-2" /&gt;
            {t('lesson.try_again')}
          &lt;/button&gt;
          &lt;button onClick={onBack} className="w-full py-4 bg-red-600/50 text-white rounded-2xl font-bold tex
            {t('lesson.go_back')}
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

return (
  &lt;div className="flex-1 flex flex-col bg-gradient-to-b from-white to-gray-50 relative overflow-hidden"&gt;
    {/* Exit Confirmation Modal */}
    {exitConfirm &amp;&amp; (
      &lt;div className="fixed inset-0 bg-black/50 z-[60] flex items-center justify-center p-6 animate-in fade-
```

```
    &lt;div className="bg-white rounded-3xl p-6 w-full max-w-sm shadow-2xl animate-in zoom-in-95 duration-3
      &lt;div className="text-center space-y-4"&gt;
        &lt;div className="w-16 h-16 bg-amber-100 rounded-full flex items-center justify-center mx-auto"&gt
          &lt;AlertCircle size={32} className="text-amber-500" /&gt;
        &lt;/div&gt;
        &lt;h3 className="text-xl font-bold text-gray-800"&gt;{t('lesson.quit_title')}&lt;/h3&gt;
        &lt;p className="text-gray-500 text-sm"&gt;{t('lesson.quit_desc')}&lt;/p&gt;
        &lt;div className="flex gap-3 pt-2"&gt;
          &lt;button
            onClick={() =&gt; setExitConfirm(false)}
            className="flex-1 py-3 bg-emerald-500 text-white rounded-xl font-bold hover:bg-emerald-600 tran
          &gt;
            {t('lesson.stay')}
          &lt;/button&gt;
          &lt;button
            onClick={onBack}
            className="flex-1 py-3 bg-gray-100 text-gray-700 rounded-xl font-bold hover:bg-gray-200 transit
          &gt;
            {t('lesson.quit')}
          &lt;/button&gt;
        &lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
)}

{/* Streak Bonus Popup */}
{showStreakBonus &amp;&amp; (
  &lt;div className="fixed top-20 left-1/2 -translate-x-1/2 z-50 animate-in fade-in slide-in-from-top-4 dur
    &lt;div className="bg-gradient-to-r from-orange-500 to-amber-500 text-white px-6 py-3 rounded-full shad
      &lt;Zap size={20} className="text-yellow-200" /&gt;
      {t('lesson.streak_bonus')} +5 XP
      &lt;Sparkles size={16} className="text-yellow-200" /&gt;
    &lt;/div&gt;
  &lt;/div&gt;
)}

{/* Header */}
&lt;div className="bg-white/90 backdrop-blur-md px-4 py-3 border-b border-gray-100 sticky top-0 z-30"&gt;
  &lt;div className="flex items-center gap-3"&gt;
    &lt;button
      onClick={handleExit}
      className="p-2 text-gray-400 hover:text-gray-600 hover:bg-gray-100 rounded-full transition-all"
    &gt;
      &lt;X size={22} /&gt;
    &lt;/button&gt;

    {/* Progress Bar */}
    &lt;div className="flex-1 relative"&gt;
      &lt;div className="h-3 bg-gray-200 rounded-full overflow-hidden"&gt;
        &lt;div
          className="h-full bg-gradient-to-r from-emerald-400 to-teal-500 rounded-full transition-all durat
          style={{ width: `${progress}%` }}
        &gt;
          &lt;div className="absolute inset-0 bg-white/30 animate-pulse rounded-full" /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div className="flex justify-between mt-1"&gt;
        &lt;span className="text-[10px] text-gray-400 font-medium"&gt;
          {currentStep + 1}/{steps.length}
        &lt;/span&gt;
        &lt;span className="text-[10px] text-gray-400 font-medium"&gt;{Math.round(progress)}%&lt;/span&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Lives */}
    &lt;HeartsDisplay lives={lives} /&gt;

    {/* XP */}
    &lt;div className="flex items-center gap-1 bg-amber-50 px-2.5 py-1 rounded-full border border-amber-200
      &lt;Zap size={14} className="text-amber-500" /&gt;
      &lt;span className="text-xs font-bold text-amber-700"&gt;
```

```
        &lt;AnimatedCounter value={xp} /&gt;
      &lt;/span&gt;
    &lt;/div&gt;
  &lt;/div&gt;

  {/* Streak indicator */}
  {streak &gt;= 2 &amp;&amp; (
    &lt;div className="mt-2 flex justify-center"&gt;
      &lt;div className="bg-gradient-to-r from-orange-100 to-amber-100 border border-orange-200 px-3 py-1 r
        &lt;Zap size={12} className="text-orange-500" /&gt;
        &lt;span className="text-xs font-bold text-orange-700"&gt;{streak} {t('lesson.streak_fire')}&lt;/sp
      &lt;/div&gt;
    &lt;/div&gt;
  )}
&lt;/div&gt;

{/* Step Type Badge */}
&lt;div className="flex justify-center pt-5 pb-2"&gt;
  &lt;div
    className={`px-5 py-2 rounded-full text-sm font-bold flex items-center gap-2 shadow-sm ${
      step.type === 'listen'
        ? 'bg-blue-50 text-blue-600 border border-blue-200'
        : step.type === 'speak'
        ? 'bg-purple-50 text-purple-600 border border-purple-200'
        : 'bg-emerald-50 text-emerald-600 border border-emerald-200'
    }`}
  &gt;
    {step.type === 'listen' &amp;&amp; &lt;Volume2 size={16} /&gt;}
    {step.type === 'speak' &amp;&amp; &lt;Mic size={16} /&gt;}
    {step.type === 'quiz' &amp;&amp; &lt;BookOpen size={16} /&gt;}
    {step.type === 'match' &amp;&amp; &lt;Link2 size={16} /&gt;}
    {step.type === 'write' &amp;&amp; &lt;Edit3 size={16} /&gt;}
    {step.type === 'listen' &amp;&amp; t('lesson.listen_and_learn')}
    {step.type === 'speak' &amp;&amp; t('lesson.speak')}
    {step.type === 'quiz' &amp;&amp; t('lesson.quiz')}
    {step.type === 'match' &amp;&amp; t('lesson.match')}
    {step.type === 'write' &amp;&amp; t('lesson.write')}
  &lt;/div&gt;
&lt;/div&gt;

{/* Main Content */}
&lt;div className={`flex-1 flex flex-col px-6 overflow-y-auto max-w-2xl mx-auto w-full ${shakeWrong ? 'anim
  &lt;style&gt;{`
    @keyframes shake {
      0%, 100% { transform: translateX(0); }
      25% { transform: translateX(-8px); }
      75% { transform: translateX(8px); }
    }
    .animate-shake {
      animation: shake 0.3s ease-in-out 2;
    }
  `}&lt;/style&gt;

  {/* Instruction */}
  &lt;h2 className="text-lg font-bold text-gray-800 text-center mt-4"&gt;
    {step.type === 'listen' &amp;&amp; t('lesson.listen_and_read')}
    {step.type === 'speak' &amp;&amp; t('lesson.pronounce_sentence')}
    {step.type === 'quiz' &amp;&amp; step.meaning}
    {step.type === 'match' &amp;&amp; step.meaning}
    {step.type === 'write' &amp;&amp; step.meaning}
  &lt;/h2&gt;

  {/* Arabic Text Display */}
  {(step.type === 'listen' || step.type === 'speak') &amp;&amp; (
    &lt;div className="flex-1 flex flex-col items-center justify-center space-y-8 py-6"&gt;
      {/* Arabic Word Card */}
      &lt;div className="bg-white rounded-3xl p-8 shadow-lg shadow-gray-200/50 border border-gray-100 w-ful
        &lt;div className="text-5xl font-arabic leading-loose text-gray-900 mb-4" dir="rtl"&gt;
          {step.highlightChar
            ? step.arabic.split('').map((char, i) =&gt; {
                if (step.arabic[i] === step.highlightChar) {
                  return (
```

```
              &lt;span key={i} className="text-red-500 font-bold relative"&gt;
                {char}
                &lt;span className="absolute -bottom-2 left-1/2 -translate-x-1/2 w-1.5 h-1.5 bg-red-4
              &lt;/span&gt;
            );
          }
          return &lt;span key={i}&gt;{char}&lt;/span&gt;;
        })
      : step.arabic}
  &lt;/div&gt;

  {/* Transliteration */}
  {showTransliteration &amp;&amp; (
    &lt;p className="text-base text-gray-400 font-mono mb-2"&gt;{step.transliteration}&lt;/p&gt;
  )}

  {/* Armenian meaning */}
  &lt;p className="text-sm text-gray-500"&gt;{step.armenian}&lt;/p&gt;

  {/* Toggle transliteration */}
  &lt;button
    onClick={() =&gt; setShowTransliteration(!showTransliteration)}
    className="mt-3 flex items-center gap-1.5 text-xs text-gray-400 hover:text-gray-600 mx-auto trans
  &gt;
    {showTransliteration ? &lt;EyeOff size={14} /&gt; : &lt;Eye size={14} /&gt;}
    {showTransliteration ? t('lesson.hide_transliteration') : t('lesson.show_transliteration')}
  &lt;/button&gt;
&lt;/div&gt;

{/* Audio Controls */}
&lt;div className="flex items-center gap-5"&gt;
  &lt;button
    onClick={handlePlayAudio}
    disabled={isPlaying}
    className={`w-16 h-16 rounded-full flex items-center justify-center shadow-lg transition-all dura
      isPlaying
        ? 'bg-blue-400 scale-95'
        : 'bg-gradient-to-br from-blue-500 to-indigo-600 hover:from-blue-600 hover:to-indigo-700 acti
    }`}
  &gt;
    {isPlaying ? (
      &lt;div className="flex items-center gap-0.5"&gt;
        &lt;div className="w-1 h-5 bg-white rounded-full animate-pulse" /&gt;
        &lt;div className="w-1 h-7 bg-white rounded-full animate-pulse delay-75" /&gt;
        &lt;div className="w-1 h-4 bg-white rounded-full animate-pulse delay-150" /&gt;
      &lt;/div&gt;
    ) : (
      &lt;Play size={30} fill="white" className="text-white ml-1" /&gt;
    )}
  &lt;/button&gt;
  &lt;button
    onClick={handlePlaySlow}
    disabled={isPlaying}
    className="w-12 h-12 bg-gray-100 text-gray-600 rounded-full flex items-center justify-center hove
    title="Slow"
  &gt;
    &lt;Turtle size={22} /&gt;
  &lt;/button&gt;
&lt;/div&gt;

{/* Hint */}
{step.hint &amp;&amp; (
  &lt;div className="w-full"&gt;
    &lt;button
      onClick={() =&gt; setShowHint(!showHint)}
      className="flex items-center gap-2 text-sm text-amber-600 hover:text-amber-700 mx-auto transiti
    &gt;
      &lt;Lightbulb size={16} /&gt;
      {showHint ? t('lesson.hide_hint') : t('lesson.show_hint')}
    &lt;/button&gt;
    {showHint &amp;&amp; (
      &lt;div className="mt-3 bg-amber-50 border border-amber-200 rounded-2xl p-4 text-center animate
```

```
            &lt;p className="text-amber-800 text-sm font-medium flex items-center justify-center gap-2"&g
              &lt;Lightbulb size={16} className="text-amber-500 flex-shrink-0" /&gt;
              {step.hint}
            &lt;/p&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    )}

    {/* Recording waveform (for speak step) */}
    {step.type === 'speak' &amp;&amp; recording &amp;&amp; &lt;WaveformAnimation active={recording} /&gt;
  &lt;/div&gt;
)}

{/* Quiz Content */}
{step.type === 'quiz' &amp;&amp; step.options &amp;&amp; (
  &lt;div className="flex-1 flex flex-col justify-center py-6 space-y-4"&gt;
    {/* Arabic word being quizzed */}
    &lt;div className="bg-white rounded-2xl p-6 shadow-md border border-gray-100 text-center mb-4"&gt;
      &lt;p className="text-4xl font-arabic text-gray-900" dir="rtl"&gt;
        {step.arabic}
      &lt;/p&gt;
      &lt;p className="text-sm text-gray-400 mt-2 font-mono"&gt;{step.transliteration}&lt;/p&gt;
    &lt;/div&gt;

    {/* Options */}
    &lt;div className="space-y-3"&gt;
      {step.options.map((option, idx) =&gt; {
        let optionStyle = 'bg-white border-gray-200 text-gray-800 hover:border-emerald-300 hover:bg-emera

        if (quizAnswered) {
          if (option.correct) {
            optionStyle = 'bg-emerald-50 border-emerald-500 text-emerald-800 ring-2 ring-emerald-200';
          } else if (idx === selectedAnswer &amp;&amp; !option.correct) {
            optionStyle = 'bg-red-50 border-red-500 text-red-800 ring-2 ring-red-200';
          } else {
            optionStyle = 'bg-gray-50 border-gray-200 text-gray-400';
          }
        } else if (idx === selectedAnswer) {
          optionStyle = 'bg-emerald-50 border-emerald-400 text-emerald-800';
        }

        return (
          &lt;button
            key={idx}
            onClick={() =&gt; handleQuizAnswer(idx)}
            disabled={quizAnswered}
            className={`w-full p-4 rounded-2xl border-2 text-left font-medium transition-all duration-300
              !quizAnswered ? 'active:scale-[0.98]' : ''
            }`}
          &gt;
            &lt;span className="w-8 h-8 rounded-full bg-gray-100 flex items-center justify-center text-sm
              {String.fromCharCode(1329 + idx)}
            &lt;/span&gt;
            &lt;span className="flex-1"&gt;{option.text}&lt;/span&gt;
            {quizAnswered &amp;&amp; option.correct &amp;&amp; &lt;CheckCircle size={22} className="text-
            {quizAnswered &amp;&amp; idx === selectedAnswer &amp;&amp; !option.correct &amp;&amp; (
              &lt;AlertCircle size={22} className="text-red-500 flex-shrink-0" /&gt;
            )}
          &lt;/button&gt;
        );
      })}
    &lt;/div&gt;
  &lt;/div&gt;
)}

{/* Match Content */}
{step.type === 'match' &amp;&amp; step.pairs &amp;&amp; (
  &lt;div className="flex-1 flex flex-col justify-center py-6"&gt;
    &lt;div className="grid grid-cols-2 gap-4"&gt;
      &lt;div className="space-y-3 flex flex-col"&gt;
        {shuffledArabics.map((ar, idx) =&gt; {
```

```
          const isMatched = matchedPairs.includes(ar);
          const isSelected = matchSelected.arabic === ar;
          return (
            &lt;button
              key={`ar-${idx}`}
              disabled={isMatched}
              onClick={() =&gt; handleMatchSelect('arabic', ar)}
              className={`h-24 p-4 rounded-2xl border-2 text-center font-bold text-2xl font-arabic transi
                isMatched
                  ? 'bg-gray-100 border-gray-200 text-gray-300 opacity-50'
                  : isSelected
                  ? 'bg-blue-50 border-blue-400 text-blue-700 w-[105%] z-10 shadow-md'
                  : 'bg-white border-gray-200 text-gray-800 hover:border-blue-300 hover:bg-slate-50 shado
              }`}
              dir="rtl"
            &gt;
              {ar}
            &lt;/button&gt;
          );
        })}
      &lt;/div&gt;
      &lt;div className="space-y-3 flex flex-col"&gt;
        {shuffledArmenians.map((am, idx) =&gt; {
          const isMatched = matchedPairs.includes(am);
          const isSelected = matchSelected.armenian === am;
          return (
            &lt;button
              key={`am-${idx}`}
              disabled={isMatched}
              onClick={() =&gt; handleMatchSelect('armenian', am)}
              className={`h-24 p-4 rounded-2xl border-2 text-center font-medium text-base transition-all
                isMatched
                  ? 'bg-gray-100 border-gray-200 text-gray-300 opacity-50'
                  : isSelected
                  ? 'bg-blue-50 border-blue-400 text-blue-700 w-[105%] ml-[5%] z-10 shadow-md'
                  : 'bg-white border-gray-200 text-gray-800 hover:border-blue-300 hover:bg-slate-50 shado
              }`}
            &gt;
              {am}
            &lt;/button&gt;
          );
        })}
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Hint */}
    {step.hint &amp;&amp; (
      &lt;div className="mt-8 text-center text-sm font-medium text-gray-400"&gt;
        {step.hint}
      &lt;/div&gt;
    )}
  &lt;/div&gt;
)}

{/* Write Content */}
{step.type === 'write' &amp;&amp; (
  &lt;div className="flex-1 flex flex-col justify-center py-6 space-y-8"&gt;
    &lt;div className="text-center"&gt;
      &lt;p className="text-gray-500 mb-2"&gt;{t('lesson.translate_this')}&lt;/p&gt;
      &lt;h3 className="text-2xl font-bold text-gray-800"&gt;{step.armenian}&lt;/h3&gt;
    &lt;/div&gt;

    &lt;div className="space-y-4"&gt;
      &lt;textarea
        dir="rtl"
        disabled={writeAnswered}
        value={writeInput}
        onChange={(e) =&gt; setWriteInput(e.target.value)}
        placeholder="[redacted]..."
        className={`w-full p-4 rounded-2xl border-2 font-arabic text-3xl transition-all min-h-[120px] foc
          writeAnswered
            ? writeCorrect
```

```
              ? 'border-emerald-500 bg-emerald-50 text-emerald-800 ring-emerald-200'
              : 'border-red-500 bg-red-50 text-red-800 ring-red-200'
            : 'border-gray-300 bg-white hover:border-blue-300 focus:border-blue-500 focus:ring-blue-100 s
        }`}
      /&gt;

      {!writeAnswered &amp;&amp; (
        &lt;button
          onClick={handleWriteSubmit}
          disabled={!writeInput.trim()}
          className="w-full py-4 bg-blue-600 text-white rounded-2xl font-bold text-lg disabled:opacity-50
        &gt;
          {t('lesson.check')}
        &lt;/button&gt;
      )}

      {writeAnswered &amp;&amp; !writeCorrect &amp;&amp; (
        &lt;div className="bg-red-50 p-4 rounded-2xl border border-red-200 text-center"&gt;
          &lt;p className="text-red-800 font-bold mb-1"&gt;{t('lesson.correct_answer_is')}&lt;/p&gt;
          &lt;p className="text-3xl font-arabic text-red-600" dir="rtl"&gt;{step.arabic}&lt;/p&gt;
        &lt;/div&gt;
      )}
    &lt;/div&gt;

    {/* Hint */}
    {step.hint &amp;&amp; (
      &lt;div className="w-full mt-4"&gt;
        &lt;button
          onClick={() =&gt; setShowHint(!showHint)}
          className="flex items-center gap-2 text-sm text-amber-600 hover:text-amber-700 mx-auto transiti
        &gt;
          &lt;Lightbulb size={16} /&gt;
          {showHint ? t('lesson.hide_hint') : t('lesson.show_hint')}
        &lt;/button&gt;
        {showHint &amp;&amp; (
          &lt;div className="mt-3 bg-amber-50 border border-amber-200 rounded-2xl p-4 text-center animate
            &lt;p className="text-amber-800 text-sm font-medium flex items-center justify-center gap-2"&g
              &lt;Lightbulb size={16} className="text-amber-500 flex-shrink-0" /&gt;
              {step.hint}
            &lt;/p&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    )}
  &lt;/div&gt;
)}

{/* Feedback Area */}
&lt;div className="py-4"&gt;
  {feedback === 'excellent' &amp;&amp; (
    &lt;div className="bg-emerald-50 border border-emerald-200 rounded-2xl p-4 flex items-center gap-3 an
      &lt;div className="w-10 h-10 bg-emerald-100 rounded-full flex items-center justify-center flex-shri
        &lt;CheckCircle size={22} className="text-emerald-500" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="font-bold text-emerald-800"&gt;{t('lesson.excellent')}&lt;/p&gt;
        &lt;p className="text-xs text-emerald-600 mt-0.5"&gt;{t('lesson.excellent_desc')}&lt;/p&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  )}
  {feedback === 'good' &amp;&amp; (
    &lt;div className="bg-amber-50 border border-amber-200 rounded-2xl p-4 flex items-center gap-3 animat
      &lt;div className="w-10 h-10 bg-amber-100 rounded-full flex items-center justify-center flex-shrink
        &lt;CheckCircle size={22} className="text-amber-500" /&gt;
      &lt;/div&gt;
      &lt;div&gt;
        &lt;p className="font-bold text-amber-800"&gt;{t('lesson.good')}&lt;/p&gt;
        &lt;p className="text-xs text-amber-600 mt-0.5"&gt;{t('lesson.good_desc')}&lt;/p&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  )}
  {feedback === 'poor' &amp;&amp; (
```

```
          &lt;div className="bg-red-50 border border-red-200 rounded-2xl p-4 flex items-center gap-3 animate-in
            &lt;div className="w-10 h-10 bg-red-100 rounded-full flex items-center justify-center flex-shrink-0
              &lt;AlertCircle size={22} className="text-red-500" /&gt;
            &lt;/div&gt;
            &lt;div&gt;
              &lt;p className="font-bold text-red-800"&gt;{t('lesson.poor')}&lt;/p&gt;
              &lt;p className="text-xs text-red-600 mt-0.5"&gt;{t('lesson.poor_desc')}&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;

    {/* Bottom Actions */}
    &lt;div className="bg-white/95 backdrop-blur-md border-t border-gray-100 px-6 py-4 pb-6 space-y-3"&gt;
      &lt;div className="max-w-2xl mx-auto w-full space-y-3"&gt;
        {/* Record button for speak steps */}
      {step.type === 'speak' &amp;&amp; (
        &lt;div className="flex justify-center pb-2"&gt;
          &lt;button
            onClick={handleRecord}
            disabled={isTranscribing}
            className={`w-20 h-20 rounded-full flex items-center justify-center shadow-xl transition-all durati
              recording
                ? 'bg-gradient-to-br from-red-500 to-rose-600 text-white scale-110 ring-4 ring-red-200 animate-
                : 'bg-gradient-to-br from-purple-500 to-indigo-600 text-white hover:from-purple-600 hover:to-in
            } ${isTranscribing ? 'opacity-75 cursor-wait' : ''}`}
          &gt;
            {isTranscribing ? (
               &lt;div className="w-8 h-8 border-4 border-white border-t-transparent rounded-full animate-spin"
            ) : recording ? &lt;MicOff size={32} /&gt; : &lt;Mic size={32} /&gt;}
          &lt;/button&gt;
        &lt;/div&gt;
      )}

      {/* Next / Continue button */}
      &lt;button
        onClick={handleNext}
        disabled={!canProceed}
        className={`w-full py-4 rounded-2xl font-bold text-lg transition-all duration-300 flex items-center jus
          canProceed
            ? 'bg-gradient-to-r from-emerald-500 to-teal-500 text-white shadow-lg shadow-emerald-200 hover:from
            : 'bg-gray-200 text-gray-400 cursor-not-allowed'
        }`}
      &gt;
        {currentStep &lt; steps.length - 1 ? (
          &lt;&gt;
            {t('lesson.continue')}
            &lt;ArrowRight size={20} /&gt;
          &lt;/&gt;
        ) : (
          &lt;&gt;
            {t('lesson.finish')}
            &lt;Trophy size={20} /&gt;
          &lt;/&gt;
        )}
      &lt;/button&gt;

      {/* Skip for listen steps */}
      {step.type === 'listen' &amp;&amp; (
        &lt;button
          onClick={handleNext}
          className="w-full py-2 text-gray-400 text-sm font-medium hover:text-gray-600 transition-colors"
        &gt;
          {t('lesson.skip')}
        &lt;/button&gt;
      )}
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
);
}
```

■ FICHIER : src\screens\PracticeScreen.tsx

```
import React, { useState } from 'react';
import { BookOpen, Headphones, PenTool, CheckCircle2 } from 'lucide-react';
import { useLanguage } from '../contexts/LanguageContext';
import VocabularyPractice from '../components/VocabularyPractice';

export default function PracticeScreen() {
  const { t } = useLanguage();
  const [activeMode, setActiveMode] = useState&lt;'vocab' | null&gt;(null);

  if (activeMode === 'vocab') {
    return &lt;VocabularyPractice onBack={() =&gt; setActiveMode(null)} /&gt;;
  }

  return (
    &lt;div className="flex-1 overflow-y-auto bg-gray-50 pb-24 md:pb-6 w-full"&gt;
      &lt;div className="bg-gradient-to-b from-blue-500 to-blue-600 px-6 pt-12 pb-8 md:rounded-b-3xl text-white s
        &lt;div className="max-w-2xl mx-auto"&gt;
          &lt;h1 className="text-2xl font-bold mb-2"&gt;■■■■■■&lt;/h1&gt;
          &lt;p className="text-blue-100 mb-4 opacity-90"&gt;
            ■■■■■■■■■■■■■■■ (Practice)
          &lt;/p&gt;
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="p-4 space-y-4 max-w-2xl mx-auto w-full"&gt;

        &lt;button
          onClick={() =&gt; setActiveMode('vocab')}
          className="w-full bg-white rounded-2xl p-4 shadow-sm border border-gray-100 flex items-center justify-b
        &gt;
          &lt;div className="flex items-center gap-4"&gt;
            &lt;div className="w-12 h-12 rounded-full bg-emerald-100 text-emerald-600 flex items-center justify-c
              &lt;BookOpen size={24} /&gt;
            &lt;/div&gt;
            &lt;div className="text-right"&gt;
              &lt;h3 className="font-bold text-gray-800 text-lg"&gt;■■■■■■■■■■&lt;/h3&gt;
              &lt;p className="text-sm text-gray-500"&gt;Flashcards (■■■■■■■)&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;CheckCircle2 className="text-emerald-500" /&gt;
        &lt;/button&gt;

        &lt;button className="w-full bg-white rounded-2xl p-4 shadow-sm border border-gray-100 flex items-center
          &lt;div className="flex items-center gap-4"&gt;
            &lt;div className="w-12 h-12 rounded-full bg-purple-100 text-purple-600 flex items-center justify-cen
              &lt;Headphones size={24} /&gt;
            &lt;/div&gt;
            &lt;div className="text-right"&gt;
              &lt;h3 className="font-bold text-gray-800 text-lg"&gt;■■■■■■■■■■■■&lt;/h3&gt;
              &lt;p className="text-sm text-gray-500"&gt;■■■■■■■ (■■■■■■■)&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;CheckCircle2 className="text-gray-300" /&gt;
        &lt;/button&gt;

        &lt;button className="w-full bg-white rounded-2xl p-4 shadow-sm border border-gray-100 flex items-center
          &lt;div className="flex items-center gap-4"&gt;
            &lt;div className="w-12 h-12 rounded-full bg-orange-100 text-orange-600 flex items-center justify-cen
              &lt;PenTool size={24} /&gt;
            &lt;/div&gt;
            &lt;div className="text-right"&gt;
              &lt;h3 className="font-bold text-gray-800 text-lg"&gt;■■■■■■■■■■■■■■■&lt;/h3&gt;
              &lt;p className="text-sm text-gray-500"&gt;■■■■■■ (■■■■■■■)&lt;/p&gt;
            &lt;/div&gt;
          &lt;/div&gt;
          &lt;CheckCircle2 className="text-gray-300" /&gt;
        &lt;/button&gt;
```

```
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\screens\ProfileScreen.tsx

```
import React, { useState } from 'react';
import { Flame, Zap, Calendar as CalendarIcon, Trophy, Target, BookOpen, Star, Globe, Pencil, Check } from 'lucid
import { useLanguage } from '../contexts/LanguageContext';

// ■■■ Types ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

interface AchievementCardProps {
  icon: React.ReactNode;
  iconBg: string;
  title: string;
  description: string;
  progress: number;
  completed?: boolean;
  xp?: number;
}

interface StatCardProps {
  icon: React.ReactNode;
  iconBg: string;
  value: string | number;
  label: string;
}

// ■■■ Constants ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

const WEEKDAY_LABELS = ['■', '■', '■', '■', '■', '■', '■'];
const TOTAL_LESSONS = 22;

// ■■■ ProfileScreen ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

export default function ProfileScreen({
  completedUnits,
  totalXP,
  streak,
}: {
  completedUnits: string[];
  totalXP: number;
  streak: number;
}) {
  const { t, language, setLanguage } = useLanguage();

  // ■■ Nom éditable ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  const [userName, setUserName] = useState&lt;string&gt;(
    () =&gt; localStorage.getItem('userName') || (language === 'hy' ? '■■■■■' : '■■■■■')
  );
  const [editingName, setEditingName] = useState(false);
  const [nameInput, setNameInput] = useState(userName);

  const saveName = () =&gt; {
    const trimmed = nameInput.trim();
    if (trimmed) {
      setUserName(trimmed);
      localStorage.setItem('userName', trimmed);
    }
    setEditingName(false);
  };

  // ■■ Données ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  const levelBadge = completedUnits.length &lt; 4 ? 'A1' : completedUnits.length &lt; 10 ? 'A2' : 'B1';

  const studyDates: string[] = (() =&gt; {
    try { return JSON.parse(localStorage.getItem('studyHistory') || '[]'); }
```

```
  catch { return []; }
})();

const ach1Done = completedUnits.length &gt;= 1;
const ach2Done = streak &gt;= 7;
const ach3Done = completedUnits.length &gt;= 10;
const ach4Done = completedUnits.length &gt;= TOTAL_LESSONS;
const achCount = [ach1Done, ach2Done, ach3Done, ach4Done].filter(Boolean).length;

const goalsPercent = Math.round((completedUnits.length / TOTAL_LESSONS) * 100);

const daysActiveLast28 = Array.from({ length: 28 }).filter((_, i) =&gt; {
  const date = new Date();
  date.setDate(date.getDate() - (27 - i));
  return studyDates.includes(date.toDateString());
}).length;

return (
  &lt;div className="flex-1 overflow-y-auto bg-gray-50 pb-24 md:pb-6"&gt;
    {/* ■■ Hero Header ■■ */}
    &lt;div className="bg-white border-b border-gray-100 px-6 pt-10 pb-6 w-full"&gt;
      &lt;div className="max-w-2xl mx-auto"&gt;
        &lt;div className="flex items-center justify-between mb-4"&gt;
          &lt;h1 className="text-2xl font-bold text-gray-900"&gt;{t('nav.profile')}&lt;/h1&gt;

          {/* Language Switcher */}
          &lt;button
            onClick={() =&gt; setLanguage(language === 'hy' ? 'ar' : 'hy')}
            className="flex items-center gap-2 bg-gray-100 hover:bg-gray-200 px-3 py-1.5 rounded-full transitio
          &gt;
            &lt;Globe size={16} className="text-gray-600" /&gt;
            &lt;span className="text-sm font-medium text-gray-700"&gt;
              {language === 'hy' ? '■■■' : '■■■■'}
            &lt;/span&gt;
          &lt;/button&gt;
        &lt;/div&gt;

        &lt;div className="flex items-center gap-5"&gt;
          {/* Avatar */}
          &lt;div className="relative"&gt;
            &lt;div className="w-20 h-20 bg-gradient-to-br from-blue-400 to-blue-600 rounded-full flex items-ce
              {userName.charAt(0).toUpperCase()}
            &lt;/div&gt;
            &lt;div className="absolute -bottom-1 -right-1 bg-blue-600 text-white text-[10px] font-bold px-1.5
              {levelBadge}
            &lt;/div&gt;
          &lt;/div&gt;

          &lt;div className="flex-1 min-w-0"&gt;
            {/* Nom éditable */}
            {editingName ? (
              &lt;div className="flex items-center gap-2"&gt;
                &lt;input
                  autoFocus
                  value={nameInput}
                  onChange={e =&gt; setNameInput(e.target.value)}
                  onKeyDown={e =&gt; { if (e.key === 'Enter') saveName(); if (e.key === 'Escape') setEditingNam
                  className="text-xl font-bold text-gray-900 border-b-2 border-blue-500 outline-none bg-transpa
                  maxLength={30}
                /&gt;
                &lt;button onClick={saveName} className="w-7 h-7 bg-blue-500 text-white rounded-full flex items
                  &lt;Check size={14} strokeWidth={3} /&gt;
                &lt;/button&gt;
              &lt;/div&gt;
            ) : (
              &lt;div className="flex items-center gap-2"&gt;
                &lt;h2 className="text-2xl font-bold text-gray-900 truncate"&gt;{userName}&lt;/h2&gt;
                &lt;button
                  onClick={() =&gt; { setNameInput(userName); setEditingName(true); }}
                  className="text-gray-400 hover:text-blue-500 transition-colors shrink-0"
                  aria-label={t('profile.edit_name')}
                &gt;
```

```
              &lt;Pencil size={15} /&gt;
            &lt;/button&gt;
          &lt;/div&gt;
        )}

        &lt;p className="text-sm text-gray-500 mt-0.5"&gt;{t('profile.member_since')}&lt;/p&gt;

        &lt;div className="mt-2 inline-flex items-center gap-1.5 bg-amber-50 border border-amber-200 text-a
          &lt;Star size={11} fill="currentColor" /&gt;
          {t('profile.bronze_league')}
        &lt;/div&gt;
      &lt;/div&gt;

      {/* Total XP pill */}
      &lt;div className="flex flex-col items-center bg-yellow-50 border border-yellow-200 rounded-2xl px-4
        &lt;Zap size={18} className="text-yellow-500 mb-0.5" /&gt;
        &lt;span className="text-lg font-bold text-gray-800 leading-none"&gt;{totalXP}&lt;/span&gt;
        &lt;span className="text-[10px] text-gray-500 font-medium mt-0.5"&gt;XP&lt;/span&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  &lt;/div&gt;
&lt;/div&gt;

&lt;div className="p-5 space-y-8 max-w-2xl mx-auto w-full"&gt;
  {/* ■■ Stats Grid ■■ */}
  &lt;section&gt;
    &lt;h2 className="text-base font-bold text-gray-700 mb-3"&gt;{t('profile.stats')}&lt;/h2&gt;
    &lt;div className="grid grid-cols-2 gap-3"&gt;
      &lt;StatCard
        icon={&lt;Flame size={22} /&gt;}
        iconBg="bg-orange-100 text-orange-500"
        value={streak}
        label={t('profile.streak')}
      /&gt;
      &lt;StatCard
        icon={&lt;BookOpen size={22} /&gt;}
        iconBg="bg-blue-100 text-blue-500"
        value={completedUnits.length}
        label={t('profile.lessons_done')}
      /&gt;
      &lt;StatCard
        icon={&lt;Trophy size={22} /&gt;}
        iconBg="bg-yellow-100 text-yellow-500"
        value={`${achCount}/4`}
        label={t('profile.achievements')}
      /&gt;
      &lt;StatCard
        icon={&lt;Target size={22} /&gt;}
        iconBg="bg-green-100 text-green-600"
        value={`${goalsPercent}%`}
        label={t('profile.goals')}
      /&gt;
    &lt;/div&gt;
  &lt;/section&gt;

  {/* ■■ Achievements ■■ */}
  &lt;section&gt;
    &lt;div className="flex items-center justify-between mb-3"&gt;
      &lt;h2 className="text-base font-bold text-gray-700"&gt;{t('profile.achievements')}&lt;/h2&gt;
      &lt;span className="text-blue-600 text-sm font-bold"&gt;{achCount}/4&lt;/span&gt;
    &lt;/div&gt;
    &lt;div className="space-y-3"&gt;
      &lt;AchievementCard
        icon={&lt;Trophy size={22} /&gt;}
        iconBg="bg-yellow-100 text-yellow-500"
        title={t('profile.ach1.title')}
        description={t('profile.ach1.desc')}
        progress={ach1Done ? 100 : 0}
        completed={ach1Done}
        xp={50}
      /&gt;
      &lt;AchievementCard
```

```
      icon={&lt;Flame size={22} /&gt;}
      iconBg="bg-orange-100 text-orange-500"
      title={t('profile.ach2.title')}
      description={t('profile.ach2.desc')}
      progress={Math.min(100, Math.round((streak / 7) * 100))}
      completed={ach2Done}
      xp={100}
    /&gt;
    &lt;AchievementCard
      icon={&lt;Target size={22} /&gt;}
      iconBg="bg-blue-100 text-blue-500"
      title={t('profile.ach3.title')}
      description={t('profile.ach3.desc')}
      progress={Math.min(100, Math.round((completedUnits.length / 10) * 100))}
      completed={ach3Done}
      xp={200}
    /&gt;
    &lt;AchievementCard
      icon={&lt;Star size={22} /&gt;}
      iconBg="bg-green-100 text-green-500"
      title={t('profile.ach4.title')}
      description={t('profile.ach4.desc')}
      progress={Math.min(100, Math.round((completedUnits.length / TOTAL_LESSONS) * 100))}
      completed={ach4Done}
      xp={500}
    /&gt;
  &lt;/div&gt;
&lt;/section&gt;

{/* ■■ Learning Calendar ■■ */}
&lt;section&gt;
  &lt;h2 className="text-base font-bold text-gray-700 mb-3 flex items-center gap-2"&gt;
    &lt;CalendarIcon size={16} className="text-gray-400" /&gt;
    {t('profile.calendar')}
  &lt;/h2&gt;
  &lt;div className="bg-white rounded-2xl border border-gray-200 shadow-sm p-4"&gt;
    &lt;div className="flex justify-between items-center mb-3"&gt;
      &lt;span className="text-sm font-bold text-gray-700"&gt;{t('profile.month_name')}&lt;/span&gt;
      &lt;div className="flex items-center gap-1.5 text-xs text-gray-500"&gt;
        &lt;div className="w-3 h-3 rounded-sm bg-green-500" /&gt;
        &lt;span&gt;{t('profile.learned')}&lt;/span&gt;
        &lt;div className="w-3 h-3 rounded-sm bg-gray-100 ml-2" /&gt;
        &lt;span&gt;{t('profile.missed')}&lt;/span&gt;
      &lt;/div&gt;
    &lt;/div&gt;

    &lt;div className="grid grid-cols-7 gap-1.5 text-center mb-1.5"&gt;
      {WEEKDAY_LABELS.map((d, i) =&gt; (
        &lt;div key={i} className="text-[10px] text-gray-400 font-bold"&gt;{d}&lt;/div&gt;
      ))}
    &lt;/div&gt;

    &lt;div className="grid grid-cols-7 gap-1.5"&gt;
      {Array.from({ length: 28 }).map((_, i) =&gt; {
        const date = new Date();
        date.setDate(date.getDate() - (27 - i));
        const isActive = studyDates.includes(date.toDateString());
        const isToday = i === 27;

        return (
          &lt;div
            key={i}
            className={`
              aspect-square rounded-lg flex items-center justify-center text-[11px] font-semibold
              transition-transform active:scale-90
              ${isActive ? 'bg-green-500 text-white' : 'bg-gray-100 text-gray-400'}
              ${isToday ? 'ring-2 ring-offset-1 ring-green-500' : ''}
            `}
          &gt;
            {date.getDate()}
          &lt;/div&gt;
        );
```

```
              })}
            &lt;/div&gt;

            &lt;div className="mt-4 pt-3 border-t border-gray-100 flex justify-around text-center"&gt;
              &lt;div&gt;
                &lt;div className="text-lg font-bold text-gray-800"&gt;{daysActiveLast28}&lt;/div&gt;
                &lt;div className="text-[10px] text-gray-500"&gt;{t('profile.days_month')}&lt;/div&gt;
              &lt;/div&gt;
              &lt;div className="w-px bg-gray-100" /&gt;
              &lt;div&gt;
                &lt;div className="text-lg font-bold text-orange-500"&gt;{streak} ■&lt;/div&gt;
                &lt;div className="text-[10px] text-gray-500"&gt;{t('profile.streak_label')}&lt;/div&gt;
              &lt;/div&gt;
              &lt;div className="w-px bg-gray-100" /&gt;
              &lt;div&gt;
                &lt;div className="text-lg font-bold text-gray-800"&gt;{Math.round((daysActiveLast28 / 28) * 100)
                &lt;div className="text-[10px] text-gray-500"&gt;{t('profile.monthly_label')}&lt;/div&gt;
              &lt;/div&gt;
            &lt;/div&gt;
          &lt;/div&gt;
        &lt;/section&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// ■■■ StatCard ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

function StatCard({ icon, iconBg, value, label }: StatCardProps) {
  return (
    &lt;div className="bg-white rounded-2xl border border-gray-100 shadow-sm p-4 flex items-center gap-3"&gt;
      &lt;div className={`w-11 h-11 rounded-xl flex items-center justify-center shrink-0 ${iconBg}`}&gt;
        {icon}
      &lt;/div&gt;
      &lt;div className="min-w-0"&gt;
        &lt;div className="text-xl font-bold text-gray-800 leading-none"&gt;{value}&lt;/div&gt;
        &lt;div className="text-[11px] text-gray-500 font-medium mt-0.5 truncate"&gt;{label}&lt;/div&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// ■■■ AchievementCard ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■

function AchievementCard({
  icon, iconBg, title, description, progress, completed = false, xp,
}: AchievementCardProps) {
  return (
    &lt;div className={`bg-white rounded-2xl border shadow-sm p-4 flex items-center gap-4 ${completed ? 'border-y
      &lt;div className={`w-12 h-12 rounded-xl flex items-center justify-center shrink-0 ${iconBg} ${completed ?
        {icon}
      &lt;/div&gt;
      &lt;div className="flex-1 min-w-0"&gt;
        &lt;div className="flex items-center justify-between gap-2 mb-0.5"&gt;
          &lt;h3 className="font-bold text-gray-800 text-sm truncate"&gt;{title}&lt;/h3&gt;
          {xp !== undefined &amp;&amp; (
            &lt;span className={`text-[10px] font-bold px-1.5 py-0.5 rounded-full shrink-0 ${completed ? 'bg-yell
              +{xp} XP
            &lt;/span&gt;
          )}
        &lt;/div&gt;
        &lt;p className="text-[11px] text-gray-400 mb-2"&gt;{description}&lt;/p&gt;
        &lt;div className="h-1.5 bg-gray-100 rounded-full overflow-hidden"&gt;
          &lt;div
            className={`h-full rounded-full transition-all duration-500 ${completed ? 'bg-yellow-400' : 'bg-blue-
            style={{ width: `${progress}%` }}
          /&gt;
        &lt;/div&gt;
      &lt;/div&gt;
      &lt;div className={`text-sm font-bold shrink-0 ${completed ? 'text-yellow-500' : 'text-gray-400'}`}&gt;
        {progress}%
      &lt;/div&gt;
```

```
    &lt;/div&gt;
  );
}
```

■ FICHIER : src\screens\SpeechSetupScreen.tsx

```
/**
 * SpeechSetupScreen — ■■■■■ ■■■■■■■■■ ■■■■■■■ (■■■■■ ■■■■ ■■■■■ ■■■■ ■■■■ ■■■■■■)
 * ■■■■■ ■■■■■ ■■■■■■■■■■ ■■■■■■■■■: ■■■■■■■ ■■■■■■■■■■■■■■ ■■■■■■■■■■■■■ ■■■■■■■
 */

import React, { useState, useEffect, useCallback } from 'react';
import {
  Volume2, Mic, CheckCircle, XCircle,
  ChevronRight, ChevronLeft, Smartphone, Settings,
  Bell, MapPin, Shield,
} from 'lucide-react';
import { useLanguage } from '../contexts/LanguageContext';
import { useArabicTTS } from '../hooks/useArabicTTS';

interface Props {
  onDone: () =&gt; void;
}

type StepId    = 'welcome' | 'synthesis' | 'recognition' | 'notifications' | 'done';
type TestState = 'idle' | 'testing' | 'ok' | 'fail' | 'denied';

type Platform = 'android' | 'ios' | 'desktop';

export default function SpeechSetupScreen({ onDone }: Props) {
  const { language } = useLanguage();
  const ar = language === 'ar';
  const { speak } = useArabicTTS();

  const [step, setStep]           = useState&lt;StepId&gt;('welcome');
  const [ttsState, setTtsState]   = useState&lt;TestState&gt;('idle');
  const [micState, setMicState]   = useState&lt;TestState&gt;('idle');
  const [notifState, setNotifState] = useState&lt;TestState&gt;('idle');
  const [platform, setPlatform]   = useState&lt;Platform&gt;('desktop');

  useEffect(() =&gt; {
    const ua = navigator.userAgent;
    if (/Android/i.test(ua))              setPlatform('android');
    else if (/iPhone|iPad|iPod/i.test(ua)) setPlatform('ios');
    else                                   setPlatform('desktop');

    // Vérifier si notifications déjà accordées
    if ('Notification' in window &amp;amp;&amp;amp; Notification.permission === 'granted') {
      setNotifState('ok');
    }
    // Vérifier si micro déjà accordé (navigator.permissions API)
    if (navigator.permissions) {
      navigator.permissions.query({ name: 'microphone' as PermissionName }).then(r =&gt; {
        if (r.state === 'granted') setMicState('ok');
        if (r.state === 'denied')  setMicState('denied');
      }).catch(() =&gt; {});
    }
  }, []);

  // ■■■■ Test TTS ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
  const testTTS = useCallback(async () =&gt; {
    setTtsState('testing');
    try {
      await speak('■■■■■■■■■ ■■■■■■ ■■■■■■■■', 1.0);
      setTimeout(() =&gt; setTtsState('ok'), 1500);
    } catch {
      setTtsState('fail');
    }
  }, [speak]);
```

```
// ■■■ Test Microphone ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const testMic = useCallback(async () =&gt; {
  setMicState('testing');
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    stream.getTracks().forEach(t =&gt; t.stop());
    setMicState('ok');
  } catch (e: any) {
    if (e?.name === 'NotAllowedError') {
      setMicState('denied');
    } else {
      setMicState('fail');
    }
  }
}, []);

// ■■■ Demander notifications ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const requestNotifications = useCallback(async () =&gt; {
  if (!('Notification' in window)) {
    setNotifState('fail');
    return;
  }
  setNotifState('testing');
  try {
    const result = await Notification.requestPermission();
    if (result === 'granted') {
      setNotifState('ok');
      // Envoyer une notification de bienvenue
      new Notification(ar ? '■■■■■ — ■■■■■■■ ■■! ■' : 'Noutq — ■■■■■ ■■■■■■■■! ■', {
        body: ar
          ? '■■■■■■ ■■ ■■■■■■■■ ■■■■■■ ■■■■■■■■■■'
          : '■■■■■ ■■■■■■■■■■■■ ■■■■ ■■■■■■■■■■ ■■■■■■■■■■■■ ■■■■■■■■■■■■■',
        icon: '/icons/icon-192x192.png',
      });
    } else {
      setNotifState('denied');
    }
  } catch {
    setNotifState('fail');
  }
}, [ar]);

// ■■■ Traductions ■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■■
const T = {
  // Étape 0 — Bienvenue / permissions globales
  welcome_title:  ar ? '■■■■■■■ ■■ ■■ ■■■■■!'    : '■■■■■ ■■■■■■■■ Noutq!',
  welcome_sub:    ar ? '■■■ ■■■■■■■ ■■■■■■ ■■■■ ■■■■■■■■■■' : '■■■■■ ■■■■■■■■ ■■ ■■■■■ ■■■■■■■■■■■■■■■■■',
  welcome_desc:   ar
    ? '■■■ ■■■■■■■■ ■■■■■■ ■■■■: ■■■■■■■■■■■■ (■■■■■■■)■ ■■■■■■■■■■■ (■■■■■■■■■■■■)■ ■■■■■■ (■■■■■■■■■■). ■■
    : '■■■■■■■■■■■ ■■■■■■ ■■■■■■ ■■■■■■■■■■ (■■■■■■■■■■■■■■■■), ■■■■■■■■■■■■■■■■ (■■■■■■■■■■■■■■■■), ■■■■■■ (■
  welcome_btn:    ar ? '■■■■■ ■■■■■■■■■'           : '■■■■■■ ■■■■■■■■■■■■■■■',

  title:    ar ? '■■■■■■ ■■■■■'         : '■■■■■■■■■■ ■■■■■■■■■■■■■',
  subtitle: ar ? '■■■■■■ ■■■ ■■■■■■'    : '■■■■■■■ ■■■■■ ■■■■■■■■',
  skip:     ar ? '■■■■■'                : '■■■ ■■■■■■■■',
  next:     ar ? '■■■■■■■■'             : '■■■■■■■■',
  done:     ar ? '■■■■■ ■■■■■■■!'       : '■■■■■■ ■■■■■■■■■!',

  // TTS
  tts_title: ar ? '■■■■■■ ■■■■■■■ (TTS)'  : '■■■■■■■■■■ ■■■■■ (TTS)',
  tts_desc:  ar
    ? '■■■■■■ ■■■■■■■■■ ■■■■■■■ ■■■■■■■■■ ■■■■■■■■■ ■■■■■ ■■■■■. ■■■ ■■■■■■ ■■■■■ ■■■■■■ ■■■■■■■.'
    : '■■■■■■■■■■■ ■■■■■■■■■ ■ ■■■■■■■■■■ ■■■■■■ ■■■■■■■■■■■■■: ■■■■■■■■■■■ ■ ■■■■■■■■■■ ■■■■■■■■■■ ■■■■■■:',
  tts_btn:  ar ? '■ ■■■■■■ ■■■■■■ ■■■■■'            : '■ ■■■■■■■ ■■■■■',
  tts_ok:   ar ? '■■■■■■! ■■■■■■ ■■■■■ ✓'             : '■■■■■■■■■! ■■■■■■ ■■■■■■■■■■ ■ ✓',
  tts_fail: ar ? '■■ ■■■■■■ ■■■■ — ■■■■■ ■■■■■■■■■ ■■■■■■' : '■■■■■ ■■■■ — ■■■■■■■ ■■■■■■ ■■■■■■■ ■■■■■■■■■■

  android_tts: ar ? [
    '① ■■■■■ ■■■■■■■■■ ■■■■■■■■',
    '② ■■■■■ ■■■■: ■■■■■■■■■ ■■■■■■■■ ← ■■■■■■ ■■■■■ ■■■■ ■■■■■',
    '③ ■■■■■ Google Text-to-Speech ■■■■■■',
    '④ ■■■■■ ■■ ← ■■■■■■ ■■■■■■■■ ■■■■■■ ← ■■■■■■■■■',
```

```
    '⑤ ████ ████ ████████',
  ] : [
    '① ██████ ████████████████',
    '② ████████████████ → ████',
    '③ ███████ Google TTS',
    '④ ████████ ████████ ████████ ████████',
    '⑤ ██████████ ■ ██████ ███████',
  ],
  ios_tts: ar ? [
    '① ████ ██████████',
    '② ████ ███: ████████ ███████ ← ████████ ████████',
    '③ ████ ███ "████████" ██ ████ "████████"',
    '④ ████ ██ ███ ████',
    '⑤ ████ ████ ████████',
  ] : [
    '① ██████ ████████████████',
    '② ████████████████ → ████████',
    '③ ███████ → ████████',
    '④ ██████████ ████',
    '⑤ ██████████ ■ ██████ ███████',
  ],
  desktop_tts: ar ? [
    '① ████ ██ ███ ██████ ██████ ██ ██████',
    '② ████ ████████ ███ (Chrome ██ Firefox)',
    '③ ████ ██ ███████ ██████ ██ ████████ ████████',
  ] : [
    '① ████████ ███ ██████ ██████ ██████████',
    '② ███████ ███ ██████████ (Chrome ███ Firefox)',
    '③ ████████ ██████ ████████████████████ ████████ ████████████████████',
  ],

  // Microphone
  mic_title: ar ? '██████████ (██████)'       : '████████ (███████████████ ██████)',
  mic_desc:  ar
    ? '███████ ██████ ██████ ██ ███ ██████. ████████ ██████ ███ ████████ ██████████.'
    : '████████████████ ██████████████████ ██████████ ██ ███ ██████: ██████████ ██████ ████ ██████████ ██
  mic_btn:    ar ? '██ ████ ███ ██████████'  : '██ ██████ ███',
  mic_ok:     ar ? '██████████ ████ ✓'         : '██████████ ████████ ■ ✓',
  mic_fail:   ar ? '██████ ███████ — ████ ██ ████████ ██████' : '██████████████████ ███████ ■ — ████████
  mic_denied: ar ? '██ ███ ██████ — ██████ ██ ██████████ ██████████' : '████████ — ████████████████ ██

  // Notifications
  notif_title: ar ? '██████████'               : '██████████████',
  notif_desc:  ar
    ? '████ ██ ████████ ██████ ██████████ ██████████ ████████ ██████ ███████. ██████ ████████ ██ ██ ███
    : '████ ██████████ ███ ██████████ ████████████ ████████████ ■ ████████████████ ██████████ ██████ ██
  notif_btn:   ar ? '■ ████ ██████████'         : '■ ████████████ ████████████████',
  notif_ok:    ar ? '██████████ ███████ ✓'         : '████████████████ ██████ ███ ✓',
  notif_fail:  ar ? '██████████ ███ ███████ ██ ███ ████████' : '████████████████ ███ ██████████ ███ ██
  notif_denied: ar ? '██ ███ ██████ — ██████ ██ ████████ ████████' : '████████ — ████████████ ██████████
  notif_skip:  ar ? '████ (██ ██████ ████████)' : '███ ███████ (████████████ ███ ████)',

  // Fin
  done_title: ar ? '███ ████! ■'   : '████████ ██! ■',
  done_desc:  ar
    ? '██████ ███████ ██████ ███ ██████████ ███████ ██ ████████ ██████ ██ ████████.'
    : '████ ██████ ██ ████ ██████ ███ ████████████████ ███ ██████████ ███ ██████████ ████████████████
};

// ███ Icône de statut ████████████████████████████████████████████████████████████
const StateIcon = ({ state }: { state: TestState }) =&gt; {
  if (state === 'ok')      return &lt;CheckCircle className="text-green-500 shrink-0" size={22} /&gt;;
  if (state === 'fail')    return &lt;XCircle className="text-red-400 shrink-0" size={22} /&gt;;
  if (state === 'denied')  return &lt;XCircle className="text-orange-400 shrink-0" size={22} /&gt;;
  if (state === 'testing') return (
    &lt;span className="animate-spin inline-block w-5 h-5 border-2 border-blue-400 border-t-transparent rounded
  );
  return &lt;div className="w-5 h-0.5 bg-gray-300 rounded shrink-0" /&gt;;
};

// ███ Étapes ████████████████████████████████████████████████████████████████████
const Steps: { id: StepId; icon: React.ReactNode; label: string }[] = [
```

```
  { id: 'synthesis',    icon: &lt;Volume2 size={14} /&gt;,      label: 'TTS' },
  { id: 'recognition',  icon: &lt;Mic size={14} /&gt;,          label: ar ? '[redacted]' : 'Mic' },
  { id: 'notifications',icon: &lt;Bell size={14} /&gt;,         label: ar ? '[redacted]' : 'Notif' },
  { id: 'done',         icon: &lt;CheckCircle size={14} /&gt;,  label: ar ? '[redacted]' : '[redacted]' },
];
const visibleSteps = Steps.filter(s =&gt; s.id !== 'welcome');
const currentIdx = visibleSteps.findIndex(s =&gt; s.id === step);

const NavChevron = () =&gt; ar
  ? &lt;ChevronLeft size={18} /&gt;
  : &lt;ChevronRight size={18} /&gt;;

const ttsGuide =
  platform === 'ios'     ? T.ios_tts :
  platform === 'android' ? T.android_tts :
  T.desktop_tts;

const platformLabel =
  platform === 'ios'     ? 'iOS' :
  platform === 'android' ? 'Android' :
  ar ? '[redacted]' : 'Desktop';

// [redacted] Écran de bienvenue [redacted]
if (step === 'welcome') {
  return (
    &lt;div className="flex flex-col h-full bg-white" dir={ar ? 'rtl' : 'ltr'}&gt;
      &lt;div className="flex-1 flex flex-col items-center justify-center px-6 text-center gap-6"&gt;
        &lt;div className="w-24 h-24 bg-gradient-to-br from-blue-500 to-indigo-600 rounded-3xl flex items-cente
          &lt;Shield size={44} className="text-white" /&gt;
        &lt;/div&gt;

        &lt;div&gt;
          &lt;h1 className="text-2xl font-bold text-gray-800 mb-2"&gt;{T.welcome_title}&lt;/h1&gt;
          &lt;p className="text-blue-600 font-medium text-sm"&gt;{T.welcome_sub}&lt;/p&gt;
        &lt;/div&gt;

        &lt;p className="text-gray-600 text-sm leading-relaxed max-w-xs"&gt;{T.welcome_desc}&lt;/p&gt;

        {/* Résumé des permissions demandées */}
        &lt;div className="w-full bg-blue-50 rounded-2xl p-4 space-y-3"&gt;
          {[
            { icon: &lt;Volume2 size={18} className="text-blue-500" /&gt;, label: ar ? '[redacted] — [redacted]
            { icon: &lt;Mic size={18} className="text-purple-500" /&gt;,   label: ar ? '[redacted] — [redacted]
            { icon: &lt;Bell size={18} className="text-amber-500" /&gt;,   label: ar ? '[redacted] — [redacted]
          ].map((item, i) =&gt; (
            &lt;div key={i} className="flex items-center gap-3"&gt;
              {item.icon}
              &lt;span className="text-sm text-gray-700"&gt;{item.label}&lt;/span&gt;
            &lt;/div&gt;
          ))}
        &lt;/div&gt;
      &lt;/div&gt;

      &lt;div className="px-5 pb-8 pt-3"&gt;
        &lt;button
          onClick={() =&gt; setStep('synthesis')}
          className="w-full py-4 bg-gradient-to-r from-blue-600 to-indigo-600 text-white
                     rounded-2xl font-bold text-base flex items-center justify-center gap-2
                     active:scale-95 transition-transform shadow-lg"
        &gt;
          &lt;span&gt;{T.welcome_btn}&lt;/span&gt;
          &lt;NavChevron /&gt;
        &lt;/button&gt;
      &lt;/div&gt;
    &lt;/div&gt;
  );
}

// [redacted] Écrans d'étapes [redacted]
return (
  &lt;div className="flex flex-col h-full bg-white" dir={ar ? 'rtl' : 'ltr'}&gt;
```

```
{/* En-tête avec progression */}
&lt;div className="bg-gradient-to-br from-blue-600 to-indigo-700 px-5 pt-12 pb-8 text-white text-center"&gt
  &lt;div className="w-16 h-16 bg-white/20 rounded-2xl flex items-center justify-center mx-auto mb-4"&gt;
    &lt;Smartphone size={32} className="text-white" /&gt;
  &lt;/div&gt;
  &lt;h1 className="text-2xl font-bold"&gt;{T.title}&lt;/h1&gt;
  &lt;p className="text-blue-100 text-sm mt-1"&gt;{T.subtitle}&lt;/p&gt;

  &lt;div className="flex items-center justify-center gap-2 mt-5"&gt;
    {visibleSteps.map((s, i) =&gt; (
      &lt;div
        key={s.id}
        className={`flex items-center gap-1 px-3 py-1 rounded-full text-xs font-medium transition-all
          ${s.id === step        ? 'bg-white text-blue-700'
          : i &lt; currentIdx      ? 'bg-white/40 text-white'
          :                         'bg-white/10 text-white/50'}`}
      &gt;
        {s.icon}
        &lt;span&gt;{s.label}&lt;/span&gt;
      &lt;/div&gt;
    ))}
  &lt;/div&gt;
&lt;/div&gt;

{/* Contenu */}
&lt;div className="flex-1 overflow-y-auto px-5 py-6 space-y-5"&gt;

  {/* ■■ Étape 1 : TTS ■■ */}
  {step === 'synthesis' &amp;&amp; (
    &lt;&gt;
      &lt;div className="bg-blue-50 rounded-2xl p-5"&gt;
        &lt;div className="flex items-center gap-3 mb-3"&gt;
          &lt;div className="w-10 h-10 bg-blue-100 rounded-xl flex items-center justify-center shrink-0"&gt
            &lt;Volume2 className="text-blue-600" size={20} /&gt;
          &lt;/div&gt;
          &lt;h2 className="font-bold text-gray-800"&gt;{T.tts_title}&lt;/h2&gt;
        &lt;/div&gt;
        &lt;p className="text-gray-600 text-sm leading-relaxed"&gt;{T.tts_desc}&lt;/p&gt;
      &lt;/div&gt;

      &lt;button
        onClick={testTTS}
        disabled={ttsState === 'testing'}
        className="w-full py-4 bg-blue-600 text-white rounded-2xl font-bold text-base
                   flex items-center justify-center gap-2
                   active:scale-95 transition-transform disabled:opacity-60"
      &gt;
        {ttsState === 'testing'
          ? &lt;span className="animate-spin w-5 h-5 border-2 border-white border-t-transparent rounded-ful
          : T.tts_btn}
      &lt;/button&gt;

      {(ttsState === 'ok' || ttsState === 'fail') &amp;&amp; (
        &lt;div className={`flex items-center gap-3 p-4 rounded-2xl
          ${ttsState === 'ok' ? 'bg-green-50' : 'bg-red-50'}`}&gt;
          &lt;StateIcon state={ttsState} /&gt;
          &lt;p className={`text-sm font-medium ${ttsState === 'ok' ? 'text-green-700' : 'text-red-600'}`}&
            {ttsState === 'ok' ? T.tts_ok : T.tts_fail}
          &lt;/p&gt;
        &lt;/div&gt;
      )}

      {ttsState === 'fail' &amp;&amp; (
        &lt;div className="bg-amber-50 border border-amber-200 rounded-2xl p-4"&gt;
          &lt;div className="flex items-center gap-2 mb-3"&gt;
            &lt;Settings size={16} className="text-amber-600 shrink-0" /&gt;
            &lt;span className="font-bold text-amber-800 text-sm"&gt;
              {platformLabel} — {ar ? '■■■■■ ■■■■■■■' : '■■■■■■■■■■ ■■■■■■'}
            &lt;/span&gt;
          &lt;/div&gt;
          &lt;ol className="space-y-2"&gt;
            {ttsGuide.map((line, i) =&gt; (
```

```
            &lt;li key={i} className="text-sm text-amber-900 leading-relaxed"&gt;{line}&lt;/li&gt;
          ))}
        &lt;/ol&gt;
      &lt;/div&gt;
    )}
  &lt;/&gt;
)}

{/* ■■ Étape 2 : Microphone ■■ */}
{step === 'recognition' &amp;&amp; (
  &lt;&gt;
    &lt;div className="bg-purple-50 rounded-2xl p-5"&gt;
      &lt;div className="flex items-center gap-3 mb-3"&gt;
        &lt;div className="w-10 h-10 bg-purple-100 rounded-xl flex items-center justify-center shrink-0"&
          &lt;Mic className="text-purple-600" size={20} /&gt;
        &lt;/div&gt;
        &lt;h2 className="font-bold text-gray-800"&gt;{T.mic_title}&lt;/h2&gt;
      &lt;/div&gt;
      &lt;p className="text-gray-600 text-sm leading-relaxed"&gt;{T.mic_desc}&lt;/p&gt;
    &lt;/div&gt;

    &lt;button
      onClick={testMic}
      disabled={micState === 'testing' || micState === 'ok'}
      className="w-full py-4 bg-purple-600 text-white rounded-2xl font-bold text-base
                 flex items-center justify-center gap-2
                 active:scale-95 transition-transform disabled:opacity-60"
    &gt;
      {micState === 'testing'
        ? &lt;span className="animate-spin w-5 h-5 border-2 border-white border-t-transparent rounded-ful
        : T.mic_btn}
    &lt;/button&gt;

    {micState === 'ok' &amp;&amp; (
      &lt;div className="flex items-center gap-3 p-4 rounded-2xl bg-green-50"&gt;
        &lt;StateIcon state="ok" /&gt;
        &lt;p className="text-sm font-medium text-green-700"&gt;{T.mic_ok}&lt;/p&gt;
      &lt;/div&gt;
    )}
    {(micState === 'fail' || micState === 'denied') &amp;&amp; (
      &lt;div className="flex items-center gap-3 p-4 rounded-2xl bg-orange-50"&gt;
        &lt;StateIcon state={micState} /&gt;
        &lt;p className="text-sm font-medium text-orange-700"&gt;
          {micState === 'denied' ? T.mic_denied : T.mic_fail}
        &lt;/p&gt;
      &lt;/div&gt;
    )}
  &lt;/&gt;
)}

{/* ■■ Étape 3 : Notifications ■■ */}
{step === 'notifications' &amp;&amp; (
  &lt;&gt;
    &lt;div className="bg-amber-50 rounded-2xl p-5"&gt;
      &lt;div className="flex items-center gap-3 mb-3"&gt;
        &lt;div className="w-10 h-10 bg-amber-100 rounded-xl flex items-center justify-center shrink-0"&g
          &lt;Bell className="text-amber-600" size={20} /&gt;
        &lt;/div&gt;
        &lt;h2 className="font-bold text-gray-800"&gt;{T.notif_title}&lt;/h2&gt;
      &lt;/div&gt;
      &lt;p className="text-gray-600 text-sm leading-relaxed"&gt;{T.notif_desc}&lt;/p&gt;
    &lt;/div&gt;

    {notifState !== 'ok' &amp;&amp; (
      &lt;button
        onClick={requestNotifications}
        disabled={notifState === 'testing'}
        className="w-full py-4 bg-amber-500 text-white rounded-2xl font-bold text-base
                   flex items-center justify-center gap-2
                   active:scale-95 transition-transform disabled:opacity-60"
      &gt;
        {notifState === 'testing'
```

```
            ? &lt;span className="animate-spin w-5 h-5 border-2 border-white border-t-transparent rounded-f
            : T.notif_btn}
        &lt;/button&gt;
      )}

      {notifState === 'ok' &amp;&amp; (
        &lt;div className="flex items-center gap-3 p-4 rounded-2xl bg-green-50"&gt;
          &lt;StateIcon state="ok" /&gt;
          &lt;p className="text-sm font-medium text-green-700"&gt;{T.notif_ok}&lt;/p&gt;
        &lt;/div&gt;
      )}
      {(notifState === 'fail' || notifState === 'denied') &amp;&amp; (
        &lt;div className="flex items-center gap-3 p-4 rounded-2xl bg-orange-50"&gt;
          &lt;StateIcon state={notifState} /&gt;
          &lt;p className="text-sm font-medium text-orange-700"&gt;
            {notifState === 'denied' ? T.notif_denied : T.notif_fail}
          &lt;/p&gt;
        &lt;/div&gt;
      )}
    &lt;/&gt;
  )}

  {/* ■■ Étape finale : Résumé ■■ */}
  {step === 'done' &amp;&amp; (
    &lt;div className="flex flex-col items-center text-center py-8 gap-5"&gt;
      &lt;div className="w-24 h-24 bg-green-100 rounded-full flex items-center justify-center"&gt;
        &lt;CheckCircle className="text-green-500" size={48} /&gt;
      &lt;/div&gt;
      &lt;h2 className="text-2xl font-bold text-gray-800"&gt;{T.done_title}&lt;/h2&gt;
      &lt;p className="text-gray-500 text-sm leading-relaxed max-w-xs"&gt;{T.done_desc}&lt;/p&gt;

      &lt;div className="w-full bg-gray-50 rounded-2xl p-4 space-y-3"&gt;
        {[
          { icon: &lt;Volume2 size={16} /&gt;, label: T.tts_title,   state: ttsState },
          { icon: &lt;Mic size={16} /&gt;,     label: T.mic_title,   state: micState },
          { icon: &lt;Bell size={16} /&gt;,    label: T.notif_title, state: notifState },
        ].map((item, i) =&gt; (
          &lt;div key={i} className="flex items-center justify-between"&gt;
            &lt;div className="flex items-center gap-2 text-sm text-gray-600"&gt;
              {item.icon}&lt;span&gt;{item.label}&lt;/span&gt;
            &lt;/div&gt;
            &lt;StateIcon state={item.state} /&gt;
          &lt;/div&gt;
        ))}
      &lt;/div&gt;
    &lt;/div&gt;
  )}
&lt;/div&gt;

{/* Pied de page — boutons de navigation */}
&lt;div className="px-5 pb-8 pt-3 border-t border-gray-100 space-y-3"&gt;
  {step === 'synthesis' &amp;&amp; (
    &lt;div className="space-y-2"&gt;
      &lt;button
        onClick={() =&gt; setStep('recognition')}
        disabled={ttsState === 'testing'}
        className={`w-full py-4 rounded-2xl font-bold text-base flex items-center justify-center gap-2 tran
          ttsState === 'testing'
            ? 'bg-gray-200 text-gray-400 cursor-not-allowed'
            : 'bg-gray-800 text-white active:scale-95'
        }`}
      &gt;
        &lt;span&gt;{T.next}&lt;/span&gt;&lt;NavChevron /&gt;
      &lt;/button&gt;
      {ttsState === 'fail' &amp;&amp; (
        &lt;p className="text-center text-xs text-amber-600 font-medium px-4"&gt;
          {ar
            ? '■■■■■■ ■■■■■■■■■■ ■■■■ ■■■■■■■■ ■■■■■■ ■■■■ ■■■■■■■ ■■■■■ ■■■■■■ ■■■■■■.'
            : '■■■■■■ ■■ ■■■■■■■■■■■, ■■■■■ ■■■■■■ ■■■■■■■■■■■ ■■ ■■■■■:'}
        &lt;/p&gt;
      )}
    &lt;/div&gt;
```

```
        )}

        {step === 'recognition' &amp;&amp; (
          &lt;div className="space-y-2"&gt;
            &lt;button
              onClick={() =&gt; setStep('notifications')}
              disabled={micState === 'testing'}
              className={`w-full py-4 rounded-2xl font-bold text-base flex items-center justify-center gap-2 tran
                micState === 'testing'
                  ? 'bg-gray-200 text-gray-400 cursor-not-allowed'
                  : 'bg-gray-800 text-white active:scale-95'
              }`}
            &gt;
              &lt;span&gt;{T.next}&lt;/span&gt;&lt;NavChevron /&gt;
            &lt;/button&gt;
            {(micState === 'fail' || micState === 'denied') &amp;&amp; (
              &lt;p className="text-center text-xs text-amber-600 font-medium px-4"&gt;
                {ar
                  ? '██████ ██████████ ███ ███████ ██████ ██ █████ ██████.'
                  : '██████ ██ ███████████, ████ ███████████████ ████████████████ ████████ ███:'}
              &lt;/p&gt;
            )}
          &lt;/div&gt;
        )}

        {step === 'notifications' &amp;&amp; (
          &lt;div className="space-y-2"&gt;
            &lt;button
              onClick={() =&gt; setStep('done')}
              disabled={notifState === 'testing'}
              className={`w-full py-4 rounded-2xl font-bold text-base flex items-center justify-center gap-2 tran
                notifState === 'testing'
                  ? 'bg-gray-200 text-gray-400 cursor-not-allowed'
                  : 'bg-gray-800 text-white active:scale-95'
              }`}
            &gt;
              &lt;span&gt;{T.next}&lt;/span&gt;&lt;NavChevron /&gt;
            &lt;/button&gt;
            {notifState !== 'ok' &amp;&amp; notifState !== 'testing' &amp;&amp; (
              &lt;button
                onClick={() =&gt; setStep('done')}
                className="w-full py-3 text-gray-400 text-sm font-medium active:scale-95 transition-transform"
              &gt;
                {T.notif_skip}
              &lt;/button&gt;
            )}
          &lt;/div&gt;
        )}

        {step === 'done' &amp;&amp; (
          &lt;button
            onClick={onDone}
            className="w-full py-4 bg-gradient-to-r from-blue-600 to-indigo-600 text-white
                       rounded-2xl font-bold text-lg flex items-center justify-center gap-2
                       active:scale-95 transition-transform shadow-lg"
          &gt;
            &lt;span&gt;{T.done}&lt;/span&gt;&lt;NavChevron /&gt;
          &lt;/button&gt;
        )}
      &lt;/div&gt;
    &lt;/div&gt;
  );
}
```

■ FICHIER : test_server_tts.ts

```
async function test() {
  const res = await fetch('http://localhost:3000/api/tts?text=test&amp;lang=ar');
  const text = await res.text();
```

```
  console.log("STATUS:", res.status);
  console.log("BODY:", text.substring(0, 100));
}
test();
```

■ FICHIER : test_tts.ts

```
import { GoogleGenAI, Modality } from "@google/genai";
import dotenv from "dotenv";
dotenv.config();

async function run() {
  const ai = new GoogleGenAI({ apiKey: process.env.GEMINI_API_KEY });
  const res = await ai.models.generateContent({
    model: "gemini-3.1-flash-tts-preview",
    contents: "■■■■■■■",
    config: {
      responseModalities: [Modality.AUDIO],
      speechConfig: { voiceConfig: { prebuiltVoiceConfig: { voiceName: "Kore" } } }
    }
  });
  console.log(res.candidates?.[0]?.content?.parts?.[0]?.inlineData?.data?.substring(0, 100));
}
run();
```

■ FICHIER : tsconfig.json

```
{
  "compilerOptions": {
    "target": "ES2022",
    "experimentalDecorators": true,
    "useDefineForClassFields": false,
    "module": "ESNext",
    "lib": [
      "ES2022",
      "DOM",
      "DOM.Iterable"
    ],
    "skipLibCheck": true,
    "moduleResolution": "bundler",
    "isolatedModules": true,
    "moduleDetection": "force",
    "allowJs": true,
    "jsx": "react-jsx",
    "paths": {
      "@/*": [
        "./*"
      ]
    },
    "allowImportingTsExtensions": true,
    "noEmit": true
  }
}
```

■ FICHIER : vercel.json

```
{
  "version": 2,
  "rewrites": [
    {
      "source": "/api/(.*)",
      "destination": "/api/index.ts"
    },
    {
```

```
      "source": "/(.*)",
      "destination": "/index.html"
    }
  ]
}
```

■ FICHIER : vite.config.ts

```
import tailwindcss from '@tailwindcss/vite';
import react from '@vitejs/plugin-react';
import path from 'path';
import {defineConfig, loadEnv} from 'vite';

export default defineConfig(({mode}) =&gt; {
  const env = loadEnv(mode, '.', '');
  return {
    plugins: [react(), tailwindcss()],
    resolve: {
      alias: {
        '@': path.resolve(__dirname, '.'),
      },
    },
    build: {
      rollupOptions: {
        output: {
          manualChunks: {
            vendor: ['react', 'react-dom'],
            ai: ['@google/genai'],
          }
        }
      }
    },
    server: {
      // HMR is disabled in AI Studio via DISABLE_HMR env var.
      // Do not modifyâ■■file watching is disabled to prevent flickering during agent edits.
      hmr: env.DISABLE_HMR !== 'true',
    },
  };
});
```